

Beyond Pattern Recognition: A New Paradigm Uniting Symbolism and Connectionism

Chao Zheng | Independent Researcher

Abstract

This study challenges the common impression that neural networks are merely statistical pattern recognizers. It proposes and empirically validates a training paradigm that uses programmatically generated ideal data to elicit their capacity for precise rule learning, thereby turning standard neural networks from probabilistic imitators into deterministic rule executors under controlled conditions. I refer to this training setup as the Neural Sculpting Paradigm: the task is generated by an executable program, the input and output spaces are fixed in advance, samples are randomly drawn from the target distribution, outputs are produced through one-shot structured prediction, and the training objective is to learn the transformation from each input to its uniquely determined output. Across symbolic rules, algorithm fitting, cellular automata, structured image transformations, and other tasks, the experiments show that standard neural networks can learn near-exact rule mappings under controlled conditions, strongly supporting the generality of this paradigm. Semantic-shuffling and position-shuffling experiments further show that the learned behavior cannot be simply reduced to memorization of surface symbol meanings or fixed positional templates. Architectural comparison experiments indicate that this capability is not unique to Transformers: MLPs, CNNs, RNNs, UNets, and diffusion models also exhibit similar abilities on some tasks, suggesting that precise rule learning is likely not an accidental property of a particular architecture, but a more general intrinsic property of neural networks as connectionist systems. Further cellular-automaton interpolation experiments and MNIST+cellular-automaton experiments show that pattern recognition and rule execution can be naturally coupled within the same model, strongly suggesting that they may not be separate abilities, but rather endogenous capacities of neural networks, and perhaps even manifestations of the same underlying ability in different data forms. Based on these results, this paper proposes a compression-as-intelligence / MDL-style explanation: the learning process of neural networks can be understood as a process of approaching representations of minimal Kolmogorov complexity, namely approximating the shortest underlying generative rule from a large number of samples. Through a series of controlled experiments, this paper systematically demonstrates, for the first time, the intrinsic potential of standard neural networks to execute algorithms precisely, reveals the essential capabilities that neural networks can display under ideal conditions, and provides experimental clues for understanding the relationship among intelligence, compression, and rule learning. It also opens a new path toward highly reliable and interpretable general AI systems. Code is available at: <https://github.com/ball-lightning6/neural-sculpting-paradigm>.

Contents

1	Act I: The Reasoning Gap in Modern AI	5
2	Act II: A Brief History of Neuro-Symbolic Computing	5
3	Act III: The Neural Sculpting Paradigm	6
3.1	Model Architecture: Transformer as a Starting Point	7
3.2	Output Paradigm: One-Shot Structured Prediction	7
3.3	Data Paradigm: From “Realistic Noise” to “Ideal Laws”	7
3.4	Paradigm Interpretation: Sculpting Precise Rules with Gradient Descent	7
3.5	Model Description	8
4	Act IV: Empirical Evidence Across Multiple Domains	8
4.1	Fundamental Capability: Symbolic Rule Learning	8
4.1.1	Cellular Automata	9
4.1.2	Inquiry into the Nature of Learning: Base-N Addition and Semantic Shuffling	9
4.2	Advanced Capability: Algorithm Learning and Planning	12
4.2.1	Case Study: Trapping Rain Water Problem	12
4.2.2	Dense Maze Navigation	13
4.3	Reasoning and Extracting Symbolic Output from Images	14
4.3.1	Visual Symbol Decoding: Dial Angle Identification	15
4.4	Image Reasoning Capability	16
4.4.1	Geometric Reasoning and Construction: Prediction of Triangle Incircle	16
4.4.2	Geometric Reasoning and Construction: Plane Tessellation	20
4.5	Image Generation via Symbolic Reasoning	26
4.5.1	Cellular Automata	26
4.5.2	Rotating Cube	30
4.6	Physical Simulation	34
4.6.1	Catenary	34
4.6.2	Planetary Orbit Simulation	39
4.7	ARC-AGI Task Related	44
4.7.1	Fluid Simulation	45
4.7.2	Block Puzzle	50
4.8	Interpretability Exploration	56
4.8.1	Edit Process Output	56
5	Act V: On the Boundaries of Learnability and Interpretability	57
6	Act VI: Universality Beyond Architecture	64
6.1	Cellular Automata	64
6.1.1	CNN Model	64
6.1.2	MLP Model	65
6.1.3	RNN Model	65
6.1.4	UNet Model	65
6.1.5	Diffusion Model	65
6.2	Trapping Rain Water	66
6.2.1	MLP Model	66
6.2.2	RNN Model	66
6.2.3	CNN Model	66
6.3	Summary and Analysis	66

7	Act VII: Bridging the Gap: The Continuum from Ideal Rules to the Real World	66
7.1	Perturbation of Precise Rules	66
7.1.1	Random Perturbation Test on Output	67
7.1.2	Random Perturbation Test on Input	67
7.1.3	Summary and Discussion	67
7.2	The Unity of Interpolation and Rule Learning	67
7.2.1	Experiment Description	67
7.2.2	Experimental Results	68
7.2.3	Summary and Discussion	68
7.3	The Unity of Pattern Recognition and Rule Learning	69
7.3.1	MNIST + Cellular Automata: From Pattern Recognition to Rule Execution . . .	69
7.3.2	MNIST + Cellular Automata: Rule Control and Image Transformation	70
8	Act VIII: Assisting Training with Decoupling — Rethinking the Relationship Between Symbolism and Connectionism	72
8.1	Binary Number Mod 3 Experiment	73
8.1.1	Experimental Part	73
8.1.2	Discussion of Experimental Results	73
8.2	Another Decoupling of the Trapping Rain Water Experiment	74
8.2.1	Experimental Part	74
8.2.2	Discussion of Experimental Results	75
8.3	Discussion on Related Philosophical Meanings	75
9	Act IX: Research on Already Converged Neural Networks	76
9.1	Studying the Last Hidden Layer via Trapping Rain Water Experiment	77
9.1.1	Experimental Part	77
9.1.2	Discussion of Experimental Results	77
9.2	Probing the Hidden Layers of a Converged MLP with Cellular Automata Task	77
9.2.1	Experimental Part	77
9.2.2	Discussion of Experimental Results	78
9.3	Decoding the Complete Evolution Sequence of Cellular Automata from the Last Hidden Layer	78
9.3.1	Experimental Part	78
9.3.2	Discussion of Experimental Results	78
10	Act X: Some Further Explorations	79
10.1	Training Visualization	79
10.2	Autoregression	79
11	Act XI: Theoretical Interpretation	79
11.1	Theoretical Assumptions and Evidence	80
11.2	Relevant Experimental Evidence	82
11.2.1	[27] NTK Related Experiments	82
11.2.2	Rule Preference Experiment	82
11.2.3	Extreme Over-parameterization Experiment	83
11.2.4	Rule OOD Generalization Experiment	83
11.3	Some Additional Thoughts	83
11.4	Related Work	85
12	Act XII: From Alchemy to Science in Deep Learning	85
	References	88

Appendix	90
A. Some Interesting Experimental Images	90
B. List of All Experiments Conducted	93
B.1 Symbolic Math Logic	93
B.2 Algorithm Learning	96
B.3 Visual Reasoning	99
B.4 Cellular Automata	99
B.5 Physics Simulation	100
B.6 ARC-AGI Exploration	101

1 Act I: The Reasoning Gap in Modern AI

Deep neural networks have achieved enormous success in image recognition, speech recognition, natural language processing, and generative AI, but controversy over their essential capabilities has never disappeared. On one hand, neural networks are often understood as high-dimensional statistical fitters, pattern recognizers, or complex function approximators: they can capture correlations from massive data, but may not truly understand rules, algorithms, or causal structures. Hallucinations in large language models, logical inconsistency, fragility in mathematical and program reasoning, and adversarial examples have further strengthened this suspicion. In sharper terms, neural networks are sometimes regarded as “stochastic parrots”: they appear to reason, but in fact only perform probabilistic imitation over complex distributions.

On the other hand, the actual behavior of neural networks continually exceeds this simple picture. Emergent abilities in large models, grokking, implicit regularization, and ideas such as “compression as intelligence” all suggest that neural networks may not merely memorize samples or fit surface patterns, but may under certain conditions tend to discover simpler, more stable, and more generalizable structures. The problem is that real-world data and open-ended tasks usually mix noise, ambiguity, memorization, semantic association, distribution shift, and evaluation uncertainty at the same time. It is therefore difficult to judge what neural networks are actually doing solely from the successes or failures of large models.

This paper argues that the truly fundamental question is not whether neural networks have already solved real-world reasoning reliably, but a more basic one: under sufficiently clear, verifiable, and controlled conditions, do neural networks possess an endogenous ability to learn and execute precise rules? If such an ability exists, does it belong only to certain special architectures, or is it a more general property of neural networks? What is its relationship to pattern recognition in the usual sense? Can it be understood as a form of compression?

Accordingly, this paper does not primarily treat the Neural Sculpting Paradigm as an engineering method for directly improving existing AI systems. Instead, it treats it as a controlled experimental field for exploring the essential properties of neural networks. In this field, tasks are generated by programs, input and output spaces are fixed in advance, samples are randomly drawn from the target distribution, outputs are produced through one-shot structured prediction, and the training objective is to learn the transformation from each input to its uniquely determined output. By deliberately excluding the noise, ambiguity, and open boundaries of real-world data, this paper examines whether standard neural networks can learn and execute precise rules under ideal conditions, and how this capability relates to architecture, pattern recognition, and compression.

The central finding is that, in this controlled setting, standard neural networks can learn near-exact rule mappings across multiple classes of deterministic tasks, and this ability is likely not an accidental property of a specific architecture. Further cellular-automaton interpolation experiments and MNIST+cellular-automaton experiments suggest that pattern recognition and rule execution may not be separate abilities, but may instead be manifestations of the same underlying ability in different data forms. Finally, this paper explains these phenomena from a compression-as-intelligence / MDL perspective: in programmatically generated ideal data, the learning process of neural networks can be understood as approximating shorter and more stable underlying generative rules from a large number of samples. The goal of this paper is not to claim that all reasoning problems in real AI systems have been solved, but to use a series of reproducible experiments to re-examine the essential capabilities that neural networks may possess as connectionist systems.

2 Act II: A Brief History of Neuro-Symbolic Computing

Combining the learning ability of neural networks with the precise reasoning ability of symbolic systems has long been a major goal of artificial intelligence research. Existing explorations around this goal can be roughly divided into two categories.

The first category is architectural modification. This line of work attempts to enhance the reasoning ability of models by designing network structures that are more suitable for symbolic computation or structured reasoning. Representative explorations include Neural Turing Machines and Differentiable Neural Computers with external memory modules, as well as later neural module networks and graph neural networks. Their common feature is an attempt to make neural networks more suitable for compositional relations, state transitions, and algorithmic processes through stronger structural bias.

The second category is logic embedding. This line of work attempts to introduce external symbolic rules, logical constraints, or probabilistic logic programs into the training process of neural networks in differentiable or learnable forms. Whether by converting rules into loss constraints through fuzzy logic, as in neural theorem proving, or by using neural networks as part of probabilistic logic programs, as in DeepProbLog, the core idea is to use some external symbolic system to guide, constrain, or supplement neural networks.

These routes are important, but they often imply a premise: the learning ability of neural networks and the reasoning ability of symbolic systems are two abilities that need to be connected through specialized structures or external rules. This paper focuses on another possibility: without attaching an external symbolic reasoner, without pre-embedding handcrafted logical rules, and without relying on specialized reasoning architectures, can standard neural networks reveal a capacity for precise rule learning under ideal data conditions?

From this perspective, the “union of symbolism and connectionism” in the title does not mean mechanically attaching an external symbolic system to a neural network, nor does it mean replacing neural learning with handcrafted rules. What this paper attempts to show is a more endogenous form of union: symbolic rules are defined by programmatically generated ideal data, and connectionist neural networks naturally approximate these rules through gradient optimization under the constraints of large numbers of deterministic samples. In other words, symbolism is not externally grafted onto neural networks; it gradually emerges from within the connectionist system under ideal data and training dynamics.

Furthermore, this union does not only mean that neural networks can learn symbolic rules. In the traditional narrative, symbolism is closer to rules, logic, and algorithms, while connectionism is closer to perception, pattern recognition, and continuous representation. However, the cellular-automaton interpolation experiments and MNIST+cellular-automaton experiments later in this paper show that precise rule execution and pattern recognition can be naturally coupled within the same neural network, the same task, and the same training process. This suggests that the two may not be separate mechanisms that need external splicing, but may be the same underlying ability revealed by connectionist systems under different data forms.

The following sections examine this question through the Neural Sculpting Paradigm and a series of experiments, and later propose a unified hypothesis for explaining these experimental phenomena and discuss its relationship to previous experimental observations and theoretical ideas.

3 Act III: The Neural Sculpting Paradigm

The core idea of the Neural Sculpting Paradigm proposed in this paper is not to design a brand-new model architecture, but to observe and demonstrate the ability of standard neural networks to learn precise rules under controlled conditions through programmatically generated ideal data and suitable input-output formats. The forms of these tasks are not limited to symbolic rules, algorithmic problems, image reasoning, physical simulation, or interpretability labels. In one sentence, they can all be understood as problems with explicit transformation relations: for a given input, there exists a mechanically executable generation or transformation process that produces a uniquely determined and verifiable output. Terms such as “symbolic task,” “rule learning,” and “image reasoning” are used in this paper mainly to facilitate discussion of these experimental phenomena, rather than to provide a strictly closed mathematical definition.

This paradigm is roughly built on the following mutually supporting principles.

3.1 Model Architecture: Transformer as a Starting Point

The early experiments in this paper mainly used the standard Transformer and its visual variants, such as ViT and Swin Transformer, because they are easy to adapt to symbolic, image, and multimodal tasks. However, later experiments show that MLPs, CNNs, RNNs, UNets, diffusion models, and other architectures can also display similar abilities on some tasks, and some tasks even converge faster with simpler architectures. Therefore, Transformer has no special ontological status in this paper. It was merely the main experimental platform at the beginning of the exploration. In the later stage of the exploration, I had already switched completely to a plain MLP for many experiments. What this paper truly focuses on is whether standard neural networks generally possess the potential to learn precise rules under ideal data conditions.

3.2 Output Paradigm: One-Shot Structured Prediction

Most experiments in this paper use one-shot structured output: the model directly predicts the complete result in a single forward pass. The output may be a fixed-length vector, multi-label binary classification, multi-class classification, a structured image, or a concatenated result with intermediate-process labels. I tend to think that autoregressive generation may increase the difficulty of learning precise rules because it introduces additional variables such as different prefix states and error accumulation, and may even make some tasks difficult to execute truly precisely. Whether autoregressive models can stably implement precise rule execution under specialized training and sufficient constraints remains an open question. For the purpose of this paper, one-shot structured output provides a cleaner and more verifiable experimental condition.

3.3 Data Paradigm: From “Realistic Noise” to “Ideal Laws”

The most central change in the Neural Sculpting Paradigm occurs in the data. This paper uses programmatic scripts to generate large amounts of deterministic, low-noise, and verifiable ideal data. In these data, there is a clear generative relation between input and output. The model does not need to infer targets from open-world noise, ambiguity, or accidental correlations; instead, it can concentrate on learning the underlying rule.

This does not mean that an arbitrarily small amount of data is sufficient. Experiments show that there are thresholds in data volume and pattern coverage. When sample coverage is sufficient, training loss and validation loss usually decrease together. When the sample size is smaller, grokking may appear: the model first memorizes the training set and only later turns toward generalization. When the sample size falls below an even smaller threshold, the model can only overfit the training set and cannot make the validation set converge. Therefore, programmatic data is not a small number of prompts, but a large set of samples that constrains the possible function space. Most experiments in this paper generate training and validation sets by randomly sampling from the target input space, but this is not necessarily the only or optimal way to construct data.

3.4 Paradigm Interpretation: Sculpting Precise Rules with Gradient Descent

The name Neural Sculpting comes from an intuitive metaphor: programmatically generated ideal data provides constraints from many different angles, continuously excluding functions inconsistent with the target rule; gradient descent adjusts the neural network under these constraints, gradually approximating a discrete, deterministic, and verifiable rule structure within a high-dimensional continuous parameter space. More simply, the paradigm discovered in this paper is fitting a precise rule using connectionist gradient descent.

The key point here is not that the neural network “knows” the human meaning of every bit in the input and output. The model faces only input-output constraints and the loss function. Even if it eventually learns addition, cellular automata, or some algorithmic task, this does not mean that it executes the corresponding steps in a human-readable form. It is still searching through gradient descent for a parameter

configuration that can compress the input-output relation, reduce loss, and generalize to unseen samples. This also leads to the later theoretical interpretation in terms of compression as intelligence and MDL.

3.5 Model Description

This paper selects different standard neural networks according to the input-output form. Symbolic reasoning and algorithmic reasoning mainly use small Transformers or MLPs; image-to-symbol tasks use ViT or Swin Transformer; image-to-image tasks use Swin Transformer+UNet, UNet, or diffusion models; some symbol-to-image or multimodal tasks encode symbolic information into image channels, or use a Transformer followed by an image-generation module.

Corresponding code includes:

```
train_tiny_transformer.py, train_mlp.py, train_image2image.py,  
train_text2image.py, train_qwen2_text2image.py,  
train_swin_image2text.py, and others.
```

Specific model settings and dataset generation scripts are described separately in the later experimental sections.

4 Act IV: Empirical Evidence Across Multiple Domains

To systematically verify the universality of the Neural Sculpting Paradigm, this paper conducted exhaustive experiments on multiple types of tasks. In addition to the tasks detailed in the main text, I also tested on over 100 tasks. Full experiment descriptions and dataset generation scripts can be found in the GitHub repository: <https://github.com/ball-lightning6/neural-sculpting-paradigm>

4.1 Fundamental Capability: Symbolic Rule Learning

This section aims to verify the core capability of this paradigm to learn precise, complex symbolic rules from the most fundamental level.

Training/Validation Split Ratio: Evaluations (eval) are generally performed at a frequency of about 1000 steps to monitor the downward trend of eval loss at any time. The validation set ratio is calculated based on this frequency so that the eval time is not too long. The validation set accounts for less than 0.01 of the dataset. Since the validation set and the dataset are identically distributed with the input space, a reliable eval loss can be verified. **Regarding Model Training Hyperparameters:** Almost all hyperparameters do not need to be changed, except that sometimes batch size is increased for simpler tasks to improve VRAM utilization, and for some tasks involving image reasoning where training loss sometimes becomes nan, the learning rate will be halved in such cases. **Model Structure Changes:** For outputting symbols, the most important thing is generally the need to change the number of labels for multi-label binary classification. **Input Space Estimation:** For some tasks where the input space is difficult to calculate directly, I use a method similar to Monte Carlo estimation, i.e., generating task-related random numbers (these random numbers are sufficient to determine a sample) a relatively large number of times, and estimating the order of magnitude of the input space based on the number of repeated samples.

Note: Many of the following experiments only list the eval loss, which is strictly speaking not formal enough. This is because I only recorded this when conducting a large number of experiments at the time. Later, I also verified the test results of some cases, proving that the test set accuracy and eval loss are consistent; a lower eval loss does indeed correspond to a higher accuracy rate. I may supplement strict experimental results later.

4.1.1 Cellular Automata

- **Task Description:** The model learns to predict and output the state of a one-dimensional cellular automata after a given number of transformations according to a given rule.
- **Input Format:** A 30-bit string composed of “0” or “1”, representing the initial state of the cellular automata.
- **Output Format:** Multi-label binary classification format with 30 labels, equivalent to thirty 0/1 strings, representing the state of the cellular automata after transformation.
- **Loss:** Average Binary Cross Entropy Loss.
- **Training Configuration:** NVIDIA RTX 4090 GPU.
- **Training Set Generation Script:** `generate_cellular_automata_1d.py`
- **Dataset:** `ca_rule110_layer15_30.jsonl`
- **Training Code:** `train_tiny_transformer.py`
- **Training Set Size:** 500,000
- **Input Space Size Estimation:** 2 to the power of 30 = 1,073,741,824
- **Ratio of Training Set Size to Input Space Size:** 0.0466%

Task-Related Design Thoughts and Discussion:

1. To rigorously test the ability of neural networks to transcend pattern recognition and learn and execute precise symbolic rules, I chose one-dimensional cellular automata (CA) as the primary experimental platform. This choice was not accidental but based on its unique ideal characteristics capable of eliminating all ambiguity. First, CA is a computational system driven entirely by localized, deterministic rules. Its behavior does not rely on any statistical priors or semantic information derived from the real world, making it an ideal controlled environment for testing precise rule-learning capability. Second, its enormous state space (for a binary cellular automata of width W , there are 2^W possible states) makes any form of “rote memorization” futile, thereby forcing the model to learn the rules themselves rather than memorizing input-output pairs. On this basis, I specifically chose Rule 110 as the core test rule. Rule 110 is not only a non-trivial rule exhibiting complex and seemingly chaotic behavior, but more critically, it has been proven to be Turing complete. This means that, in principle, Rule 110 can simulate any computational process. Therefore, having a Transformer learn the evolution of Rule 110 is tantamount to testing whether the model has the potential to learn a form of general computation. Successfully learning this rule will provide the solidest and most convincing experimental evidence for the argument that “neural networks can internalize complex algorithms.”
2. Some task parameters:
 - Width: 30
 - Rule: 110
 - Layers: 15

Experimental Results and Analysis: The evaluation loss converged to 0.000068. This extremely low evaluation loss and the fact that the training set size is far smaller than the input space fully demonstrate that the neural network has learned this rule.

Final Convergence Results:

```
--- Step 477000 ---  
Train Loss: 0.001766  
Eval Loss: 0.000068
```

4.1.2 Inquiry into the Nature of Learning: Base-N Addition and Semantic Shuffling

- **Task Description:** The model learns to predict and output the addition of two base-N numbers, while comparing the training results of semantic shuffling and position shuffling.

- **Input Format:** A 16-symbol string: the first 8 symbols represent one base-16 number, and the last 8 symbols represent another.
- **Output Format:** Multi-label binary classification format with 33 labels, equivalent to thirty-three 0/1 strings, representing the addition result with a 33-bit binary number.
- **Loss:** Average Binary Cross Entropy Loss.
- **Training Configuration:** NVIDIA RTX 4090 GPU.
- **Training Set Generation Script:** `generate_symbol_add_shuffle_dataset.py`
- **Datasets:**
 - `adder_8bit_base16_train.jsonl` — No semantic shuffle, no position shuffle
 - `adder_8bit_base16_sem_shuffled_train.jsonl` — Semantic shuffle, no position shuffle
 - `adder_8bit_base16_pos_shuffled_train.jsonl` — No semantic shuffle, position shuffle
 - `adder_8bit_base16_sem_shuffled_pos_shuffled_train.jsonl` — Semantic shuffle, position shuffle
- **Training Code:** `train_tiny_transformer.py`
- **Training Set Size:** 500,000
- **Input Space Size Estimation:** 16 to the power of 16 = 18,446,744,073,709,551,616 approximately 1.84×10^{19}
- **Ratio of Training Set Size to Input Space Size:** 2.71×10^{-14}

Task-Related Design Thoughts and Discussion:

1. The model learns to perform addition on two 8-digit hexadecimal numbers. To explore the essence of model learning, I designed a “Semantic Shuffling” control experiment, including two types of shuffling: one uses random different symbols to represent hexadecimal numbers, and the other randomly selects the positional arrangement of hexadecimal symbols, no longer adopting the practice where the first 8 symbols and last 8 symbols are two hexadecimal numbers respectively. Reasons for designing the experiment this way:
 - **Semantic Shuffle:** To determine that what the transformer learns is the structure between symbols relative to the task, irrelevant to what the specific symbols are.
 - **Position Shuffle:** To determine that the transformer’s learning of this task, and the vast majority of tasks, does not depend on the arrangement of input symbols.
2. Some task-related parameters and details:
 - Base: Hexadecimal
 - Digits for each of the two numbers: 8
 - Under no semantic shuffle: 0123456789abcde are used as hexadecimal number symbol representations.
 - Under semantic shuffle: Different visible ASCII characters are arbitrarily chosen as hexadecimal number symbol representations.
 - Under no position shuffle: The first 8 symbols of the input string are one 8-digit hexadecimal number, and the last 8 symbols are the other.
 - Under position shuffle: At the beginning of the training dataset script, a position shuffle mapping is generated, and this mapping consistency is maintained throughout the dataset. Corresponding code is as follows:

```
if SHUFFLE_POSITIONS:
    position_map = list(range(INPUT_LEN))
```

```

random.shuffle(position_map)
POSITION_SHUFFLE_MAP = {
    from_idx: to_idx
    for from_idx, to_idx in enumerate(position_map)
}

```

Experimental Results and Analysis: The results in the table below strongly support the core claim. In both the “no semantic shuffle” and “semantic shuffle” settings, the model eventually converged to extremely low loss levels (<0.0001), with no significant performance difference. This indicates that the learned behavior is difficult to explain as mere surface symbol-pattern memorization, and is closer to the abstract structure behind addition.

Note: The training loss decline process generated by different random data and training random seeds may vary greatly, so the difficulty of training cannot be analyzed through a single comparison. However, in multiple attempts, it was found that the convergence of the 4 experimental settings exists stably.

Since the training process is relatively short, all training processes of the 4 experiments are listed here, represented by eval loss.

A: no semantic shuffle, no position shuffle. B: no semantic shuffle, position shuffle. C: semantic shuffle, no position shuffle. D: semantic shuffle, position shuffle.

step	A	B	C	D
1000	0.534105	0.505196	0.511109	0.564834
2000	0.438036	0.263462	0.446237	0.497787
3000	0.347195	0.001700	0.365586	0.461389
4000	0.324581	0.001392	0.179969	0.395561
5000	0.254971	0.000123	0.063128	0.328988
6000	0.129849	0.000042	0.001083	0.247512
7000	0.066946	0.000024	0.000203	0.159110
8000	0.044260		0.000123	0.149018
9000	0.032533		0.000051	0.139221
10000	0.001005			0.103275
11000	0.000325			0.080524
12000	0.000099			0.052849
13000				0.022913
14000				0.002443
15000				0.002063
16000				0.000327
17000				0.000472
18000				0.000272
19000				0.000293
20000				0.000409
21000				0.000173
22000				0.000164
23000				0.000391
24000				0.000243
25000				0.000095

4.2 Advanced Capability: Algorithm Learning and Planning

4.2.1 Case Study: Trapping Rain Water Problem

- **Task Description:** Derived from the classic LeetCode algorithm problem, [42. Trapping Rain Water - LeetCode](#) [19].
- **Input Format:** A 30-bit 01 string. Every consecutive 3-bit 01 string represents the height of a bar (0-7) in 3-bit binary format, with a total of ten bars.
- **Output Format:** A 30-bit 01 string. Every consecutive 3-bit 01 string represents the amount of water trapped by a bar (0-7) in 3-bit binary format, with a total of ten bars.
- **Loss:** Average Binary Cross Entropy Loss.
- **Training Configuration:** NVIDIA RTX 4090 GPU.
- **Training Set Generation Script:** `generate_trapping_rain_water_decoupled.py`
- **Dataset:** `trapping_rain_water_decoupled_n10_b3_train.jsonl`
- **Training Code:** `train_tiny_transformer.py`
- **Training Set Size:** 300,000
- **Input Space Size Estimation:** 2 to the power of 30 = 1,073,741,824, approximately $1.07e9$
- **Ratio of Training Set Size to Input Space Size:** 0.0279%

42. Trapping Rain Water

Solved 

Hard  Topics  Companies

Given n non-negative integers representing an elevation map where the width of each bar is 1, compute how much water it can trap after raining.

Example 1:



Input: `height = [0,1,0,2,1,0,1,3,2,1,2,1]`

Output: 6

Explanation: The above elevation map (black section) is represented by array `[0,1,0,2,1,0,1,3,2,1,2,1]`. In this case, 6 units of rain water (blue section) are being trapped.

Example 2:

Input: `height = [4,2,0,3,2,5]`

Output: 9

Constraints:

- `n == height.length`
- `1 <= n <= 2 * 104`
- `0 <= height[i] <= 105`

Illustration of the Trapping Rain Water problem.

Task-Related Design Thoughts and Discussion:

1. LeetCode is a natural repository for testing the algorithmic capabilities of this paradigm. I tested many algorithmic problems from it, and many of them can be fitted under this setting. The Trapping Rain Water problem is categorized as hard on LeetCode, and its solution cannot be simulated by a simple symbolic rule. The exact fitting mechanism remains unclear. However, I do not think

the Transformer first parses the format and then executes a human-style algorithm step by step. A neural network cannot know, by any method, the human meaning of each input or output bit, nor can it know the meaning and structure of this task in a human context. It only faces the constraints between inputs and outputs, and the optimization target defined by the loss. Therefore, I am more inclined to believe that the model is not executing explicit algorithmic steps as humans understand them, but is using gradient descent to approximate a deterministic mapping from input to output. This is also why I describe this paradigm as using connectionist gradient descent to approximate discreteness.

2. Regarding why the output is not the total amount of trapped water: I previously trained directly with the binary number of the total amount as the target, but found that the effect was not good and convergence was difficult. Therefore, I adopted this decoupled format, which allowed for extremely rapid convergence. Here, I reduced the burden of a summation on the Transformer, which should be the reason for the model's rapid convergence. Additionally, another experiment I conducted also encountered difficulties in convergence because it mixed too many simple operations serially; this is a very interesting phenomenon worth studying and will be discussed later.
3. Some task parameters:
 - Number of bars: 10
 - Number of binary bits used to represent bar height and trapped water: 3
 - Range of bar height / trapped water amount: 0-7

Experimental Results and Analysis: After 16,000 steps of training, the model evaluation loss converged to 0.000059. This result indicates that this paradigm can effectively learn complex algorithms that require understanding global information (such as left and right highest walls) and achieves precise modeling of the problem structure through the decoupling of output formats.

Final Convergence Results:

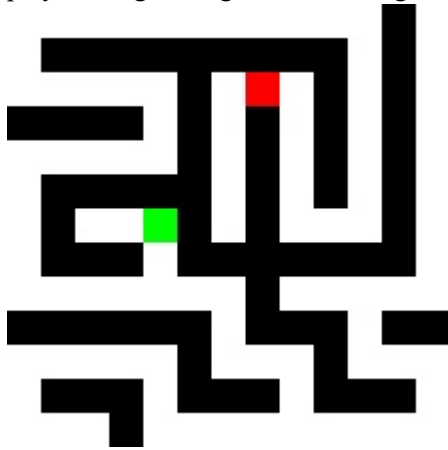
```
--- Step 16000 ---
Train Loss: 0.000570
Eval Loss: 0.000059
```

4.2.2 Dense Maze Navigation

- **Task Description:** Within a complex dense maze designed for human players, the model learns to predict the “next step” direction (up/down/left/right) of the optimal path from any starting location to the destination.
- **Input Format:** A 169-character string composed of “0”, “1”, “s”, or “t”, representing the initial state of the maze. Every consecutive 13 characters represent a row of the maze. 0 represents open ground, 1 represents a wall, s represents the starting point, and t represents the target point. There is exactly one s and one t. By default, there is a ring of walls around the 13*13 maze.
- **Output Format:** 4-class classification.
- **Loss:** 4-class Cross Entropy Loss.
- **Training Configuration:** NVIDIA RTX 4090 GPU.
- **Training Set Generation Script:** `generate_maze_dense.py`
- **Dataset:** `maze_optimized_13_13_dataset.jsonl`
- **Training Code:** `train_tiny_transformer.py`
- **Training Set Size:** 2,000,000
- **Input Space Size Estimation:** Using Monte Carlo method to estimate 2,000,000 times without duplicate samples, tentatively roughly estimated as 2 to the power of 169 approximately $7.48e50$.
- **Ratio of Training Set Size to Input Space Size:** Approx. $2.673e-45$

The following figure shows a typical maze, using white for open ground, black for walls, green for the starting point, and red for the destination. This is a typical maze game suitable for human

play, distinguishing it from maze generation algorithms that randomly generate independent wall points.



Task-Related Design Thoughts and Discussion:

1. Based on confidence in the algorithm fitting capability of this paradigm, the design of this maze experiment initially used an algorithm equivalent to randomly generating wall points. However, practice proved that such mazes might be very easy for humans, who can spot the shortest path (or at least a feasible path) at a glance, but for the model, the number of branches and complexity far exceed the current maze experiment setup.
2. The reason for choosing the direction of the next step along the shortest path is that this format is relatively simple to design. Direct output of the shortest path would require knowing the length of the shortest path in advance; even with other measures, the task design becomes less elegant. Another important reason is that such a design implies another possibility: the maze task can be seen as a task of choosing the optimal action at a certain state. This can be linked to reinforcement learning. Training state-action pairs in this way can test the potential of this paradigm to complete reinforcement learning tasks, see the discussion on arc-agi-3 later. Training also has two methods: direct supervised learning of the optimal action and reinforcement learning exploration of the optimal action, but due to time and energy constraints, I did not complete the experiment of directly using reinforcement learning to explore the optimal action.
3. Some task parameters:
 - Width: 13
 - Height: 13

Experimental Results and Analysis: The evaluation loss converged to 0.000014, indicating that the model can stably predict the next optimal move under this fixed maze distribution. This suggests that part of the input-to-optimal-action mapping in traditional search problems can, under controlled data conditions, be amortized into the weights of a feedforward network. The extremely low evaluation loss means that the model has learned to navigate similar mazes, and also implies that at the length scale of the optimal paths in this setting, the overall path accuracy should be very high.

Final Convergence Results:

```
--- Step 279000 ---  
Train Loss: 0.000083  
Eval Loss: 0.000014
```

4.3 Reasoning and Extracting Symbolic Output from Images

This section will demonstrate the powerful potential of this paradigm in handling multimodal tasks, including decoding symbols from visual information.

4.3.1 Visual Symbol Decoding: Dial Angle Identification

- **Task Description:** Image reasoning. By inputting an image resembling a dial, output the occupation of all angle intervals by these line segments of different colors and thicknesses.
- **Input Format:** 224*224 image.
- **Output Format:** Multi-label binary classification format with 36 labels, equivalent to thirty-six 0/1 strings, representing whether each interval is occupied by line segments. 36 intervals divide the 360-degree angle equally, with each interval being 10 degrees.
- **Loss:** Average Binary Cross Entropy Loss.
- **Training Configuration:** NVIDIA RTX 4090 GPU.
- **Training Set Generation Script:** `generate_line_angle_to_vector.py`
- **Dataset:** `line_angle` folder.
- **Training Code:** `train_swin_image2text.py`
- **Model Used:** Swin Transformer.
- **Training Set Size:** 200,000
- **Input Space Size Estimation:** 389,329,651
- **Ratio of Training Set Size to Input Space Size:** 0.0514%

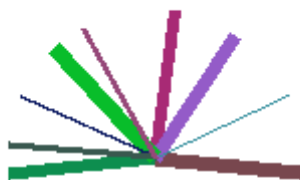


Illustration of the dial-angle visual symbol decoding task.

Task-Related Design Thoughts and Discussion:

1. After the algorithm fitting capability was proven, I thought of the application of Transformers in images, so I wanted to test whether image Transformers have similar reasoning capabilities. So I used ViT to test tasks related to checkerboards, and the results were very good. Later, I designed this task with the initial intention of testing pure image understanding, rather than simply identifying symbols in images and then applying the already proven symbolic rule learning capability and algorithm fitting capability of Transformers.
2. This task initially used the ViT model, but the effect was not good, so it was later switched to the currently used Swin Transformer.
3. To increase diversity, randomness was added to the color and thickness of the line segments.
4. Some task parameters:
 - Image resolution: 224*224
 - Number of intervals: 36
 - Angle per interval: 10

Experimental Results and Analysis: The evaluation loss converged to 0.019183. This is not perfect convergence, but it shows that the model can extract fairly stable, abstract, and symbolized angle-interval information from raw pixels.

Final Convergence Results:

```
--- Step 24000 ---  
Train Loss: 0.016392  
Eval Loss: 0.012753
```

4.4 Image Reasoning Capability

This section will demonstrate the paradigm’s pure image reasoning capabilities to distinguish it from symbolic rule learning capabilities and algorithm fitting capabilities.

4.4.1 Geometric Reasoning and Construction: Prediction of Triangle Incircle

- **Task Description:** The input to the model is an image of a randomly generated green triangle, and the task is to generate the image of its mathematically unique, correct red incircle.
- **Input Format:** 224*224 image.
- **Output Format:** 224*224 image.
- **Loss:** MSELoss.
- **Training Configuration:** NVIDIA RTX 4090 GPU.
- **Training Set Generation Script:** `generate_triangle_to_incircle.py`
- **Dataset:** `incircle_dataset` folder.
- **Training Code:** `train_image2image.py`
- **Model Used:** Swin Transformer + unet.
- **Training Set Size:** 150,000
- **Input Space Size Estimation:** Estimated using Monte Carlo method to be approx. $2.8e+12$.
- **Ratio of Training Set Size to Input Space Size:** Approx. $5.4e-8$

Task-Related Design Thoughts and Discussion:

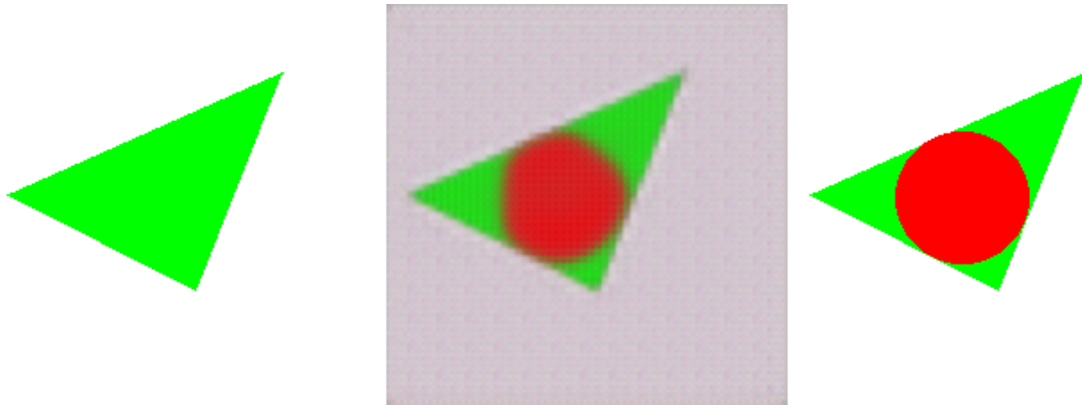
1. To further test pure image reasoning capability and separate it from symbolic rule learning and algorithm fitting, I designed this incircle experiment. In theory, humans can decompose this task into recognizing the three point positions, computing the incircle with geometric formulas, and then drawing it. But a neural network cannot know these human-defined geometric steps, nor can it be naturally explained as executing this explicit algorithm. What this task really tests is whether an image model can directly learn the deterministic mapping from triangle images to incircle images from a large number of samples, without an explicit geometric program. Given the analog nature of images, this reasoning capability has potential to extend to larger domains such as pure experimental observation data. Of course, real observation data contains noise, and may therefore degenerate back into the previous pattern-recognition paradigm; this issue is discussed later.
2. The most interesting aspect of this task and the subsequent related tasks is that the training process shows, in an intuitive form, how the model output gradually approaches the target rule. Some intermediate outputs naturally evoke the impression that a “machine mind” is forming: the model does not learn the rule all at once, but gradually excludes incorrect structures and preserves structures closer to the target during training. The training process is visualized through evaluation images.
3. Some task parameters:
 - Image resolution: 224*224

Experimental Results and Analysis: The final training results show that, under this setting, the model can stably generate incircle images close to the target. This indicates that image models can indeed learn this type of structured geometric mapping through training. In this example alone, an interpolation

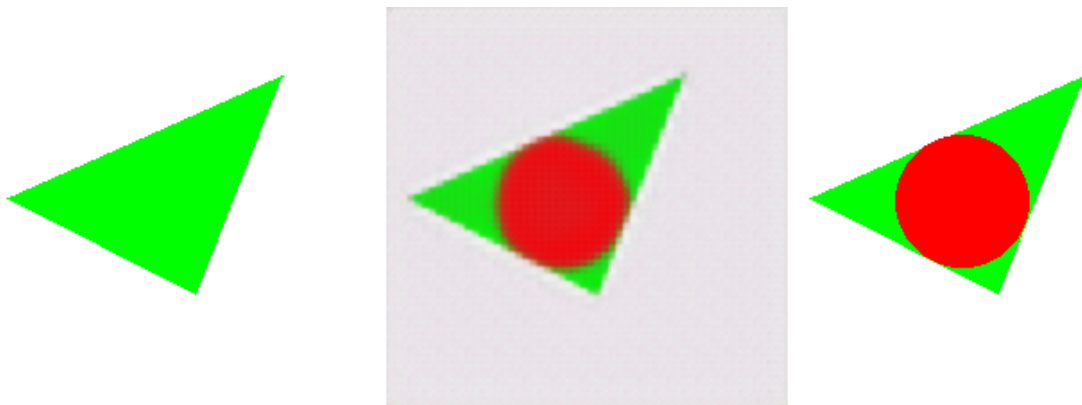
explanation cannot be completely ruled out, but later, more complex image-rule tasks further weaken that explanation. At the same time, the training process provides an intuitive window into how the model gradually forms geometric output structures.

Experimental Process Display:

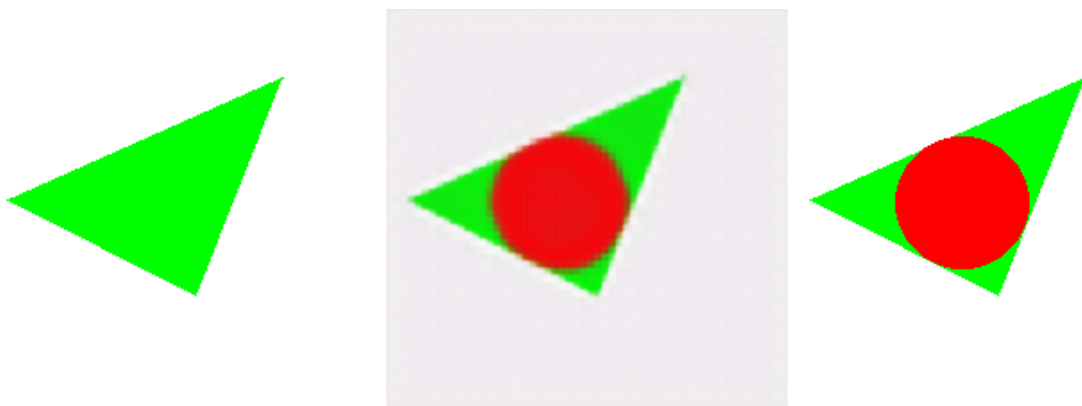
Left is the input image, right is the ground truth, middle is the generated image.



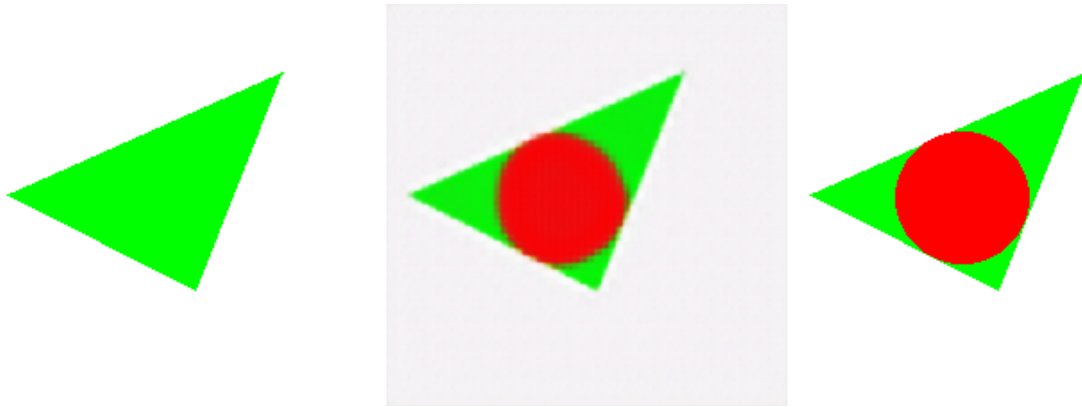
Triangle incircle experiment: 200-step training visualization. Left: input image; middle: model-generated image; right: ground truth.



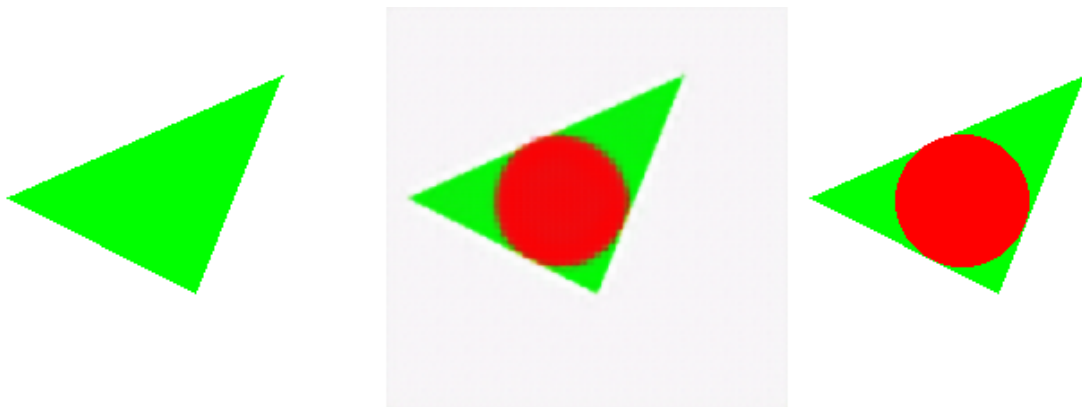
Triangle incircle experiment: 400-step training visualization.



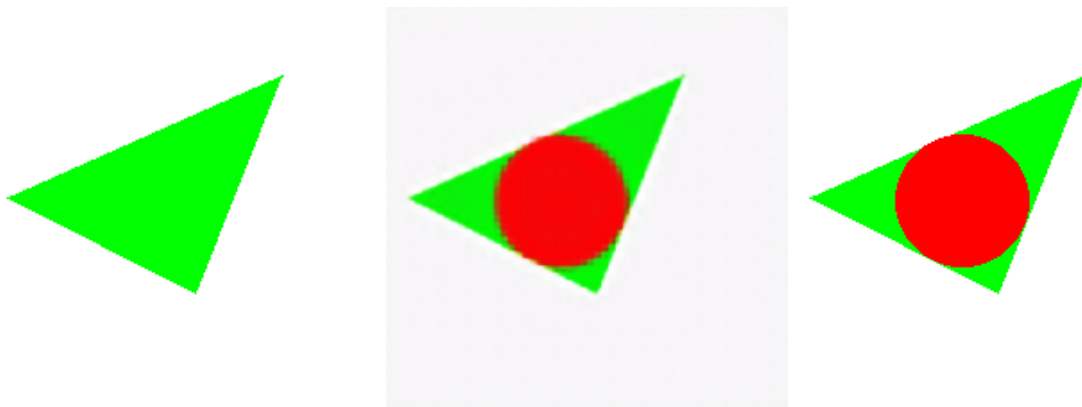
Triangle incircle experiment: 600-step training visualization.



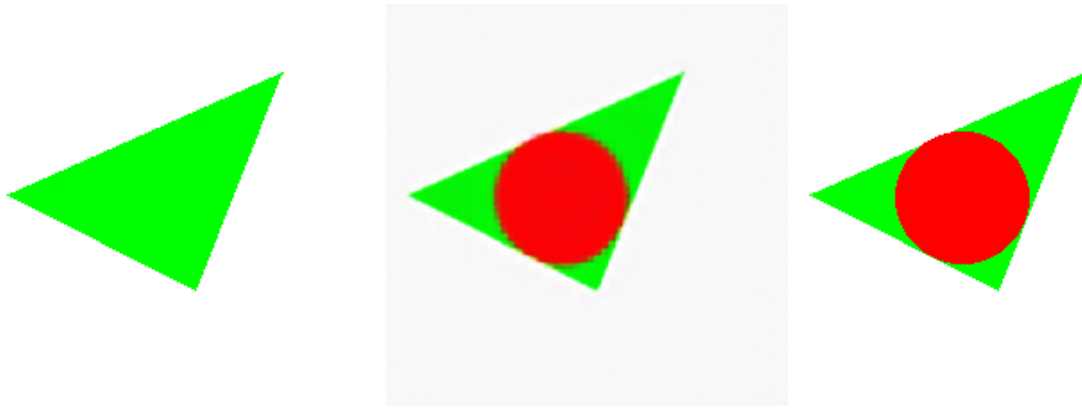
Triangle incircle experiment: 800-step training visualization.



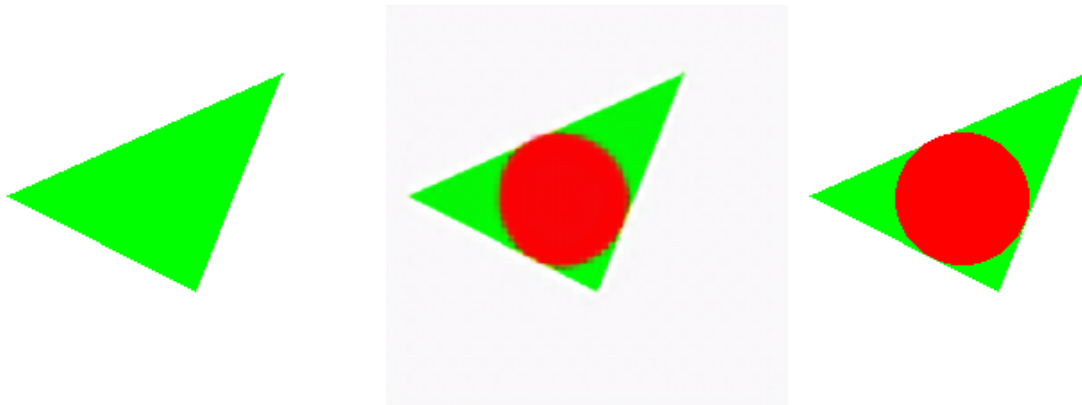
Triangle incircle experiment: 1000-step training visualization.



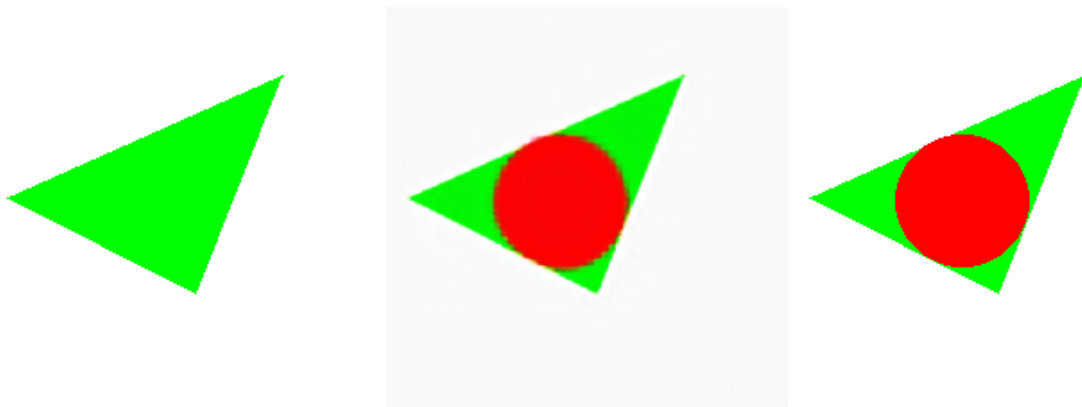
Triangle incircle experiment: 1200-step training visualization.



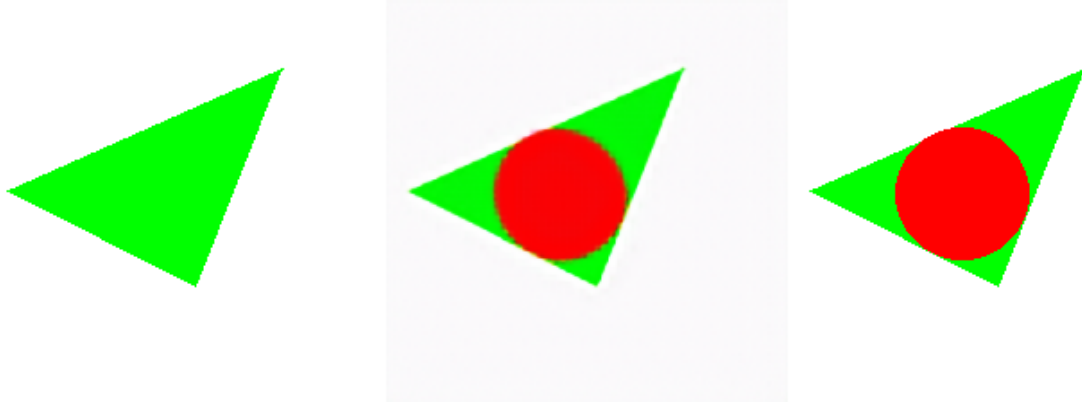
Triangle incircle experiment: 1400-step training visualization.



Triangle incircle experiment: 1600-step training visualization.

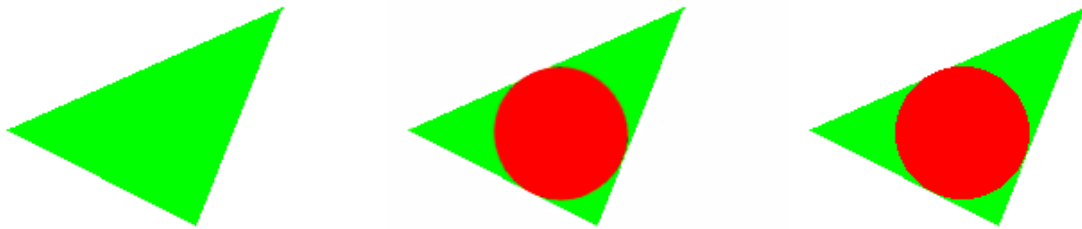


Triangle incircle experiment: 1800-step training visualization.



Triangle incircle experiment: 2000-step training visualization.

Due to the slowing down of changes later, the final training result is displayed directly.



Triangle incircle experiment: 14000-step final training result.

4.4.2 Geometric Reasoning and Construction: Plane Tessellation

- **Task Description:** The input to the model is an image of a randomly generated green triangle, and the output image is a plane tessellation of the green triangle, where green triangles and red triangles alternate and share adjacent edges.
- **Input Format:** 224*224 image, containing one randomly generated green triangle image.
- **Output Format:** 224*224 image, which is the plane tessellation result of the input image, with green and red triangles alternating and adjacent by edges.
- **Loss:** MSELoss.
- **Training Configuration:** NVIDIA RTX 4090 GPU.
- **Training Set Generation Script:** generate_triangle_to_tessellation.py
- **Dataset:** tessellation_dataset_224 folder.
- **Training Code:** train_image2image.py
- **Model Used:** Swin Transformer + unet.
- **Training Set Size:** 150,000
- **Input Space Size Estimation:** Estimated using Monte Carlo method to be approx. $2.71e10$.
- **Ratio of Training Set Size to Input Space Size:** Approx. $5.5e-6$

Task-Related Design Thoughts and Discussion:

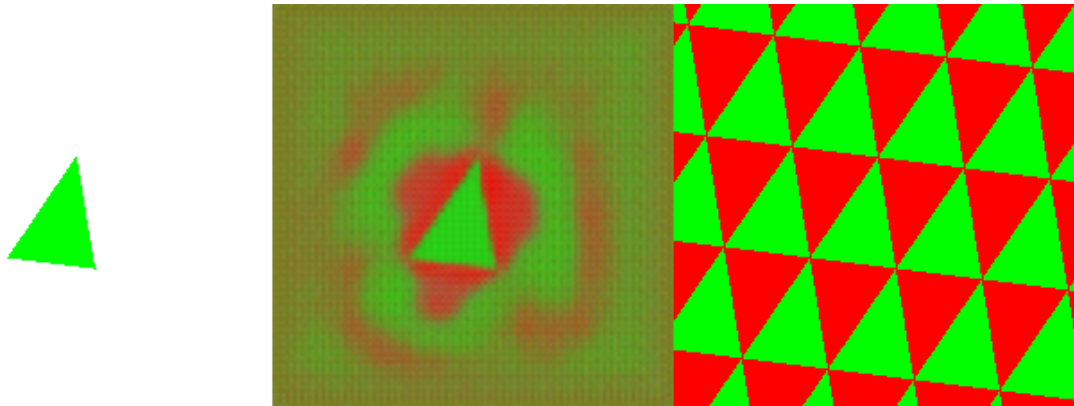
1. This task can completely rule out the possibility of solving the task by interpolation rather than learning precise rules. The reason is that far away in the plane tessellation, the error of such interpolation will gradually expand to the point where the rules can no longer be fitted by naive interpolation methods.

2. To ensure appropriate task difficulty, constraints were placed on the shape and size of the input green triangle; see the training set generation script code for details.
3. Some task parameters:
 - Image resolution: 224*224

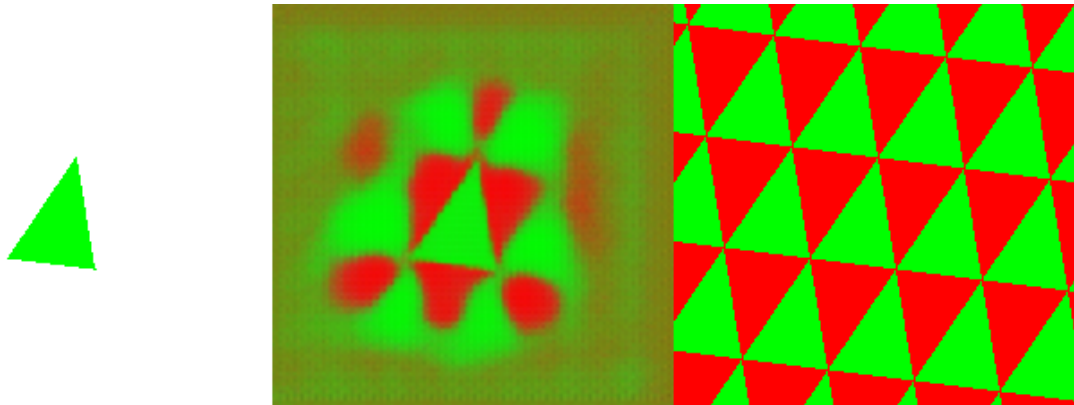
Experimental Results and Analysis: The final training results show that, under this setting, the model has learned to generate plane tessellation images consistent with the rule. This indicates that image models can fit not only local geometric relations, but also image transformation rules requiring long-range structural consistency. This experiment further weakens the explanation that the model is merely performing naive interpolation. The training process shows that the output first forms correct structures near the input triangle and then gradually expands outward, providing a very intuitive window into the learning process.

Experimental Process Display:

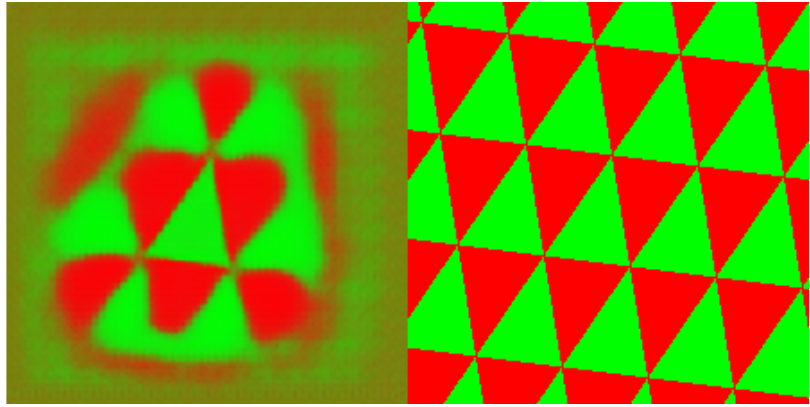
Left is the input image, right is the ground truth, middle is the generated image.



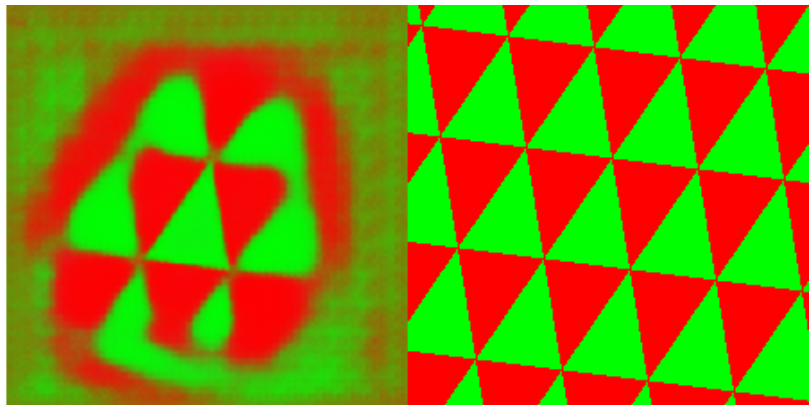
Plane tessellation experiment: 200-step training visualization.



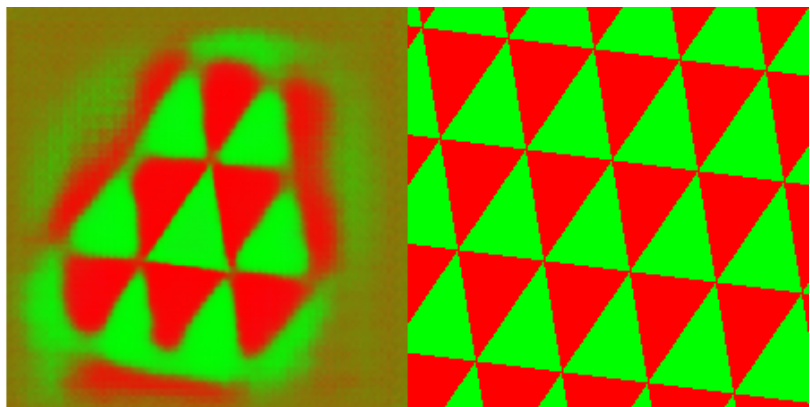
Plane tessellation experiment: 400-step training visualization.



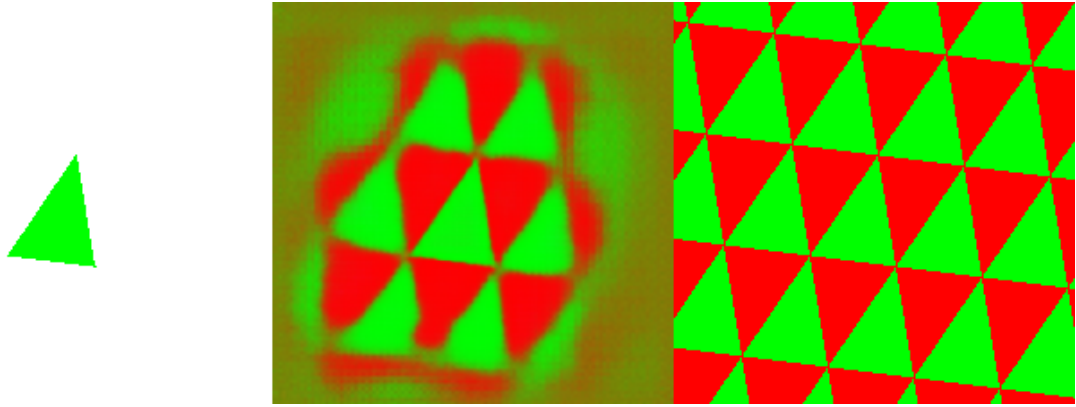
Plane tessellation experiment: 600-step training visualization.



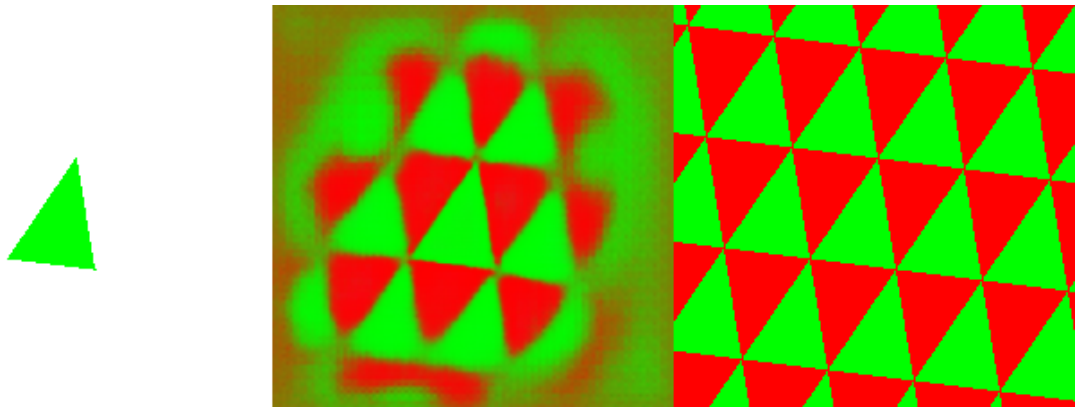
Plane tessellation experiment: 800-step training visualization.



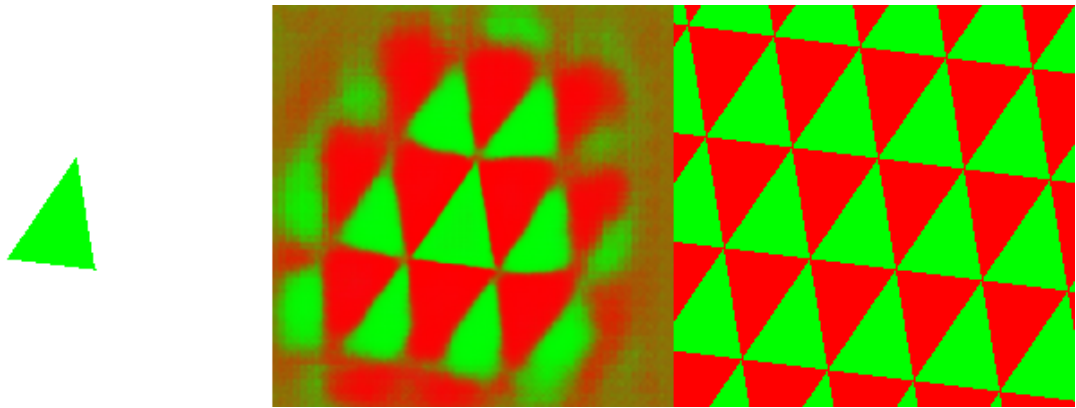
Plane tessellation experiment: 1000-step training visualization.



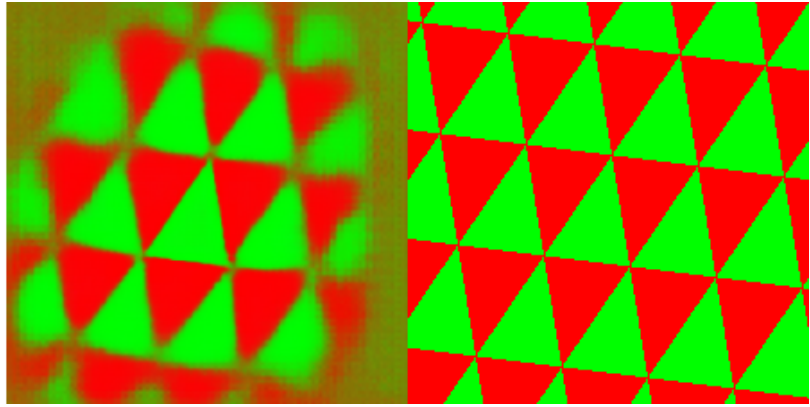
Plane tessellation experiment: 1200-step training visualization.



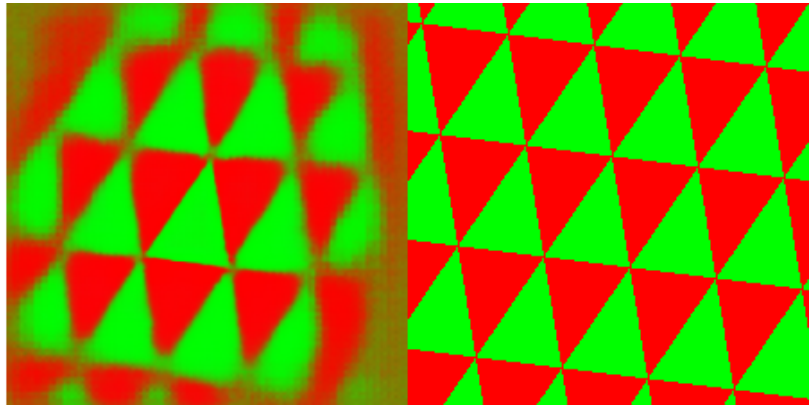
Plane tessellation experiment: 1400-step training visualization.



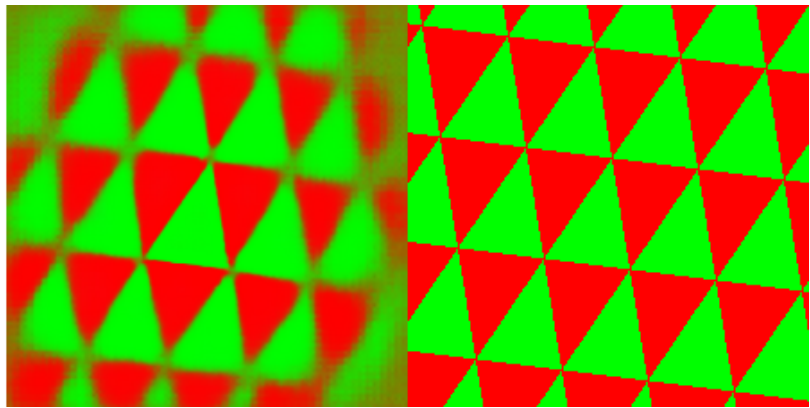
Plane tessellation experiment: 1600-step training visualization.



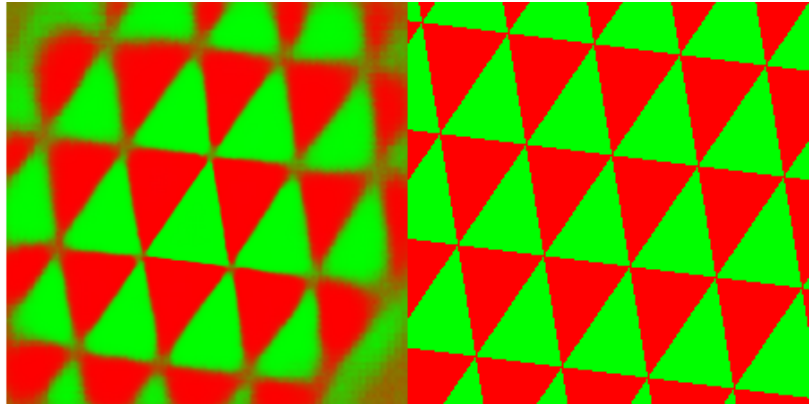
Plane tessellation experiment: 1800-step training visualization.



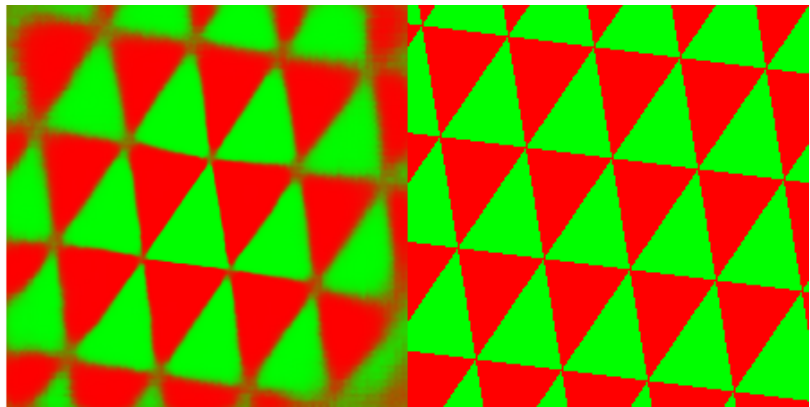
Plane tessellation experiment: 2000-step training visualization.



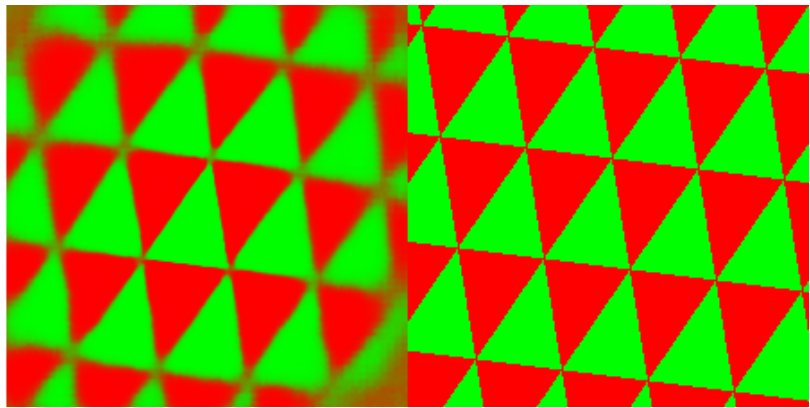
Plane tessellation experiment: 3000-step training visualization.



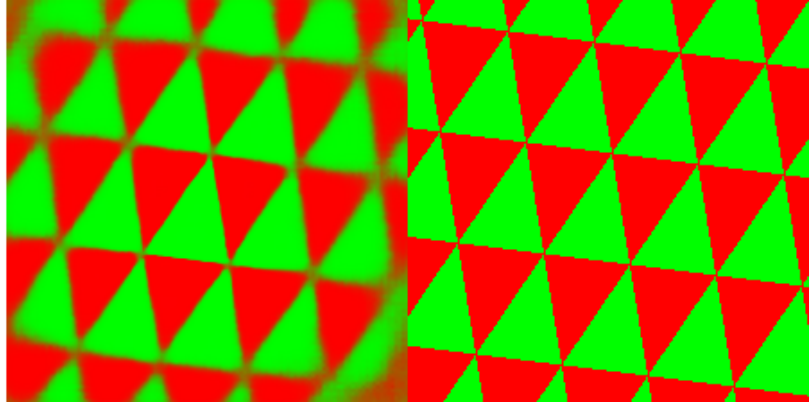
Plane tessellation experiment: 4000-step training visualization.



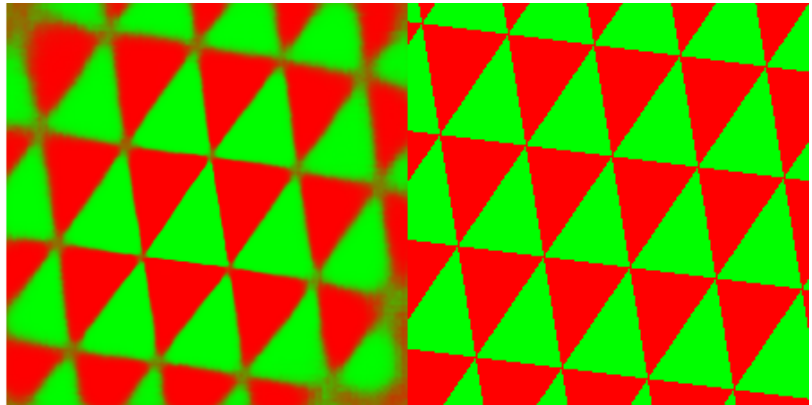
Plane tessellation experiment: 5000-step training visualization.



Plane tessellation experiment: 6000-step training visualization.

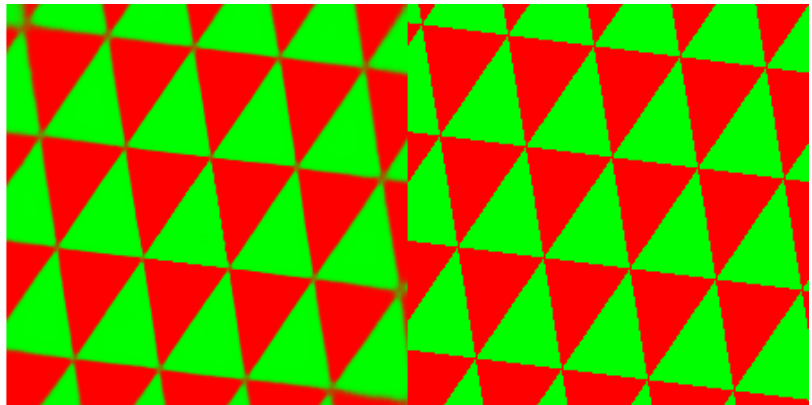


Plane tessellation experiment: 7000-step training visualization.



Plane tessellation experiment: 8000-step training visualization.

Due to the slowing down of changes later, the final training result is displayed directly.



Plane tessellation experiment: 150000-step final training result.

4.5 Image Generation via Symbolic Reasoning

This section will demonstrate the capability of this paradigm to process symbolic inputs, learn the corresponding rules, and generate corresponding images.

4.5.1 Cellular Automata

- **Task Description:** The model learns to predict and output the state of a one-dimensional cellular automata after a given number of transformations according to a given rule.

- **Input Format:** A 36-bit string composed of “0” or “1”, representing the initial state of the cellular automata.
- **Output Format:** 240*240 image.
- **Loss:** MSELoss.
- **Training Configuration:** NVIDIA RTX 4090 GPU.
- **Training Set Generation Script:** generate_cellular_automata_1d_to_grid_image.py
- **Dataset:** ca_render_dataset_240 folder.
- **Training Code:** train_qwen2_text2image.py
- **Model Used:** transformer + unet.
- **Training Set Size:** 300,000
- **Input Space Size Estimation:** 2 to the power of 36 = 68,719,476,736
- **Ratio of Training Set Size to Input Space Size:** 4.4e-6

Task-Related Design Thoughts and Discussion:

1. This task is intended to verify the capability of the Transformer for image generation via symbolic reasoning. This mode is not text-to-image in the traditional sense, but focuses on verifying its reasoning ability.
2. In the output image, the 1D 36-bit cellular automata state is arranged in row-major order into a 6*6 checkerboard grid image, where black represents 1 and white represents 0.
3. Some task parameters:
 - Grid dimension: 6*6
 - Grid size: 40*40 pixels
 - Image resolution: 240*240
 - Cellular automata rule: rule 110
 - Rule evolution layers: 3

Experimental Results and Analysis: The final training results show that, even when the output is an image, the model can stably fit the cellular-automaton rule. This indicates that when the output changes from a symbolic sequence to a structured image, the model can still learn the underlying discrete rule. More importantly, the model learns very quickly, and the training process provides a direct window into the formation of a “machine mind”: it does not construct the final image step by step according to human-imagined explicit logic, but gradually approximates a discrete rule in a connectionist manner through gradient descent. This is one of the core phenomena repeatedly emphasized in this paper.

Left is ground truth, right is model generated image.



100-step result. Symbol input: 111110011010001000111010000110.



200-step result. Symbol input: 111110011010001000111010000110.



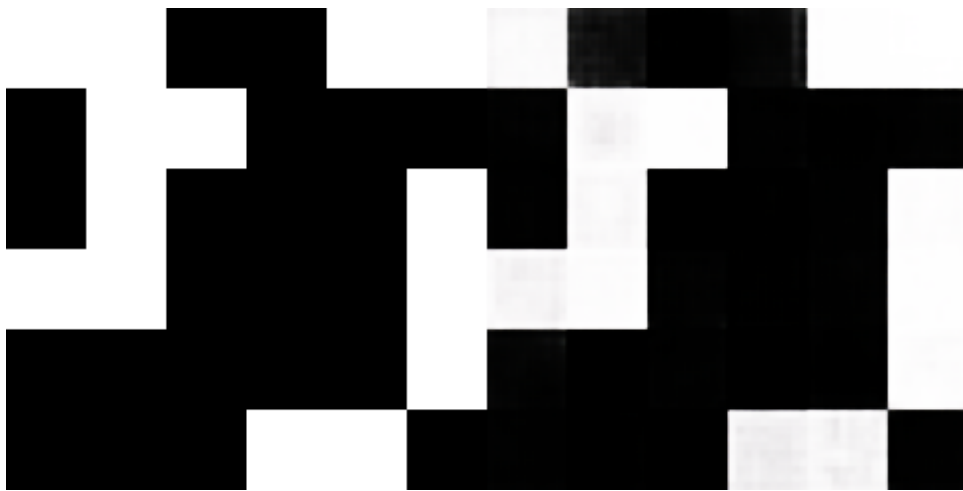
300-step result. Symbol input: 111110011010001000111010000110.



400-step result. Symbol input: 111110011010001000111010000110.



500-step result. Symbol input: 111110011010001000111010000110.



600-step result. Symbol input: 111110011010001000111010000110.



700-step result. Symbol input: 111110011010001000111010000110.



800-step result. Symbol input: 111110011010001000111010000110.

4.5.2 Rotating Cube

- **Task Description:** The model learns to predict and output an image of a colored hexahedron rotated according to rotation angles.
- **Input Format:** A 24-bit string composed of “0” or “1”, representing 3 rotation angles. Every consecutive 8 bits represent one rotation angle.
- **Output Format:** 256*256 image.
- **Loss:** MSELoss.
- **Training Configuration:** NVIDIA RTX 4090 GPU.
- **Training Set Generation Script:** `generate_cube_rotation_pillow_with_anchor.py`
- **Dataset:** `cube_dataset_final_highlight` folder.
- **Training Code:** `train_qwen2_text2image.py`
- **Model Used:** transformer + unet.
- **Training Set Size:** 300,000
- **Input Space Size Estimation:** 16,777,216
- **Ratio of Training Set Size to Input Space Size:** 1.79%

Task-Related Design Thoughts and Discussion:

1. In this task, a special difficulty was encountered: the model found it hard to associate the vertices corresponding to the rotation angles with the symbolic input, leading to a failure to converge. The solution was to always highlight a special vertex in the final step of generating training set images, regardless of occlusion relationships. After adopting this solution, the model was finally able to converge on this task. In another task where I used symbols to indicate the position of triangles and output the plane tessellation result, the model encountered the same difficulty; it could not understand which green triangle in the output image the symbol corresponded to. The solution was similar: I changed the color of the triangle indicated by the symbolic input to black in the output image, and everything became normal. This should be a fairly typical problem, i.e., it is difficult for the model to determine how to map symbolic input to a specific feature in the output image, and the solutions should also be similar.
2. Regarding the white cross lines on each face, they are just for aesthetics and may also have some auxiliary training effects.
3. The 3 rotation angles of symbolic input are pitch, yaw, and roll; please refer to the training set generation code for specific correspondence.
4. Actually, I have another version of the model that does not use the method of lora fine-tuning qwen2-0.5b, but uses a transformer with fewer total parameters than qwen2-0.5b but more param-

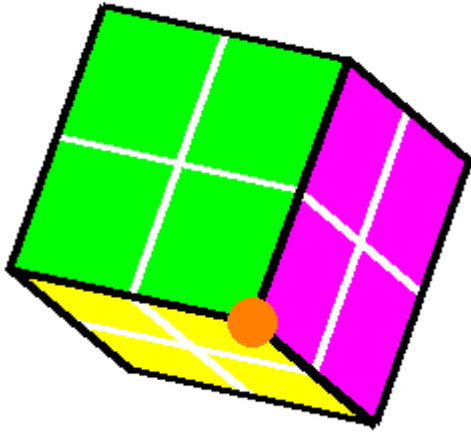
eters than the lora parameters of qwen2-0.5b. Practice has proven that the effect is not as good as the learning effect of qwen2-0.5b. This paper considers this an interesting phenomenon.

5. Some task parameters:

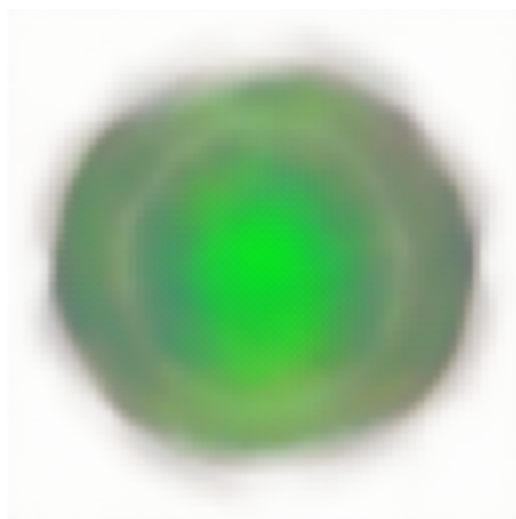
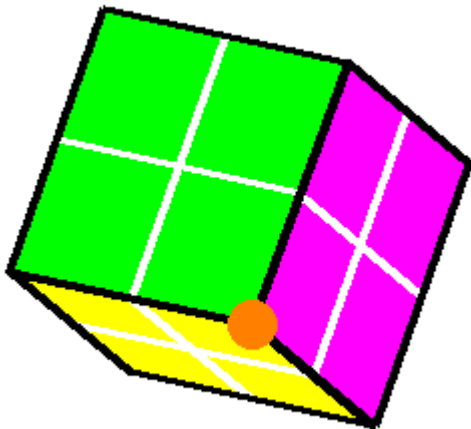
- Image resolution: 256*256

Experimental Results and Analysis: The final training results show that the model has completely learned this task. This indicates that this mode of generating images from symbols can also preserve the image reasoning capability of the Transformer. And given that I used lora fine-tuning of qwen2-0.5b, it can be said that this image reasoning capability, at least the capability to generate images, is not limited to image transformers like Swin Transformer, ViT, etc. The learning speed of the model is not very fast, possibly because the task is relatively difficult, providing direct observation of the formation process of the machine mind. In this task, can be seen that at first, the model did not yet understand rotation angles and simply learned to generate circles and relatively averaged filling colors. Later, the model slowly understood the concept of rotating cubes, and even later, the model slowly filled in precise details.

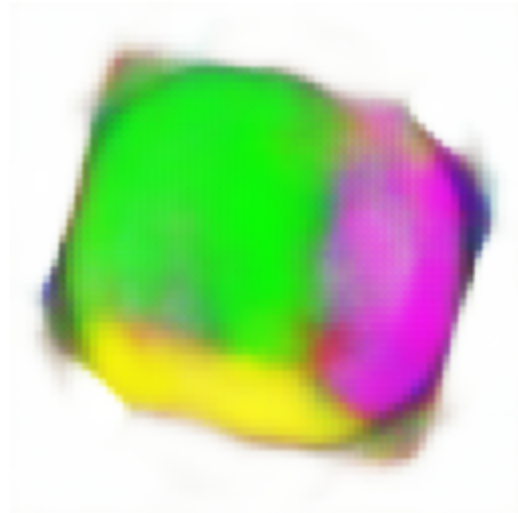
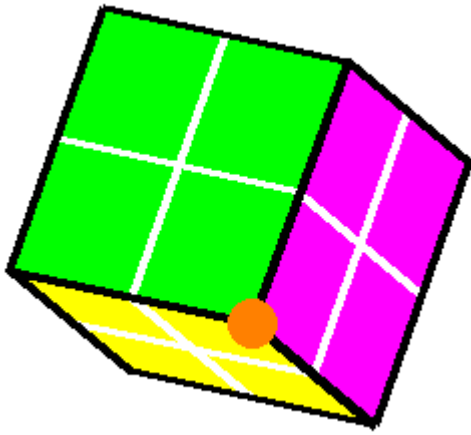
Left is ground truth, right is model generated image.



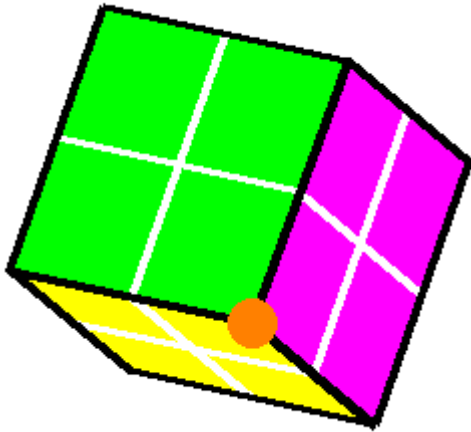
500-step result. Symbol input: 100101101000100100110010.



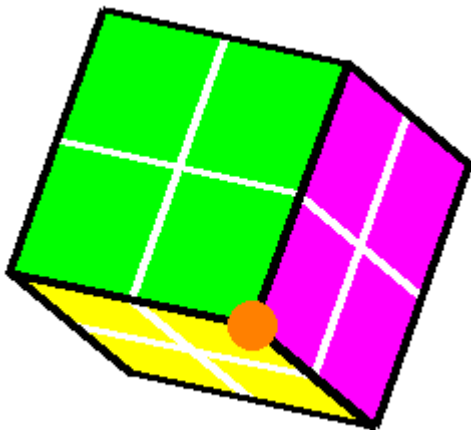
1000-step result. Symbol input: 100101101000100100110010.



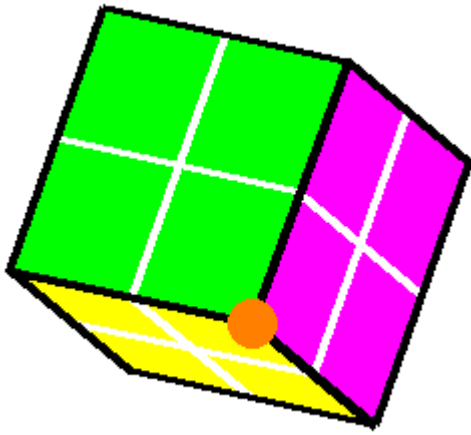
1500-step result. Symbol input: 100101101000100100110010.



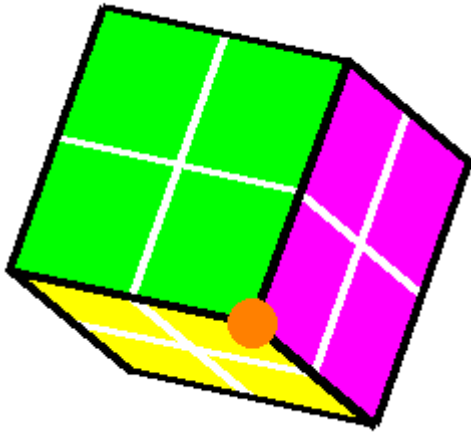
2000-step result. Symbol input: 100101101000100100110010.



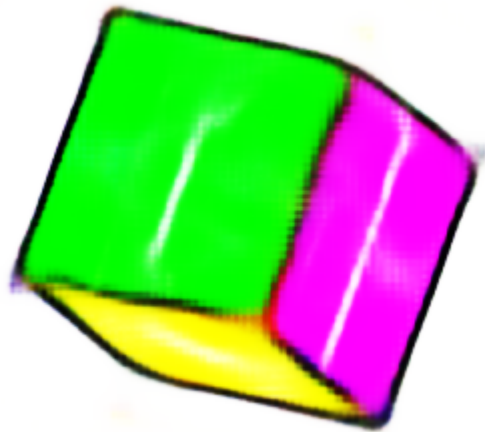
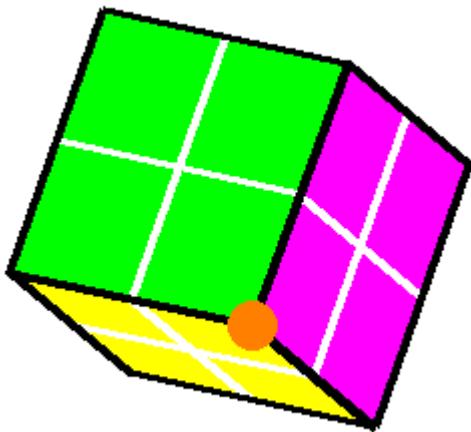
2500-step result. Symbol input: 100101101000100100110010.



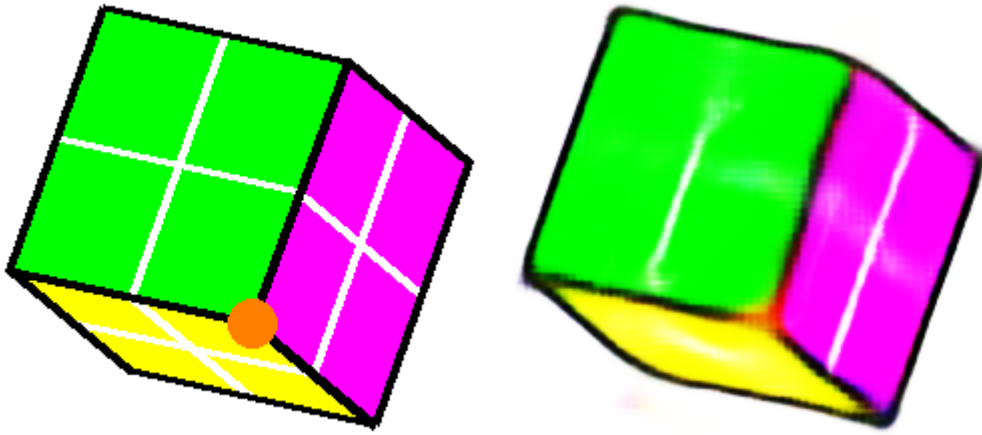
3000-step result. Symbol input: 100101101000100100110010.



4000-step result. Symbol input: 100101101000100100110010.

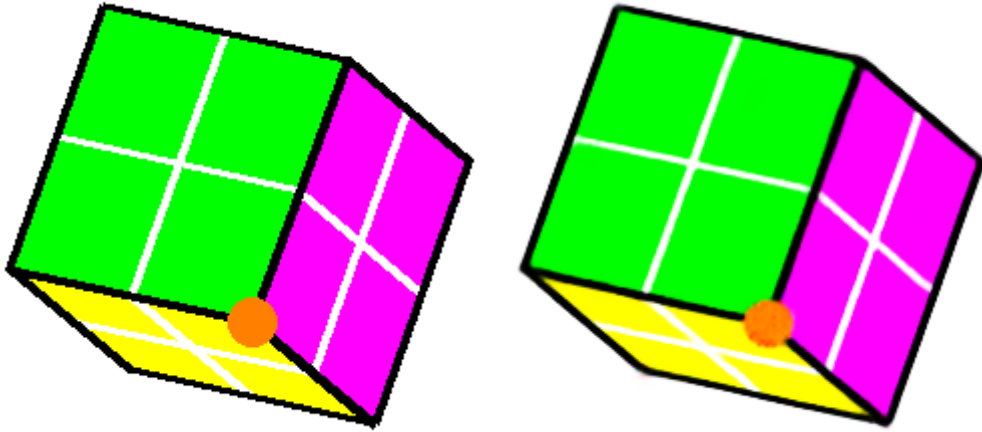


6000-step result. Symbol input: 100101101000100100110010.



8000-step result. Symbol input: 100101101000100100110010.

Due to the slowing down of changes later, the final training result is displayed directly.



133600-step final training result. Symbol input: 100101101000100100110010.

4.6 Physical Simulation

This section will demonstrate the capability of this paradigm in simulating physical processes.

4.6.1 Catenary

- **Task Description:** Given the endpoints at both ends of a catenary, and a given passing point on the catenary, the model learns to draw the catenary trajectory uniquely determined by physical laws (principle of minimum potential energy).
- **Input Format:** 224*224 image.
- **Output Format:** 224*224 image.
- **Loss:** MSELoss.
- **Training Configuration:** NVIDIA RTX 4090 GPU.
- **Training Set Generation Script:** generate_catenary_curve_from_points.py
- **Dataset:** catenary_dataset_v5_CONSTRUCTIVE folder.
- **Training Code:** train_image2image.py
- **Model Used:** Swin Transformer + unet.
- **Training Set Size:** 150,000

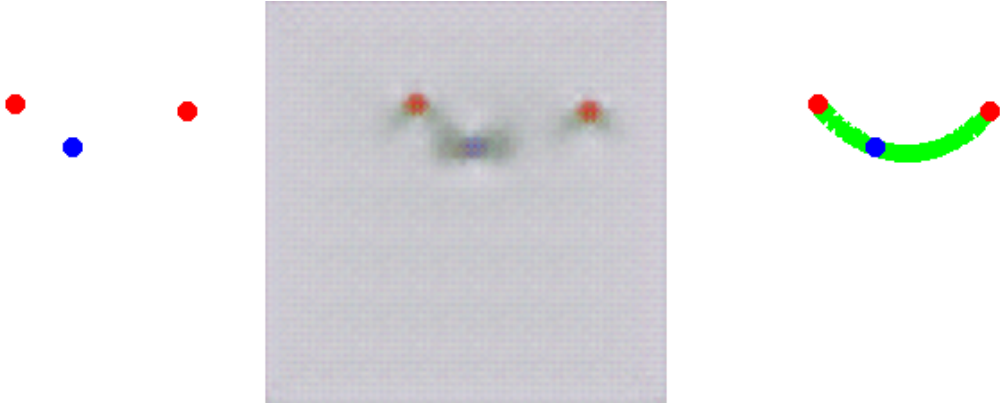
- **Input Space Size Estimation:** Estimated using Monte Carlo method to be approx. $9.3e9$.
- **Ratio of Training Set Size to Input Space Size:** Approx. $1.61e-5$

Task-Related Design Thoughts and Discussion:

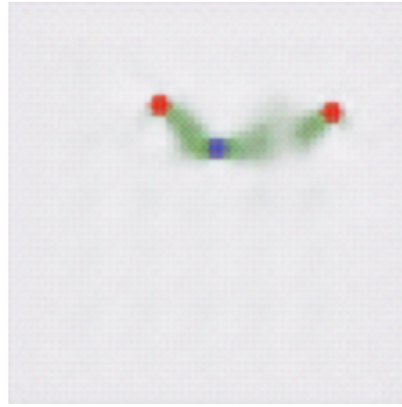
1. This task is intended to verify the physics-related simulation capability of this paradigm, implemented through an image reasoning model.
2. Although this problem has the possibility of being solved by interpolation, the model and hyper-parameter configuration used simultaneously solved other problems that are impossible to solve by interpolation, thus ruling out this possibility.
3. The reason for adding the restriction of a catenary passing point is mainly because setting the catenary to a fixed length or representing the catenary length by some other means felt less optimal than this setup. Furthermore, adding the passing point restriction also increased the input space, further ruling out the possibility of memorization or interpolation.
4. Some task parameters:
 - Image resolution: $224*224$

Experimental Results and Analysis: The final training results indicate that the model has completely learned the laws of the catenary. This shows that the learning capability of this paradigm has expanded from pure mathematical and logical rules to the simulation of fundamental physical laws of our real universe, but at the same time, the precision of such simulation needs further investigation. The model's learning speed is very fast, providing direct observation of the formation process of the machine mind. It can be seen that the learning process revealed in this example did not spend too much time on the trajectory of the catenary, but mainly on optimizing the image output. At the same time, it was noted that the training set I generated was flawed, with burrs in the output catenary images, while the output of the model after convergence was relatively smooth. This phenomenon is worth further study.

Left is input image, middle is model generated image, right is ground truth.



Catenary experiment: 200-step training-process visualization. Left: input image; middle: model-generated image; right: ground truth.



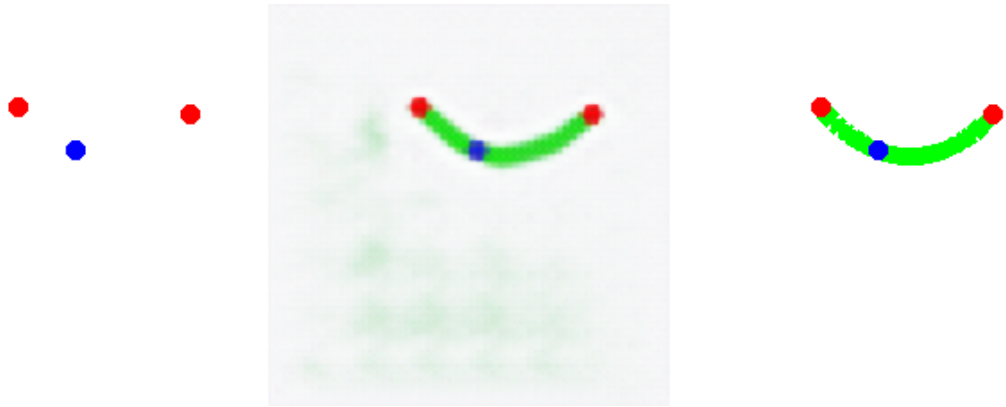
Catenary experiment: 400-step training-process visualization. Left: input image; middle: model-generated image; right: ground truth.



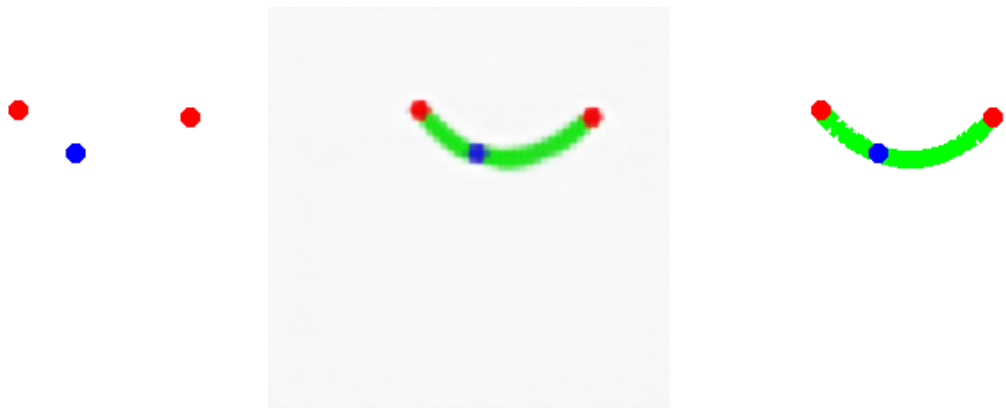
Catenary experiment: 600-step training-process visualization. Left: input image; middle: model-generated image; right: ground truth.



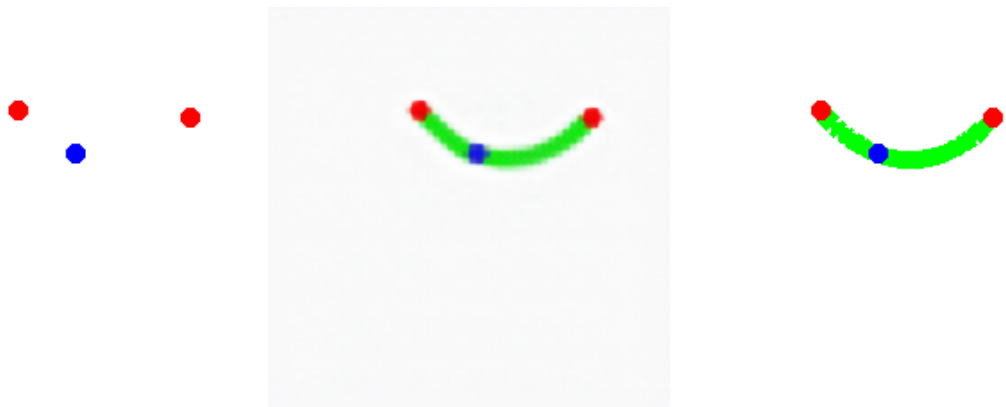
Catenary experiment: 800-step training-process visualization. Left: input image; middle: model-generated image; right: ground truth.



Catenary experiment: 1000-step training-process visualization. Left: input image; middle: model-generated image; right: ground truth.



Catenary experiment: 1200-step training-process visualization. Left: input image; middle: model-generated image; right: ground truth.



Catenary experiment: 1400-step training-process visualization. Left: input image; middle: model-generated image; right: ground truth.



Catenary experiment: 1600-step training-process visualization. Left: input image; middle: model-generated image; right: ground truth.

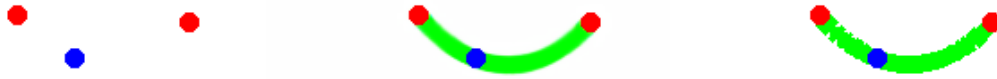


Catenary experiment: 1800-step training-process visualization. Left: input image; middle: model-generated image; right: ground truth.



Catenary experiment: 2000-step training-process visualization. Left: input image; middle: model-generated image; right: ground truth.

Due to the slowing down of changes later, the final training result is displayed directly.



Catenary experiment: 19000-step final training result. Left: input image; middle: model-generated image; right: ground truth.

4.6.2 Planetary Orbit Simulation

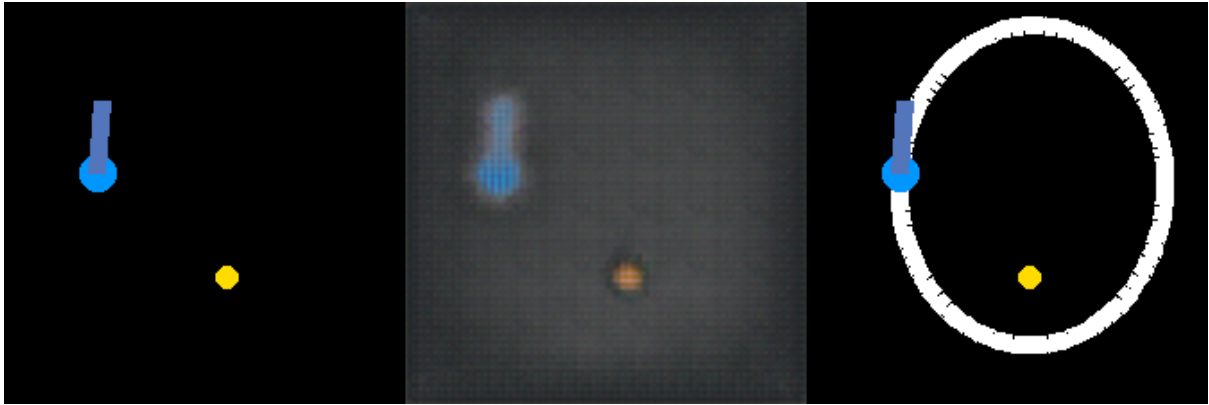
- **Task Description:** Given the fixed position of a star, the initial position of a planet, the magnitude of the initial velocity, and the direction of the initial velocity, predict the planetary orbit.
- **Input Format:** 224*224 image.
- **Output Format:** 224*224 image.
- **Loss:** MSELoss.
- **Training Configuration:** NVIDIA RTX 4090 GPU.
- **Training Set Generation Script:** `generate_orbital_path_from_initial_state.py`
- **Dataset:** `orbital_dataset_256_separated_v3` folder.
- **Training Code:** `train_image2image.py`
- **Model Used:** Swin Transformer + unet.
- **Training Set Size:** 150,000
- **Input Space Size Estimation:** Estimated using Monte Carlo method to be approx. $2.8e11$.
- **Ratio of Training Set Size to Input Space Size:** Approx. $5.36e-7$

Task-Related Design Thoughts and Discussion:

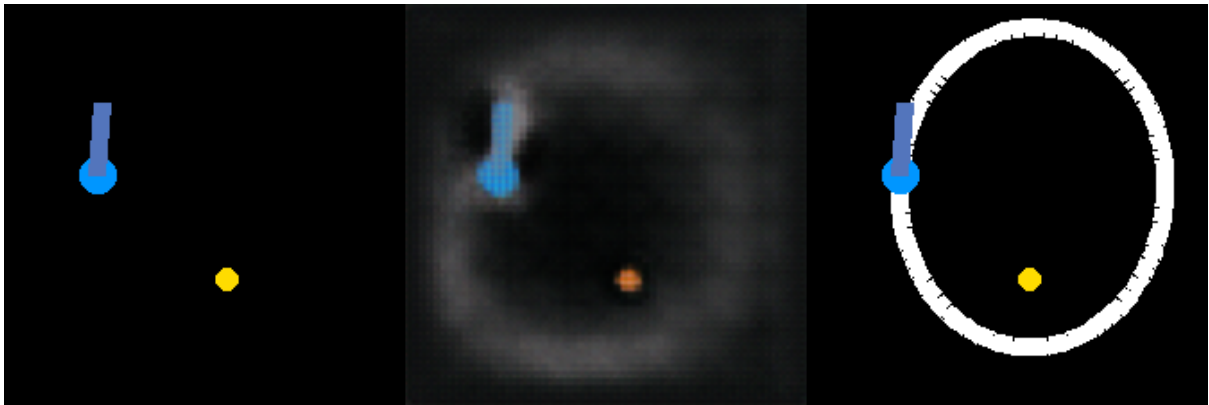
1. This task is intended to further verify the physics-related simulation capability of this paradigm, implemented through an image reasoning model.
2. Although this problem has the possibility of being solved by interpolation, the model and hyperparameter configuration used simultaneously solved other problems that are impossible to solve by interpolation, thus ruling out this possibility.
3. The fixed position of the star and the initial position of the planet are relatively easy to represent in the input image. Regarding the magnitude and direction of the planet's initial velocity, I represent it in this way: a short ray segment is emitted from the planet's initial position, with color representing the velocity magnitude and the direction of the segment representing the velocity direction. For the correspondence between velocity magnitude and color, see the training set generation script code.
4. Some task parameters:
 - Image resolution: 224*224

Experimental Results and Analysis: The final training results indicate that the model has completely learned the laws related to planetary orbits, but at the same time, the precision of such simulation needs further investigation. The model provides direct observation of the formation process of the machine mind. The learning process of this task reveals an interesting phenomenon: since the orbit in the output image is set to white, during the learning process, especially in the early stages, this whiteness diffuses in the output image, reminiscent of probability clouds in quantum mechanics.

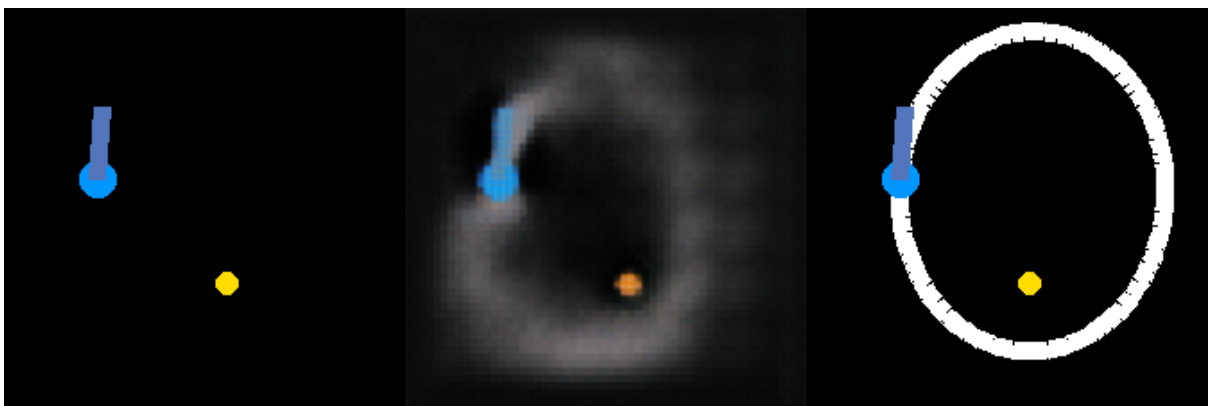
Left is input image, middle is model generated image, right is ground truth.



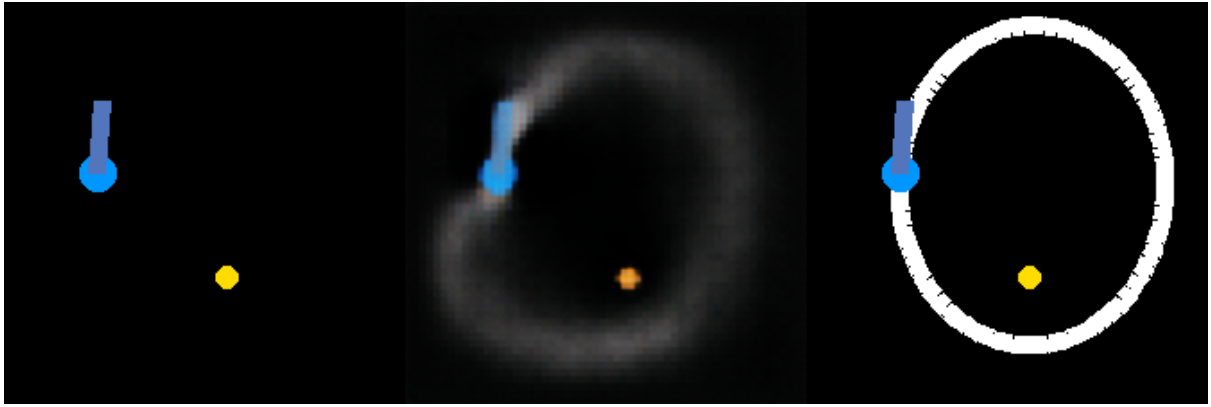
Planetary orbit simulation: 200-step training-process visualization. Left: input image; middle: model-generated image; right: ground truth.



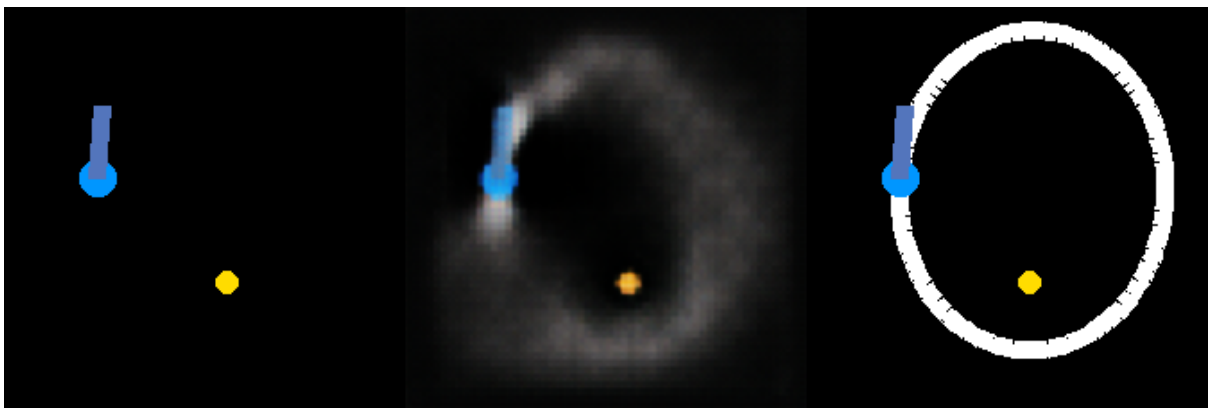
Planetary orbit simulation: 400-step training-process visualization. Left: input image; middle: model-generated image; right: ground truth.



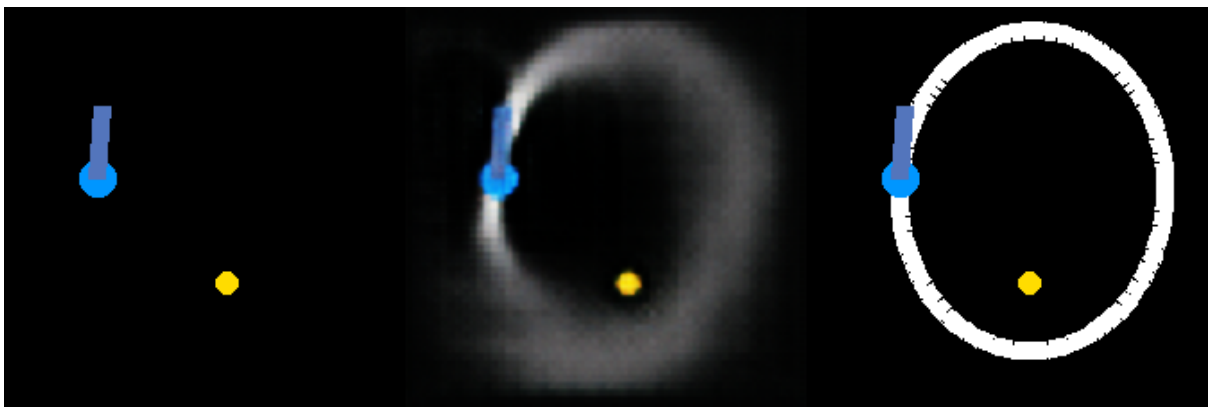
Planetary orbit simulation: 600-step training-process visualization. Left: input image; middle: model-generated image; right: ground truth.



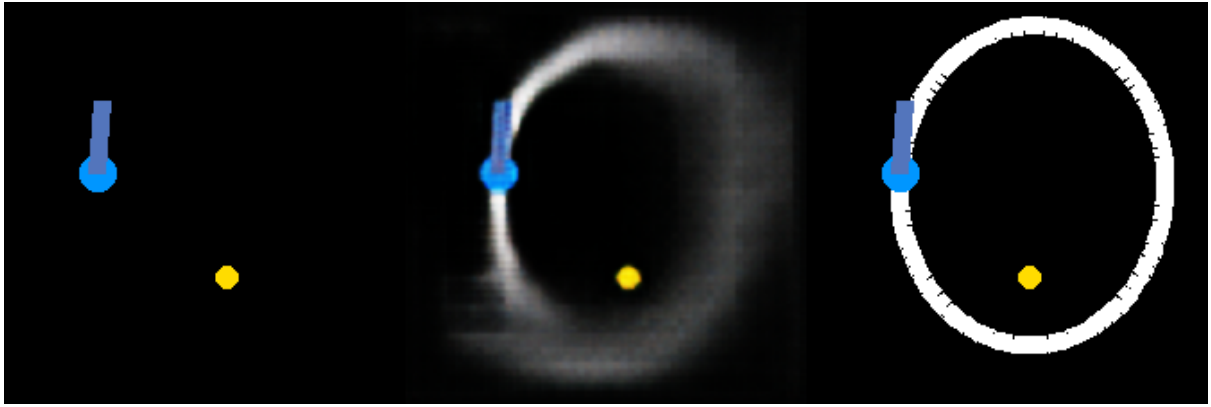
Planetary orbit simulation: 800-step training-process visualization. Left: input image; middle: model-generated image; right: ground truth.



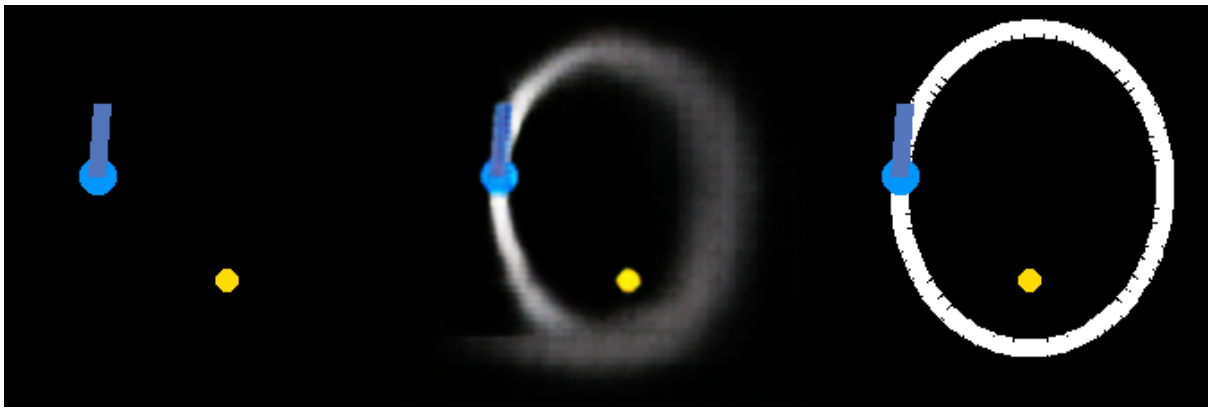
Planetary orbit simulation: 1000-step training-process visualization. Left: input image; middle: model-generated image; right: ground truth.



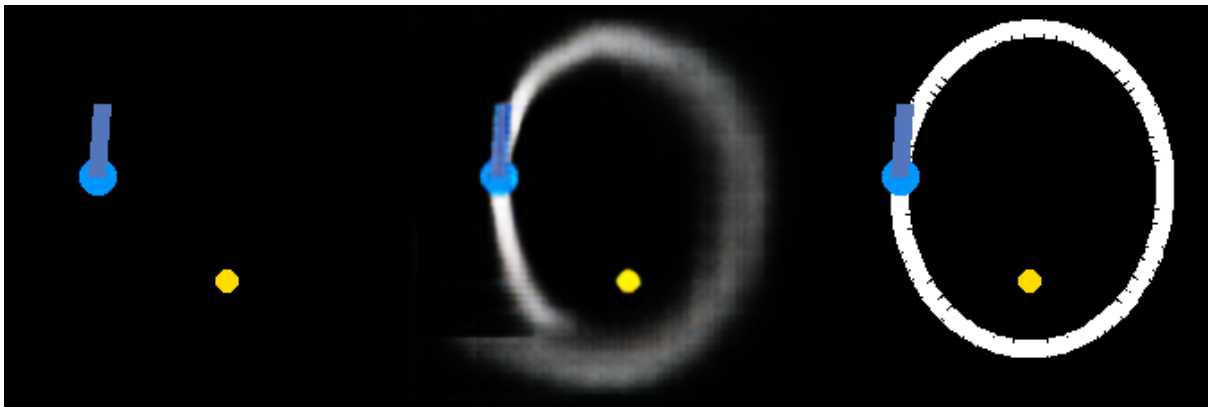
Planetary orbit simulation: 2000-step training-process visualization. Left: input image; middle: model-generated image; right: ground truth.



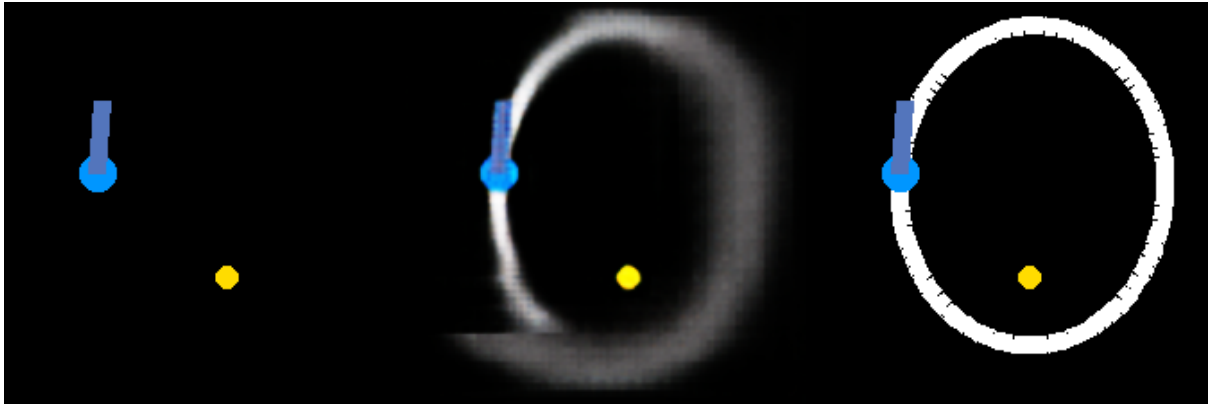
Planetary orbit simulation: 3000-step training-process visualization. Left: input image; middle: model-generated image; right: ground truth.



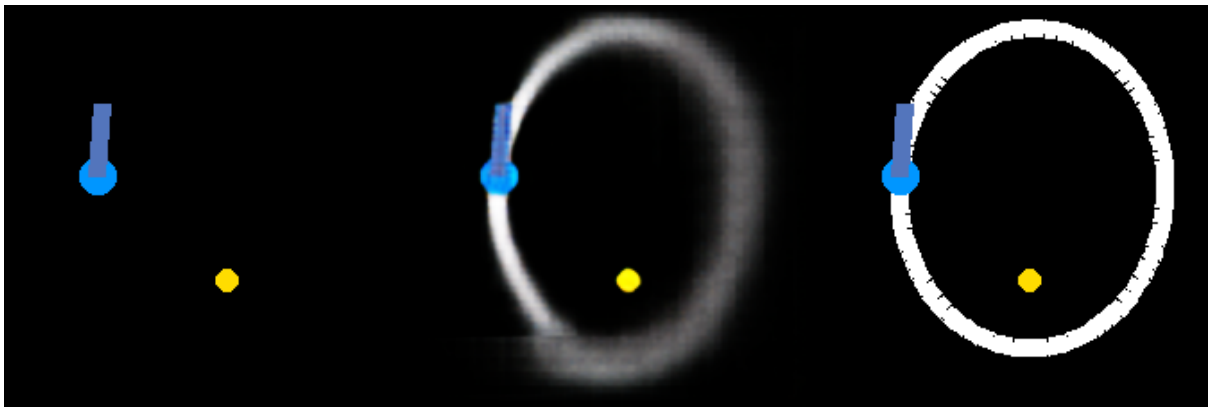
Planetary orbit simulation: 4000-step training-process visualization. Left: input image; middle: model-generated image; right: ground truth.



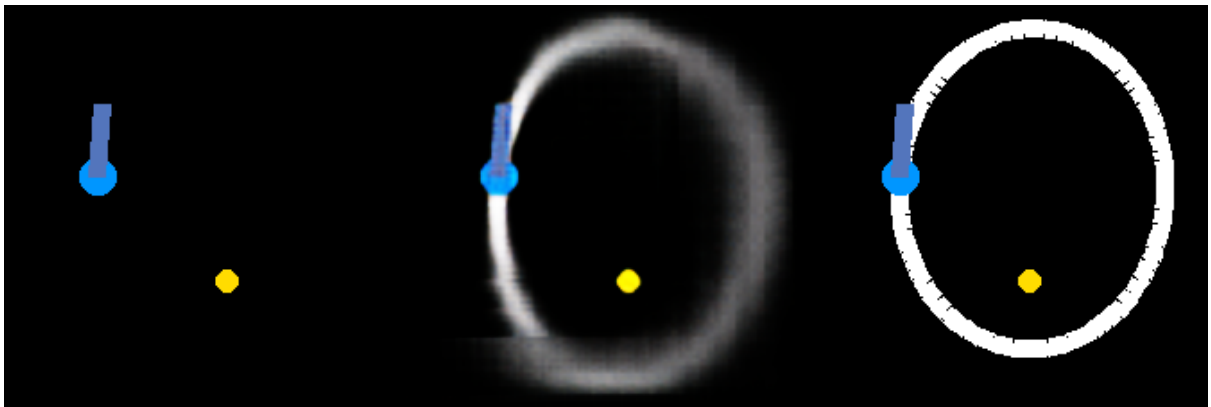
Planetary orbit simulation: 5000-step training-process visualization. Left: input image; middle: model-generated image; right: ground truth.



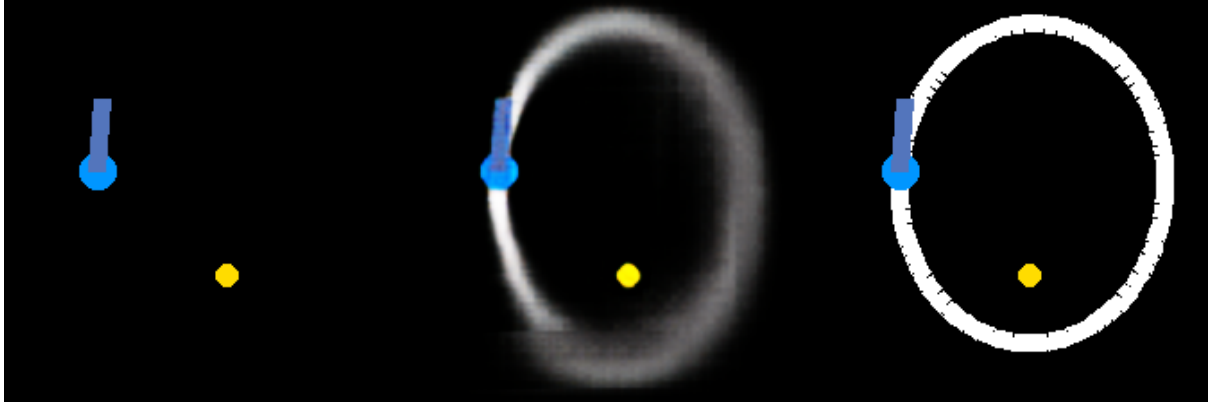
Planetary orbit simulation: 6000-step training-process visualization. Left: input image; middle: model-generated image; right: ground truth.



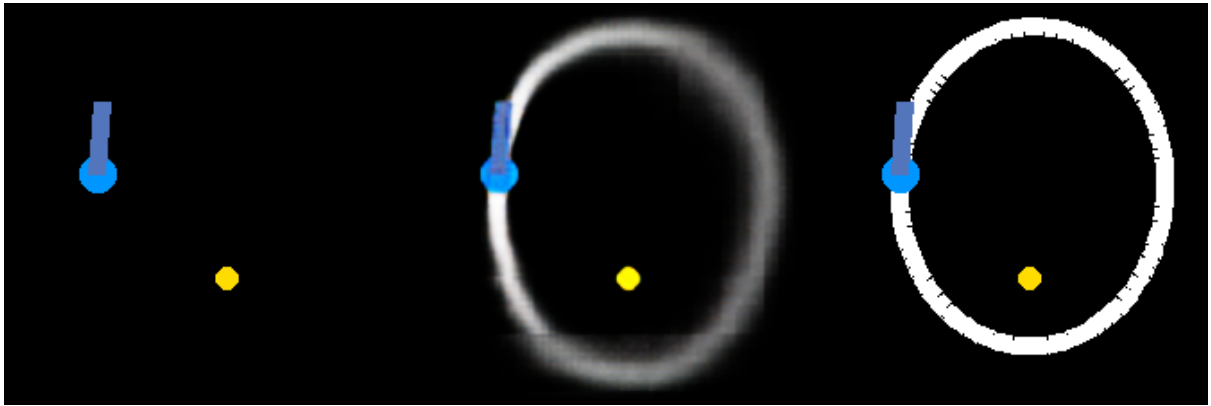
Planetary orbit simulation: 7000-step training-process visualization. Left: input image; middle: model-generated image; right: ground truth.



Planetary orbit simulation: 8000-step training-process visualization. Left: input image; middle: model-generated image; right: ground truth.

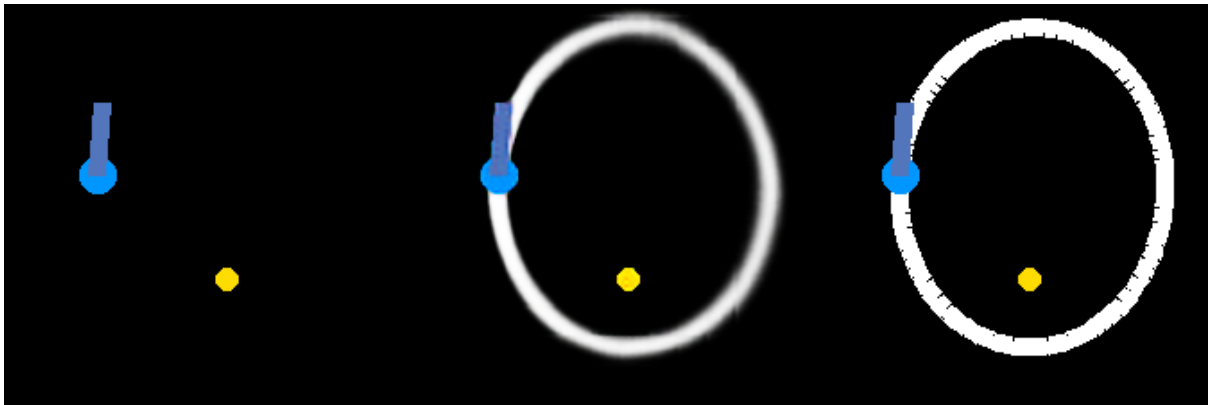


Planetary orbit simulation: 9000-step training-process visualization. Left: input image; middle: model-generated image; right: ground truth.



Planetary orbit simulation: 10000-step training-process visualization. Left: input image; middle: model-generated image; right: ground truth.

Due to the slowing down of changes later, the final training result is displayed directly.



Planetary orbit simulation: 156400-step final training result. Left: input image; middle: model-generated image; right: ground truth.

4.7 ARC-AGI Task Related

This section will demonstrate the capability of this paradigm in handling [12,13] arc-agi-1/2 tasks.

My Experimental Method Introduction:

1. The pattern of the arc-agi-1/2 dataset is to provide a very small number of examples, around 3, requiring the rules of the game to be comprehended from these examples, and generalization and

problem-solving to be completed on new examples. I tried using my method strictly according to the requirements of arc-agi-1/2, but it was impossible to complete, resulting in extremely severe overfitting. Therefore, I adopted a roundabout method. I first manually observed the logic of a task, assisted by interactive programming with LLMs (such as Gemini 2.5 Pro) to write and debug training data generation scripts for these complex tasks. Then I ran the script to generate a large dataset, generally around 150,000-200,000, and finally trained on this dataset and sought generalization.

2. Because my experimental method requires a large amount of time and energy, including time for manual observation of task logic and generation and modification of training set generation scripts, I only attempted it on 16 tasks: 8 arc-agi-1 tasks and 8 arc-agi-2 tasks, selected randomly and based on the principle that the training set generation script is relatively simple. The results were quite good; the vast majority of problems converged quickly, and a few problems required long-term training but could also converge. This universality is shocking, considering that I was randomly selecting 16 problems.
3. Adopted image reasoning mode, Swin Transformer + unet model.
4. Adopted the color image representation method from the arc-agi-1/2 dataset official website.
5. The horizontal and vertical dimensions of grid points were kept consistent throughout the dataset.
6. Generally, a dataset of 150,000 numbers is sufficient to complete training, but due to time and energy constraints, I did not try to find the minimum number required.

4.7.1 Fluid Simulation

- **Task Description:** From [ARC Prize - Puzzle #36a08778](#)
- **Input Format:** 224*224 image.
- **Output Format:** 224*224 image.
- **Loss:** MSELoss.
- **Training Configuration:** NVIDIA RTX 4090 GPU.
- **Training Set Generation Script:** `generate_arc_fluid_simulation.py`
- **Dataset:** `arc_fluid_final_dataset` folder.
- **Training Code:** `train_image2image.py`
- **Model Used:** Swin Transformer + unet.
- **Training Set Size:** 150,000
- **Input Space Size Estimation:** Using Monte Carlo method, running 2,000,000 times without duplicate samples.
- **Ratio of Training Set Size to Input Space Size:** As above, the training set size can be considered an extremely small fraction of the input space.

Task-Related Design Thoughts and Discussion:

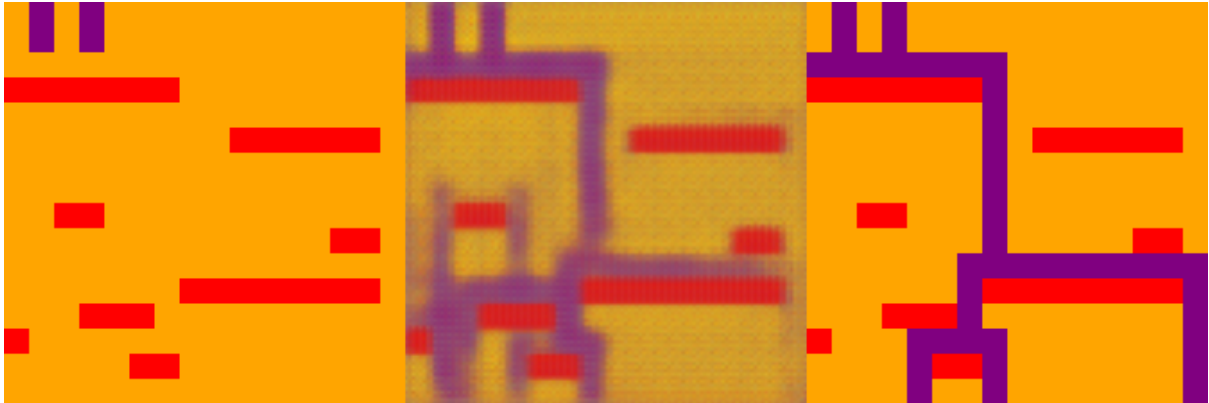
1. To further verify the pure image reasoning capability of the Transformer on existing arc-agi-1/2 datasets, I extended my attempts but adopted another roundabout method, as explained above.
2. Some task parameters:
 - Image resolution: 224*224

Experimental Results and Analysis: The final training results show that, under this programmatically generated isomorphic-data setting, the model can stably fit the rule of this task. It should be emphasized that this is not a direct solution to the official ARC-AGI few-shot setting. Rather, it shows that when task rules are programmatically expanded into a sufficiently large set of training samples, the model can learn the abstract transformation structure within them. The training process shows that the model first tends to learn simpler but incorrect local rules, and then gradually abandons these rules and approaches the true task rule. This provides a scalable experimental entry point for studying rule

learning in ARC-AGI-style tasks. Moreover, the reasoning nature of this task itself largely rules out the possibility that the model is performing naive interpolation.

Experimental Process Display:

Left is input image, right is ground truth, middle is generated image.



ARC-AGI fluid simulation: 200-step training-process visualization. Left: input image; middle: model-generated image; right: ground truth.



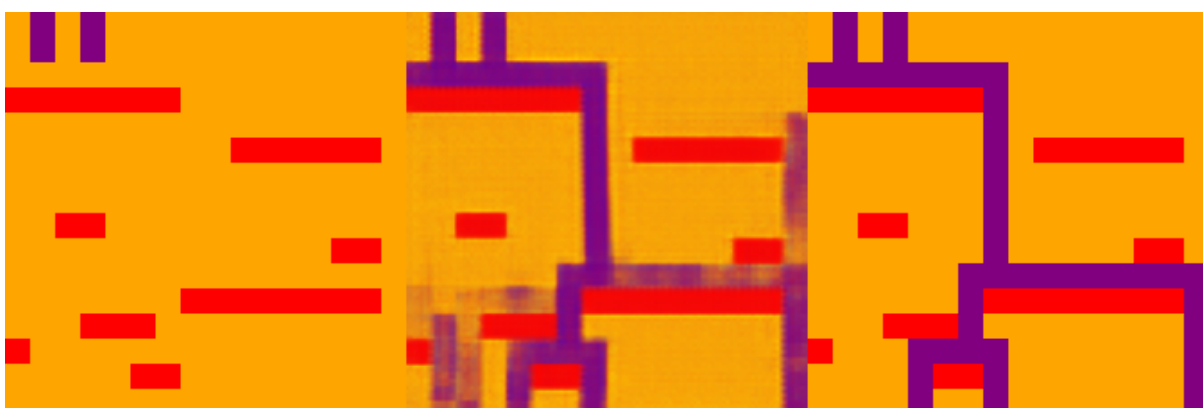
ARC-AGI fluid simulation: 400-step training-process visualization. Left: input image; middle: model-generated image; right: ground truth.



ARC-AGI fluid simulation: 600-step training-process visualization. Left: input image; middle: model-generated image; right: ground truth.



ARC-AGI fluid simulation: 800-step training-process visualization. Left: input image; middle: model-generated image; right: ground truth.



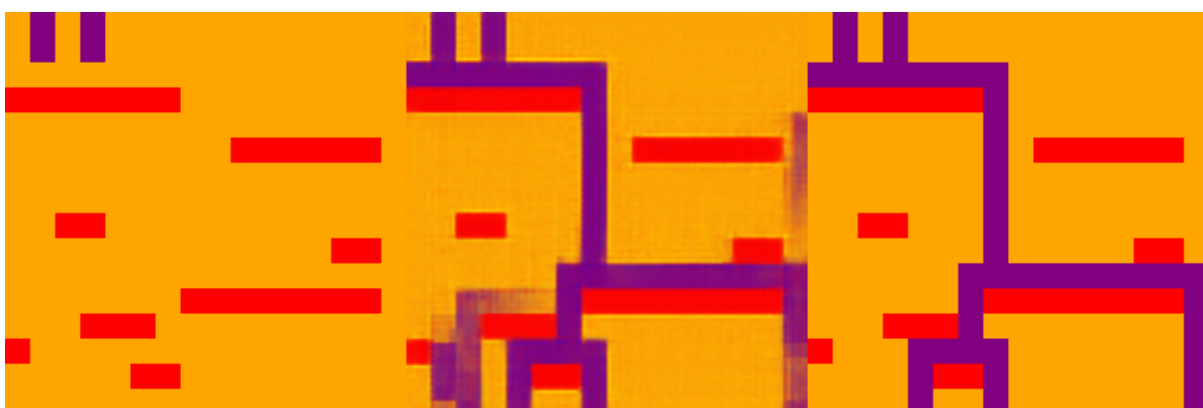
ARC-AGI fluid simulation: 1000-step training-process visualization. Left: input image; middle: model-generated image; right: ground truth.



ARC-AGI fluid simulation: 1200-step training-process visualization. Left: input image; middle: model-generated image; right: ground truth.



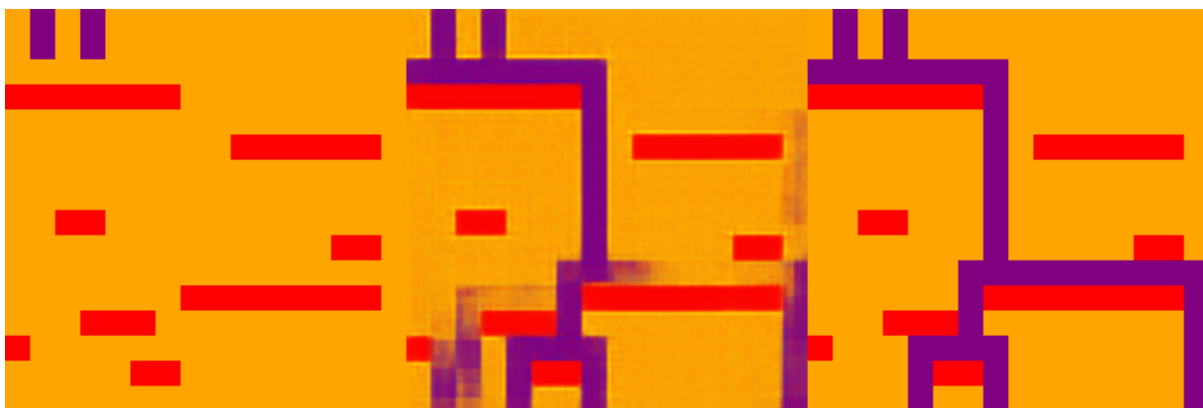
ARC-AGI fluid simulation: 1400-step training-process visualization. Left: input image; middle: model-generated image; right: ground truth.



ARC-AGI fluid simulation: 1600-step training-process visualization. Left: input image; middle: model-generated image; right: ground truth.



ARC-AGI fluid simulation: 1800-step training-process visualization. Left: input image; middle: model-generated image; right: ground truth.



ARC-AGI fluid simulation: 2000-step training-process visualization. Left: input image; middle: model-generated image; right: ground truth.



ARC-AGI fluid simulation: 3000-step training-process visualization. Left: input image; middle: model-generated image; right: ground truth.



ARC-AGI fluid simulation: 4000-step training-process visualization. Left: input image; middle: model-generated image; right: ground truth.



ARC-AGI fluid simulation: 5000-step training-process visualization. Left: input image; middle: model-generated image; right: ground truth.

Due to the slowing down of changes later, the final training result is displayed directly.



ARC-AGI fluid simulation: 39600-step final training result. Left: input image; middle: model-generated image; right: ground truth.

4.7.2 Block Puzzle

- **Task Description:** From [ARC Prize - Puzzle #898e7135](#)
- **Input Format:** 224*224 image.
- **Output Format:** 224*224 image.
- **Loss:** MSELoss.
- **Training Configuration:** NVIDIA RTX 4090 GPU.
- **Training Set Generation Script:** `generate_arc_jigsaw_puzzle_advanced.py`
- **Dataset:** `arc_jigsaw_puzzle_mine_dataset` folder.
- **Training Code:** `train_image2image.py`
- **Model Used:** Swin Transformer + unet.
- **Training Set Size:** 200,000
- **Input Space Size Estimation:** Using Monte Carlo method, running 1,000,000 times without duplicate samples.
- **Ratio of Training Set Size to Input Space Size:** As above, the training set size can be considered an extremely small fraction of the input space.

Task-Related Design Thoughts and Discussion:

1. To further verify the pure image reasoning capability of the Transformer on existing arc-agi-1/2 datasets, I extended my attempts but adopted another roundabout method, as explained above.

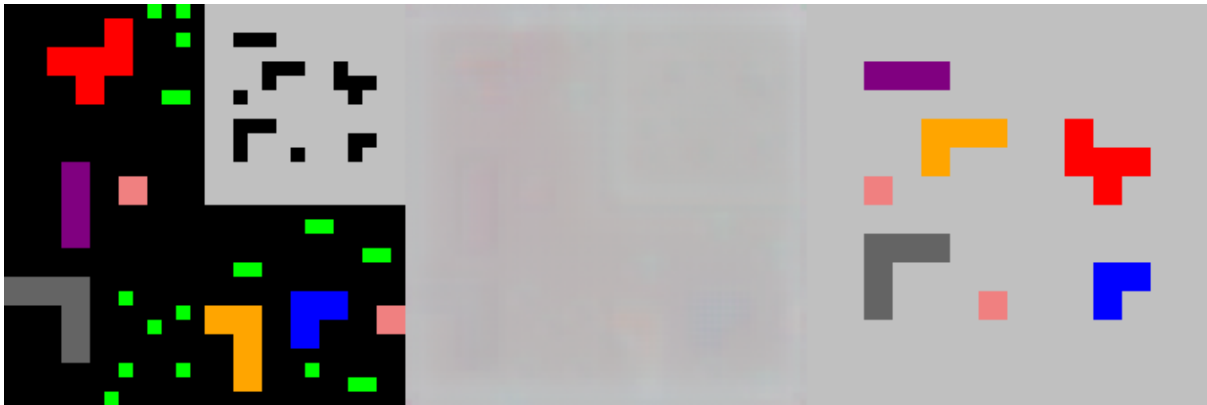
2. Some task parameters:

- Image resolution: 224*224

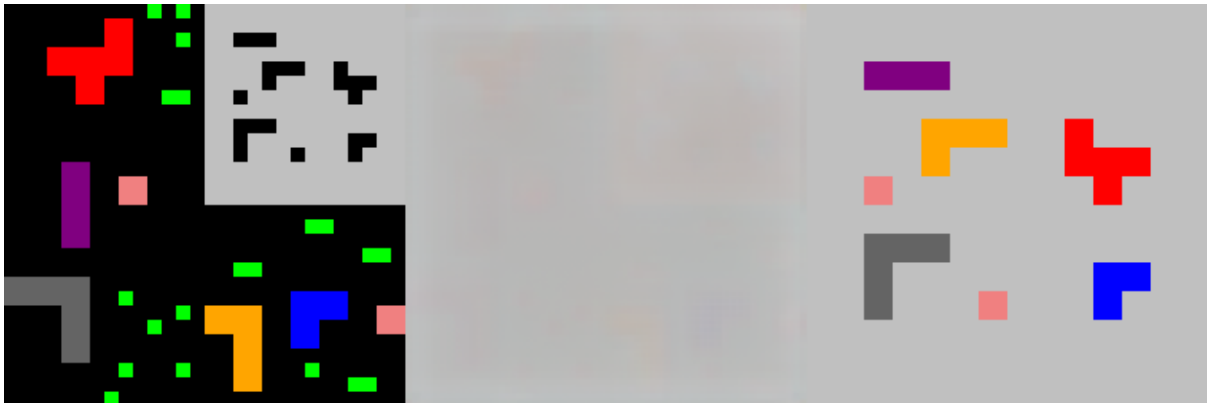
Experimental Results and Analysis: The final training results show that, under this programmatically generated isomorphic-data setting, the model can stably fit the rule of this task. As with the fluid task, this result is not equivalent to directly solving the official ARC-AGI few-shot setting; rather, it shows that abstract transformation tasks expanded programmatically can be learned by image models. The training process also shows a gradual learning process from background color, to shape and position, and finally to color-filling rules. Moreover, the reasoning nature of this task itself largely rules out the possibility that the model is performing naive interpolation.

Experimental Process Display:

Left is input image, right is ground truth, middle is generated image.



ARC block puzzle: 10000-step training-process visualization. Left: input image; middle: model-generated image; right: ground truth.



ARC block puzzle: 20000-step training-process visualization. Left: input image; middle: model-generated image; right: ground truth.



ARC block puzzle: 30000-step training-process visualization. Left: input image; middle: model-generated image; right: ground truth.



ARC block puzzle: 40000-step training-process visualization. Left: input image; middle: model-generated image; right: ground truth.



ARC block puzzle: 50000-step training-process visualization. Left: input image; middle: model-generated image; right: ground truth.



ARC block puzzle: 60000-step training-process visualization. Left: input image; middle: model-generated image; right: ground truth.



ARC block puzzle: 70000-step training-process visualization. Left: input image; middle: model-generated image; right: ground truth.



ARC block puzzle: 80000-step training-process visualization. Left: input image; middle: model-generated image; right: ground truth.



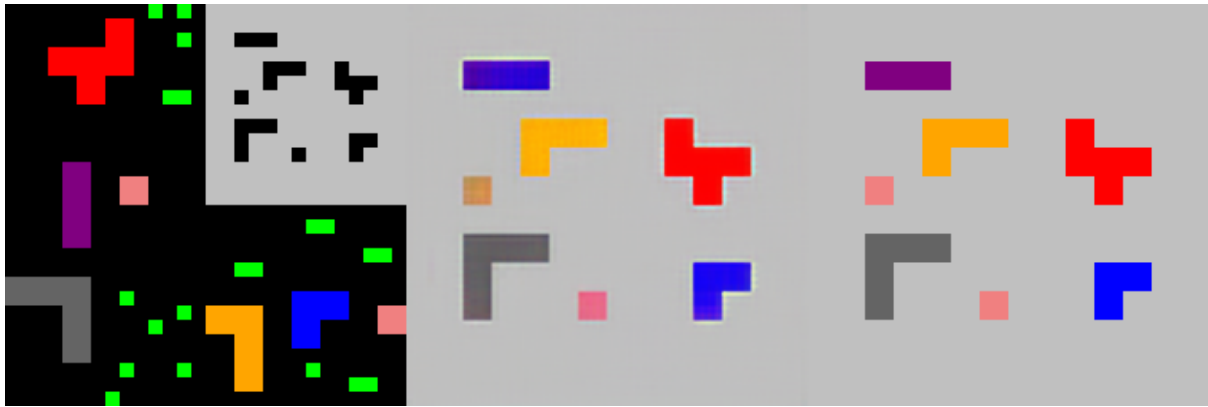
ARC block puzzle: 90000-step training-process visualization. Left: input image; middle: model-generated image; right: ground truth.



ARC block puzzle: 100000-step training-process visualization. Left: input image; middle: model-generated image; right: ground truth.



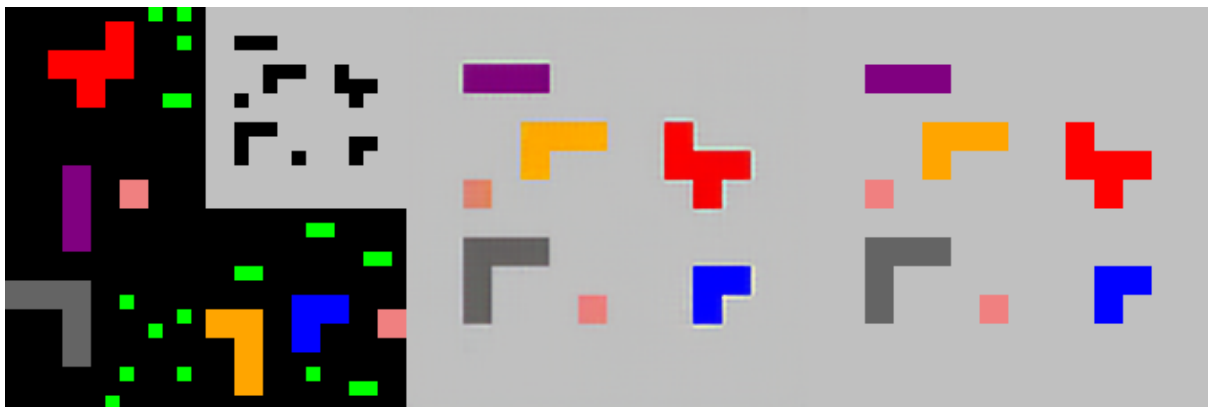
ARC block puzzle: 150000-step training-process visualization. Left: input image; middle: model-generated image; right: ground truth.



ARC block puzzle: 200000-step training-process visualization. Left: input image; middle: model-generated image; right: ground truth.



ARC block puzzle: 250000-step training-process visualization. Left: input image; middle: model-generated image; right: ground truth.



ARC block puzzle: 300000-step training-process visualization. Left: input image; middle: model-generated image; right: ground truth.

Due to the slowing down of changes later, the final training result is displayed directly.



ARC block puzzle: 530400-step final training result. Left: input image; middle: model-generated image; right: ground truth.

4.8 Interpretability Exploration

This section will demonstrate the capability of this paradigm to output reasoning processes.

4.8.1 Edit Process Output

- **Task Description:** The model learns to predict and output the shortest edit process between two 0/1 strings.
- **Input Format:** A 30-bit string composed of “0” or “1”. The first 15 bits represent string A, and the last 15 bits represent string B.
- **Output Format:** Multi-label binary classification format with 450 labels, equivalent to four hundred and fifty 0/1 strings. Every consecutive 30 bits form a group. The first 15 bits of a group represent the intermediate state of the string during the editing process, and the last 15 bits represent the mask of the string to distinguish cases where the string length is less than 15 bits during editing.
- **Loss:** Average Binary Cross Entropy Loss.
- **Training Configuration:** NVIDIA RTX 4090 GPU.
- **Training Set Generation Script:** `generate_edit_distance_explainable.py`
- **Dataset:** `edit_distance_path_unique_final.jsonl`
- **Training Code:** `train_tiny_transformer.py`
- **Training Set Size:** 500,000
- **Input Space Size Estimation:** $2 \text{ to the power of } 30 = 1,073,741,824$
- **Ratio of Training Set Size to Input Space Size:** 0.0466%

Task-Related Design Thoughts and Discussion:

1. This theoretically does not count as directly explaining the weights of neurons or the functioning of a module, but rather an interpretability guided by interpretable labels. The method of operation for this kind of interpretability is to directly add the internal processes that need to be observed as labels; practice shows that this method is practical. For example, for some problems, since the network can fit the answer, in many cases, I can be sure that it must also understand some internal states or intermediate processes in some way, and at this time, it can be exposed through this method. As for the original meaning of purer neural network interpretability, I believe this paradigm can achieve a cleaner laboratory environment to observe neural network weight changes, which will be discussed later.
2. Some task details:
 - The script ensures that no string length exceeds 15 during the editing process, otherwise my output format cannot represent it.

- And the script ensures that there is one and only one minimum editing path, otherwise it will greatly increase the difficulty of network convergence.
- The choice of edit distance strings is limited to two types, 0 and 1, to reduce task difficulty without changing the essence of the problem.

Experimental Results and Analysis: The evaluation loss converged to 0.001404. This success indicates that this paradigm can not only give answers but also has the potential to output problem-solving steps, providing a new, actionable idea for solving the neural network black box problem.

Final Convergence Results:

```
--- Step 66000 ---
Train Loss: 0.001995
Eval Loss: 0.001404
```

5 Act V: On the Boundaries of Learnability and Interpretability

I have summarized the characteristics of this paradigm:

1. Input and output formats should be well-designed. The input format should minimize the parsing burden on the transformer as much as possible. For output, it is preferable to use multi-class classification corresponding to cross-entropy loss or multi-label binary classification corresponding to average binary cross-entropy loss. You can design the meaning of the output format according to the characteristics of the task, as long as the format in the entire dataset is consistent. For example, if the output is a number, you can use multi-label binary classification to represent a binary number of corresponding bits.

2. Regarding learning difficulty:

Previously, when doing symbolic rule learning tasks, I observed a phenomenon that the structure of the task is also important. It seems that forward symbolic rule tasks are easy to learn, while reverse solving for inputs based on outputs is not easy to learn, unless this reverse is actually another form of forward symbolic rule. Now it seems that this is because such “reverse solving” tasks are more similar to algorithm simulation problems, so the learning difficulty is greater.

Another observed phenomenon is that tasks with branching conditional judgments may exacerbate learning difficulty, such as a certain indicator bit indicating the use of completely different rules to calculate subsequent operands. Thinking about it now, branching judgments are essentially rules themselves, but they just exacerbate the complexity of symbolic rules, making them harder to learn.

Additionally, positions can be left in the input to control task rules, i.e., Input = Task Rule + Operands. This idea has been successfully verified in cellular automata-related experiments.

Furthermore, another observed phenomenon is that tasks that serially mix too many simple logics are also difficult to learn. If a symbolic transformation rule serially combines different simple symbolic rules, even if each rule itself is very simply to learn, it may still combine into an extremely difficult task to learn. A simple example is the Trapping Rain Water experiment in the experimental section; by eliminating the final step of summing rain amounts and decoupling the problem, that task was able to converge quickly.

Forward symbolic rule learning has degrees of difficulty. For example, in 1D 30-bit cellular automata, the learning difficulty of different rules varies, and the learning difficulty of applying different layers of rules also varies. Basically, the more layers, the harder the learning. There are also some fundamentally unlearnable tasks, such as RSA encryption, which, although a simple forward mathematical calculation, is indeed impossible to learn.

Therefore, rule difficulty affects the speed of loss decline. Regarding this, to take a specific example, for cellular automata, Rule 110, the number of evolution layers—at least when there are few—strictly follows a law where the more layers, the more difficult the training, and the slower the model convergence. This brings to mind [15] Computational Irreducibility proposed by Wolfram; he indeed used the evolution of cellular automata as an example.

In addition, the training process itself is a kind of computational irreducibility; the process of training/sculpting cannot be omitted. Moreover, the simpler the problem, the faster the training; the harder the problem, the slower the training. If we do not consider whether the essence of the problem is friendly to neural networks or whether the input/output format is friendly to neural networks, and if we control the scale of inputs for different problems to be roughly consistent, the speed of loss decline during the training process can even be used to estimate the learnability or computational irreducibility of the problem. For example, I once tried to train RSA encryption, and the loss did not drop at all; nothing was learned from start to finish. Using cellular automata as an example here again, both input and output are 30-bit 01 strings, so the rule type and number of evolution layers can completely determine the mapping from input to output. But this is just one way of mapping from input space to output space. Why is this one type easier to learn than—mapping rules of cellular automata with more evolution layers? And also obviously easier to learn than a completely random permutation mapping? — A completely random permutation mapping is obviously unlearnable because it contains no patterns. This is a very interesting question, but exploring it is beyond my capability. Furthermore, the unlearnability of the RSA algorithm confirms that the simple algorithms created by cryptographers are very successful in this sense.

So, through this, it is possible to quantify task difficulty, learnability/computational irreducibility through the method of actual training tasks, this kind of Monte Carlo-like method. This may have potential in mathematics. Why are some rules deemed complex and others simple? And will this form a new theory?

Additionally, regarding model capacity, intuition suggests that larger/more suitable model capacity for the task should make training faster. Therefore, it might be the combination of model capacity and task difficulty that determines the speed and process of loss convergence during training.

3. Regarding the situation where transformation rules in the training set are inconsistent. Even if they are simple rules, if different data in the training set apply different rules, it is also difficult to learn, because there is no indication of which rule should be applied. The model can only try to achieve some balance based on the principle of reducing loss. If many rules are mixed but it cannot be identified which simple rule to use, then the training loss and eval loss will oscillate at a high point because different rules are pulling towards an equilibrium point. The level of this loss value will depend on the purity of the rules in the training set.

In addition, mod 3 is a surprisingly unlearnable rule. It is very simple, but the loss remains high. It may be related to that phenomenon of rule mixing, but at least from a human perspective, it intuitively does not look like a mixed rule. I conducted an experiment where the input is a 20-bit binary number, represented by a 20-bit 01 string, calculating its mod 3 result, represented by a 2-bit binary number, and the loss is mean bce loss. Now comes the interesting result. I generated four datasets. The first dataset has no restrictions on input. The second dataset restricts mod 3 not to be 0. The third dataset restricts mod 3 not to be 1. The fourth dataset restricts mod 3 not to be 2. The result of training is that their respective eval loss stabilized at the following results:

- Dataset without restrictions —— 0.636
- Restriction mod 3 cannot be 0 —— 0.693
- Restriction mod 3 cannot be 1 —— 0.347
- Restriction mod 3 cannot be 2 —— 0.347

Analyzing this result is very interesting. Let me first give a fact: if the model cannot learn anything

from the data, for a bce loss, if the probability of the label being 1 is p , then the model will reduce the loss to $-p \ln p - (1 - p) \ln(1 - p)$. Afterward, I will see that these loss results prove that the model did not learn anything from the data, i.e., mod 3 is unlearnable. The reasons are as follows:

For the dataset without restrictions, the probability of the high bit being 1 for the output 2-bit binary number is $1/3$, and the probability of the low bit being 1 is also $1/3$. Therefore, the optimal loss $= -\frac{1}{3} \ln(\frac{1}{3}) - \frac{2}{3} \ln(\frac{2}{3}) = 0.6365$, which is very close to the experimental result.

For the dataset restricting mod 3 not to be 0, the probability of the high bit being 1 for the output 2-bit binary number is $1/2$, and the probability of the low bit being 1 is also $1/2$. Therefore, the optimal loss $= -\frac{1}{2} \ln(\frac{1}{2}) - \frac{1}{2} \ln(\frac{1}{2}) = 0.693$, which is very close to the experimental result.

For the dataset restricting mod 3 not to be 1, the probability of the high bit being 1 for the output 2-bit binary number is $1/2$, and the probability of the low bit being 1 is 0. Therefore, the optimal loss $= -\frac{1}{2} \ln(\frac{1}{2}) = 0.347$, which is very close to the experimental result.

For the dataset restricting mod 3 not to be 2, the probability of the high bit being 1 for the output 2-bit binary number is 0, and the probability of the low bit being 1 is $1/2$. Therefore, the optimal loss $= -\frac{1}{2} \ln(\frac{1}{2}) = 0.347$, which is very close to the experimental result.

Therefore, it can be considered that, at least under the experimental setting used at that time, the model did not truly learn the mod 3 rule. This failure is more likely due to task representation and the training landscape, rather than merely insufficient fitting ability.

A similar phenomenon appears in the Tower of Hanoi experiment. If I only train on the states along the optimal path from the initial state to the final state for a fixed number of disks, and train the optimal action for those states, convergence is very rapid. But when I extend the training set to the entire Tower of Hanoi state space for the same fixed number of disks, the loss stays at a relatively high level and stops decreasing, likely not because of insufficient fitting ability. Comparing the entire Tower of Hanoi state space with the mod 3 problem, one can see that mod 3 in binary format has something like recursive or fractal structure, while Tower of Hanoi is even more naturally recursive or fractal. Is this because of input/output format design, or because of the problem itself? Does this paradigm fail to effectively identify rules involving recursion or fractals? I tend to think it is the latter. Then why does training on only the optimal path from the initial state to the final state converge rapidly? One possible explanation is that, on this optimal path, the task can be characterized by a simpler non-recursive recognition rule, which greatly reduces learning difficulty.

So, do these two problems related to recursion or fractals relate to the mixed rules mentioned at the beginning? This paper believes they might be related because when the model cannot identify recursive rules, it might perceive them as some form of complex mixed rules, and it cannot identify which input corresponds to which rule.

(Note: Both the Mod 3 problem and the full-state space Tower of Hanoi problem have been successfully solved. For the discussion on the Mod 3 problem, please refer to Section 8.1; for the full-state space Tower of Hanoi problem, please refer to the open-source code at `algorithms/generate_hanoi_decoupled_scaffold.py`. The shared core insight is that such recursive problems can be viewed as tasks with significant effective depth but lacking sufficient intermediate cues. Due to the nature of recursion, phenomena similar to gradient cancellation are prone to occur, making it difficult to establish the connection between inputs and outputs. The solution is to provide decoupled information as intermediate process hints.)

4. In this paradigm, data for many tasks can be generated by programs at low cost and at large scale. Theoretically, I need to make the training set identically distributed with the entire input space, so the simplest method is random sampling in the input space. But here comes another question: could there be another way to construct the training set that makes the model's learning effect better? For example, if a certain branch condition of the rule appears less frequently, random

sampling is difficult to cover enough patterns. My hypothesis is that perhaps there could be more ingenious ways of dataset generation to better cover various patterns of the problem. Because based on practical experience, as long as there is sufficient pattern coverage, the training set can achieve results, possess anti-overfitting capabilities, and meet the model’s learning needs; this is also why the total amount of the training set does not need to be very large. Analyzing from this perspective, the reason why the past deep learning paradigm of pattern recognition required large amounts of training sets becomes relatively clear: because those tasks do not have precise transformation rules, more data is needed to cover various patterns in the input space.

This paradigm usually does not take OOD generalization in the traditional sense as its main goal. More precisely, it asks whether, when the training data covers only a tiny fraction of the entire input space, the model can stably generalize to the vast remaining unseen input space under the same distribution. For tasks with a fixed input space and data generated from the same program distribution, this kind of generalization is analogous to generalization in traditional deep learning, but its boundary is clearer and its verifiability is stronger.

5. Controllable AI and Interpretability

Regarding interpretability, as explained above, this is not a direct explanation of individual neuron weights or the role of a module, but rather a form of functional interpretability guided by interpretable labels. The operational method is to directly add the internal processes one wants to observe into the labels. Practice shows that this method is useful. For example, in many tasks, since the network can stably fit the final answer, it is reasonable to infer that its internal representation very likely contains information related to intermediate states or intermediate processes. At that point, adding these intermediate processes as supervised labels may expose the originally hidden internal computation in the form of functional labels.

Regarding more controllable AI, because of the task form of this paradigm, the error modes of non-autoregressive models fitting precise rules are not the same as “hallucinations” in open-ended LLM generation. In practice, I have not observed similar hallucination-like phenomena. One possible reason is that this paradigm is based on rule tasks: the internal rules are extremely precise and fundamentally non-smooth, so the task cannot be solved merely by interpolation. Therefore, under sufficiently ideal data and training conditions, the model is more likely to show insufficient rule learning or insufficient precision, rather than groundless fabrication as in open-ended generation.

Another related finding is that the batch size during training cannot be too small. If the batch size is too small, learning significantly slows down. It is speculated that because the tasks in this paradigm are precise rules, the gradient directions of each sample vary more greatly, so a certain scale of batch size is needed to average the gradients.

6. In many experiments, training loss and validation loss can be observed to decrease together, and validation loss is often lower than training loss. This indicates that when the training set covers enough task patterns, the paradigm has strong resistance to overfitting. Conversely, if the task is difficult and key patterns are insufficiently covered, the model may still overfit or fail to converge.
7. What this paradigm learns is the structural relationship between symbols, rather than the human semantics of the symbols themselves. For the model, 0, 1, or an emoji can all simply be symbolic placeholders; what truly matters is the structural relation these placeholders form in the dataset. In other words, “structure” is the real “language”: what the model ultimately learns is not what 0 means, nor what 1 means, but the operation structure jointly satisfied by these symbols. The previous base-N addition and semantic-shuffling experiments support this point.

8. Regarding the overall philosophical implications of this paradigm

- **Data:** The data needs to cover the key patterns in the problem and should, as much as possible, follow the same distribution as the target input space. Whether strict identical dis-

tribution is necessary remains open to discussion, but random sampling is usually the most convenient and direct method. What truly matters here is not the absolute amount of data, but whether the pattern coverage is sufficient. Data is like a reference sample or model provided to the neural network. This process is somewhat similar to a Monte Carlo method: instead of explicitly writing down the solving logic, one gradually approximates the target mapping through large numbers of samples and training. The richer the data, the clearer and more concrete the “reference” provided to the neural network becomes. The more training there is, the better the neural network fits. From the perspective of data programming, the scale and quality of the dataset are key factors determining model performance.

- **Paradigm Metaphor:** It’s as if the neural network is carving the appearance of input-output patterns similar to data on a block of marble. Data is the model, and the neural network is the sculptor. Using computing power/training is like carving on marble; it uses continuity to approximate a deterministic symbolic transformation rule or fit an unknown but deterministic algorithm. Insufficient training, where loss has not yet dropped to the extreme, is like the carving not being finished. If the problem is too difficult, exceeding the capability of this neural network, it is like the capability required for this sculpture exceeds that of the sculptor. Even if training is insufficient or the problem is too difficult, the neural network also fits and effectively suppresses the uncertainty of the output space, just like an unfinished sculpture resembles the model more than the original marble.

The most interesting point of this paradigm is that it uses a continuous gradient-descent process to approximate discrete, deterministic, and verifiable symbolic rules or algorithmic mappings. This process itself may have mathematical significance worthy of further study. Furthermore, this paper argues that the paradigm has the potential to compress the uncertainty of problem outputs in cases where people have not yet found a general solution, although it may not be able to provide a precise algorithmic process. This means that the neural network is indeed learning a symbolic or algorithmic task, but it learns in its own unique connectionist way, rather than executing explicit algorithmic steps one by one as humans imagine. It faces a coupled loss and gradient-descent process, and is essentially only searching for a parameter configuration that can reduce loss. For example, on an algorithmic problem, if one tries to precisely correspond “which part of the trained weights is executing which step of the algorithm,” this paper argues that this is very likely impossible. The neural network does not know the position and meaning of input symbols in the human context; it faces only input-output constraints and the optimization target of gradient descent.

I now feel that this paradigm is consistent with what Demis Hassabis called advocating for the compression of structural intuition and Ilya’s belief that compression is intelligence. [1] AlexNet proved that handcrafted features were not good enough, and direct learning of image features using connectionism yields better results. This paradigm proves that even for precise symbolic rules, this route is correct to some extent. Moreover, as mentioned earlier, even on tasks that cannot fully converge, the model can often compress the uncertainty of the answer.

As for why this connectionist compression attempt might be effective. My feeling is that Demis Hassabis mentioned the complexity of the world is exceeding the carrying limit of human language and logic, but this explanation may have limitations. I am reminded of a lyric from Bob Dylan’s song “Things Have Changed”: *All the truth in the world add up to one big lie*. But this does not mean the failure of logic. My point of view is that every proposition in a long logical chain has its own unique context, and often when people combine them into a logical chain, they ignore these contexts to some extent, leading to the distortion of the final logical chain. Such scenes are also frequently seen in real life: a person is talking about his thoughts, and although every sentence he says seems correct and irrefutable, it eventually leads to an absurd result. This is where connectionism comes in handy, directly docking with real data and reality verification, recalibrating its own intuition and logic. This is like symbolic rules emerging from connectionism. Human thinking also accumulates observations and experiences first, then summarizes laws; induction of

rules comes first, then deduction. Deduction cannot replace induction. As mentioned earlier, if deduction is not treated rigorously enough, it becomes very fragile, and at this time, it is still necessary to resort to observations and experiences corresponding to connectionism as well as reality verification.

However, a purely end-to-end structural compression explanation may still be incomplete. For some recursive, long-chain, or highly coupled tasks, even when the underlying rule is deterministic, the model may still fail to establish a stable gradient path directly between the raw input and output. The decoupling experiments above show that introducing intermediate-process labels, structured representations, or other decoupled information can significantly change the training landscape and make problems that were originally difficult to learn become learnable. Perhaps this is precisely the role of logical cognition or structural insight: humans cannot cope with the world through intuition alone, but also need structural analysis to decompose complex problems into forms that are easier to learn and understand.

9. Exploring black boxes should be simpler through this paradigm. First, for trainable problems in this paradigm, the loss can drop to very close to 0. In this case, firstly, it is indeed parameters learned by gradient descent black box; secondly, unlike other image recognition and NLP, learned symbolic rules can be researched more traceably. Separately training different trainable problems on the same model, and then studying the weight changes of the neural network or the changes in verification output during the training process, etc., is easier to study in a sterile laboratory environment than the previous paradigms of pattern recognition images, NLP, and audio.

Moreover, what is more conducive to this point is that manually designing required task rules and generating datasets is very simple.

Simultaneously, experiments related to image reasoning can visualize the formation process of the machine mind. For example, in the task of drawing an incircle in a triangle, saving a current eval image every certain number of steps during training allows for a very intuitive view of the progress of the learning process and how the model might be learning the related task.

10. If this paradigm can realize greater potential in the future, it should be usable in some scenarios to learn the input-output laws of unknown systems: even when people temporarily do not know the complete working principle of a system, they can train a neural network on programmatic or experimental data so that it gives reliable predictions for new inputs. In this sense, it is somewhat similar to the [21] Software 2.0 mode described by Andrej Karpathy.

This method should play a role in AI for Science in the future, especially for exploring systems where large amounts of data can be generated or collected, but the explicit laws are not yet clear.

11. This paradigm can also be used for rule extraction: given input and output, let the model predict the underlying transformation rule. I have already conducted experiments on cellular automata tasks and obtained good results. However, in that experiment, I controlled the dataset generation process to ensure that a single input-output pair could uniquely determine the transformation rule and the number of evolution layers. Real-world problems often may not satisfy this condition of determining the rule from a single sample. Therefore, if one wants to extract rules from a large number of input-output samples, it may be necessary to design multi-sample aggregation or voting mechanisms, allowing the model to infer the shared rule from a set of samples.
12. If we can have some understanding of the relationship between model weights and fitted rules, is it possible to proactively manually design weights to simulate a certain rule? Of course, this is likely very difficult, even though clear symbolic rules make it look a bit simpler. So perhaps using training to obtain weights is still a better method. The value of this question may mainly lie in theory.

13. A phenomenon was observed during training: sometimes when the loss drops very close to 0, it suddenly jumps to a high loss. It is speculated that this might be caused by the non-smooth nature of this type of task.
14. A further idea is that this experimental workflow could be highly automated. Humans would only need to collaborate with AI to write training-set generation scripts; subsequent data generation, training, metric recording, and preliminary analysis could all be completed automatically. In this way, whether different tasks are learnable, how difficult they are to learn, and what representations or decoupling information they require would themselves form a new kind of experimental data for algorithm researchers, neural-network theorists, and even mathematicians to analyze further.
15. Regarding the limitation of synthetic data, it should be made clear that this paper does not make a fundamental structural innovation to the model itself; rather, it changes the data conditions and task form. Once the data becomes noisy, ambiguous, or polysemous real-world data, the ideal premises of the neural sculpting paradigm no longer fully hold. Therefore, this paradigm may not be directly applicable to all complex real-world problems. Its more important value may be to remind us that deep neural networks themselves have the potential to fit precise symbolic rules, and to provide a more controllable and cleaner experimental environment for studying neural network behavior. By further understanding the nature of neural networks in such an environment, we may gain deeper insight into real-world problems such as LLM hallucinations and CV model failures, and obtain clues for building more controllable and interpretable AI systems.

Furthermore, this paradigm might have more direct applications to some problems in the symbolic world.

16. Regarding the recent paper by Apple, [14] *The Illusion of Thinking: Understanding the Strengths and Limitations of Reasoning Models via the Lens of Problem Complexity*, in this paper, some problems where large models cannot reason reliably are narrated. I used my paradigm to train on 4 problems: Tower of Hanoi, Blocksworld, River Crossing, and Peg Solitaire, all achieving success, although the Tower of Hanoi problem has some subtleties mentioned above, and the convergence of the Blocksworld problem is also not perfect enough. See the open-source code for details.

Additionally, I noticed some opposing voices at the time, saying that inability to reason normally was due to token limitations. So, for the Tower of Hanoi problem, I designed a format 1>2 meaning moving the top disk of the first pillar to the second pillar, separated by semicolons. Then I asked the large model to reply with the solution to the Tower of Hanoi problem in this format. The result found that basically $n=6$ is the limit; going higher will cause problems. So I believe that large models indeed cannot effectively reason about these problems without calling tools (such as python). I have also open-sourced the script for checking whether the answer in this format is correct, see `eval_hanoi.py` in the code repository.

17. Rethinking the meaning of this paradigm: This paradigm seems surprisingly similar to the situation in the field of computer vision before the appearance of [1] AlexNet. At that time, mainstream methods relied on handcrafted features (such as [17] SIFT, [18] HOG), which can be seen as a kind of “prior symbolic knowledge” about the visual world. The success of AlexNet lies precisely in replacing this manual bias with end-to-end learning (induction). In the field of reasoning, current neuro-symbolic computing, whether modifying architecture or embedding logic, is essentially injecting human understanding of the reasoning process as a prior knowledge into the model, and this predetermined structure, this inductive bias, may be untrue or inappropriate.
18. Regarding the philosophical implication of this paradigm: I feel that even if it does not refute the prior existence of concepts, it can powerfully prove that concepts can grow spontaneously from experience. As for further discussion, due to personal capability limitations, I can only stop here.

19. There is a very clear correspondence between decoupling and [22] chain of thought or [23] curriculum learning, which may indicate that the profound meaning of this concept is not unique to the tasks studied in this paper.

6 Act VI: Universality Beyond Architecture

In this section, I explore whether other deep learning models have capabilities similar to those of the Transformer. I explored five types of models: MLP, RNN/LSTM, CNN, UNet, and diffusion models. The experiments show that the ability to fit precise rules is not unique to Transformers, but is a more general intrinsic potential of deep neural networks. At the same time, the specific architecture still matters greatly: if the network structure weakens or loses key information during propagation, or if its inductive bias cannot adapt to the long-range dependency structure of the task, this potential may fail to appear in a concrete task. Therefore, architecture does not determine whether this capability exists, but strongly affects whether it can be trained out, and with what efficiency and convergence path.

I used MLP and RNN/LSTM to fit the symbolic versions of cellular automata and the Trapping Rain Water algorithm problem. Of course, the input here is no longer token-like as in Transformers, but real numbers. CNNs were used to fit cellular automata datasets and image-version Trapping Rain Water datasets where inputs are images and outputs are symbols. UNet and Diffusion were used to fit cellular automata datasets where both inputs and outputs are images.

Dataset Generation Scripts:

- Symbolic Cellular Automata: `generate_cellular_automata_1d.py`
- Symbolic Trapping Rain Water: `generate_trapping_rain_water_decoupled.py`
- Image Cellular Automata: `generate_cellular_automata_image_and_label.py`
- Image Trapping Rain Water: `generate_trapping_rain_water_image_to_symbol.py`

6.1 Cellular Automata

Dataset Introduction:

All are 1D 36-bit cellular automata datasets, dataset size is 300,000, evolution rule is 110, evolution layer is 2. The only exception is the MLP model; possibly because it is too suitable for the cellular automata task and has overly strong fitting ability, I increased the evolution layer to 6.

Model structures can be found in the open-source code.

6.1.1 CNN Model

Training Code: `train_convnext.py`

I adopted the [5] ConvNeXt model. Here is the training process:

step	Training Loss	Validation Loss	Validation Bit Acc	Exact Match
1000	0.49467600	0.27210647	84.55833333%	0.10000000%
2000	0.11814510	0.05764289	97.36018519%	36.80000000%
3000	0.04807972	0.03248511	98.63425926%	56.20000000%
4000	0.00763491	0.00266492	100.00000000%	100.00000000%
5000	0.00186690	0.00128128	100.00000000%	100.00000000%

It can be seen that convergence was completed at an extremely fast speed, and 100% exact-match accuracy on the validation set was reached at 4000 steps. This indicates that, at least on this problem, CNN also has symbolic rule learning capability.

6.1.2 MLP Model

Training Code: `train_mlp.py`

For a giant MLP, the cellular automata dataset with an evolution layer of 2 converged too quickly, so I had to regenerate a dataset with an evolution layer of 6. Facts proved that it also reached 100% accuracy of exact match on the validation set very quickly at the end of 5 epochs.

Preliminary analysis suggests that because MLP preserves the hyper-distance connections of neurons too well, it may not be inferior to Transformer on this problem.

6.1.3 RNN Model

Training Code: `train_lstm.py`

Here are some small designs. First, I must input the input state of the cellular automata completely at the first time step, rather than inputting one bit per time step. Second, I cannot adopt autoregressive output, so I must take the output of a certain time step as the result. Third, I finally deliberately chose a time step different from the evolution steps of the cellular automata. Since the evolution steps of the cellular automata are 2, I chose the output time step of the RNN model to be 3. Although completely consistent time steps and evolution steps are definitely the most ideal model design, what I want to prove here is my previous viewpoint: the model does not execute tasks according to the logic or algorithmic steps usually assumed by humans. Under this experimental setting, the model is forced to learn a transformation function where repeating the transformation 3 times continuously equals the change of cellular automata rule 110 twice. The learning success of this setting can provide strong support for my viewpoint on this paradigm.

The experimental results also prove that convergence was completed at 1000 steps, reaching 100% accuracy of exact match on the validation set.

Subsequently, I changed the 3 time steps to 5/7/9, which can also reach 100% accuracy of exact match on the validation set in a very short time.

6.1.4 UNet Model

Training Code: `train_unet.py`

Using MSELoss, but it quickly got stuck at a high position of 0.0915 and stopped decreasing completely. It might be that the network architecture itself is not suitable for this task, pending further analysis.

Training Code: `train_unet_v2.py`

Using MSELoss, after several hours of training, MSELoss dropped below $1e-5$, achieving complete fitting at the visual level, and the precise matching accuracy for all pixel levels reached over 99%.

The difference in performance between the two versions of UNet may reflect that the information-propagation mechanism or inductive bias of the first version cannot adapt to this task. This phenomenon does not deny the intrinsic potential of neural networks to fit precise rules; instead, it shows that this potential needs suitable architectures and information pathways to become visible. If the structural design weakens or even loses key information during propagation, the model may fail to fit certain rule tasks. Once the information pathway and inductive bias better match the task structure, training efficiency and the convergence path can change significantly.

6.1.5 Diffusion Model

Training Code: `train_diffusion.py`

After several hours of training, the fitting was finally completed. Although this architecture eventually fitted this task, there is a huge difference in fitting efficiency compared to Swin-UNet, again supporting our conclusion that inductive bias determines significant differences in learning efficiency and convergence paths.

6.2 Trapping Rain Water

Task Description: Derived from the classic LeetCode algorithm problem, [42. Trapping Rain Water - LeetCode](#)

The format adopts the same decoupled output format as in the experimental section above. The difference is that the number of bars is adjusted to 12, so both input and output have $12 \times 3 = 36 = 6 \times 6$ bits. This facilitates inputting a 6×6 black-and-white checkerboard image for CNN-related experiments.

6.2.1 MLP Model

Training Code: `train_mlp.py`

Best Result: Validation Loss: 0.0002, Bit Acc: 99.99%, Exact Match: 99.90%

6.2.2 RNN Model

Training Code: `train_lstm.py`

I chose a time step of 3 here.

Best Result: Validation Loss: 0.0001, Bit Acc: 100.00%, Exact Match: 100.00%

6.2.3 CNN Model

Training Code: `train_convnext.py`

I adopted the [5] ConvNeXt model.

Best Result: Epoch 6 | Training Loss: 0.00003849 | Validation Loss: 0.00001896 | Validation Bit Accuracy: 100.00000000% | Validation Exact Match Rate: 100.00000000%

6.3 Summary and Analysis

I used cellular automata experiments to examine symbolic rule deduction, and the Trapping Rain Water experiment to examine algorithm fitting. This series of experiments shows that this paradigm is not exclusive to Transformers; many other models have similar capabilities. This supports a more general judgment: precise rule learning is likely not an accidental property of a specific architecture, but an intrinsic capability that connectionist deep learning neural networks can reveal under suitable data conditions. Moreover, Transformer may not necessarily be the best model; for different problems within this paradigm, the best model may also differ. Due to time and energy constraints, I am personally unable to explore this further here.

7 Act VII: Bridging the Gap: The Continuum from Ideal Rules to the Real World

In this section, I mainly discuss two issues. In all the content discussed above, I focused my energy on datasets with precise transformation rules. So in the first part of this section, I will introduce random perturbations to the input and output of datasets with precise rules and observe the training progress and final loss convergence.

In the second part, I designed an experiment to show that neural networks' rule learning capability and continuous-value interpolation capability are not contradictory, but can appear simultaneously and become coupled within the same task.

7.1 Perturbation of Precise Rules

I chose the one-dimensional cellular automata task, with evolution rule 110, evolution layer 2, and the number of bits of the one-dimensional cellular automata selected as 30. The specific perturbation practice

is to randomly flip the input or output with a certain probability during the data generation process (i.e., $1 \rightarrow 0$ or $0 \rightarrow 1$). For simplicity, I only tested cases of flipping input alone or flipping output alone. The validation set uses the precise rule cellular automata dataset without random flipping.

Dataset Code: `generate_cellular_automata_1d_perturbed.py`

Training Code: `train_tiny_transformer.py`

7.1.1 Random Perturbation Test on Output

Random Flip Probability	Stable Eval Loss Value
0.1%	0.000692
1%	0.008490
10%	0.094094

7.1.2 Random Perturbation Test on Input

Random Flip Probability	Stable Eval Loss Value
0.1%	0.001895
1%	0.021377
10%	0.230316

7.1.3 Summary and Discussion

It can be seen that whether it is input or output perturbation, the difficulty of task learning increases with the increase of perturbation probability, which is also very natural. Additionally, input perturbation has a greater impact than output perturbation. This may be due to the inherent non-smooth nature of rule tasks themselves. It can be guessed that if the evolution layer is increased, input perturbation will make task learning even more difficult. So, roughly speaking, there may be a continuum from noise-free precise transformation rule datasets to real-world datasets.

7.2 The Unity of Interpolation and Rule Learning

This section uses an experiment to show that neural networks' rule learning capability and continuous-value interpolation capability are not mutually exclusive, but can emerge in a coupled way within the same training process.

7.2.1 Experiment Description

To verify the ability of neural networks to fuse discrete rule learning and continuous value processing in a single task, I designed a logic-perception hybrid experiment. This experiment is based on the cellular automata image generation task. In the input and output images, the 1D 36-bit cellular automata state is arranged in row-major order into a 6*6 checkerboard grid image, where black represents 1 and white represents 0. But the following modifications were made to the input and output:

- **Input End:** The input image is no longer pure black and white blocks, but the grid points representing logical black and white are replaced by colored blocks taking random values within two different grayscale intervals, thereby introducing continuous perceptual information.
- **Output End:** The output image generated by the model is required to satisfy two conditions simultaneously:

- **Logical Correctness:** The black and white patterns of the grid points must strictly follow the multi-step evolution rules of the cellular automata.
- **Perceptual Relevance:** The final greyscale value of each grid point must be precisely calculated based on its corresponding input grid point's original greyscale value and a simple conditional function (preserve if white, invert if black). Specifically, assuming the input grid point's greyscale value is x , if the logical pattern of the output grid point is the same as the input grid point, the output grid point's greyscale value remains x ; otherwise, it is $255 - x$.

This design ingeniously forces the model to simultaneously look through the continuous greyscale values of the input to execute discrete logical reasoning, and remember these greyscale values to complete the final continuous value mapping, thereby testing its two core capabilities of rule learning and interpolation simultaneously in an indivisible task.

Dataset Code: `generate_cellular_automata_1d_to_grid_image_interp.py`

Training Code: `train_image2image.py`

Loss: MSELoss

Some task parameters:

Grid dimension: 6×6

Grid size: 40×40 pixels

Image resolution: 240×240

Cellular automata rule: rule 110

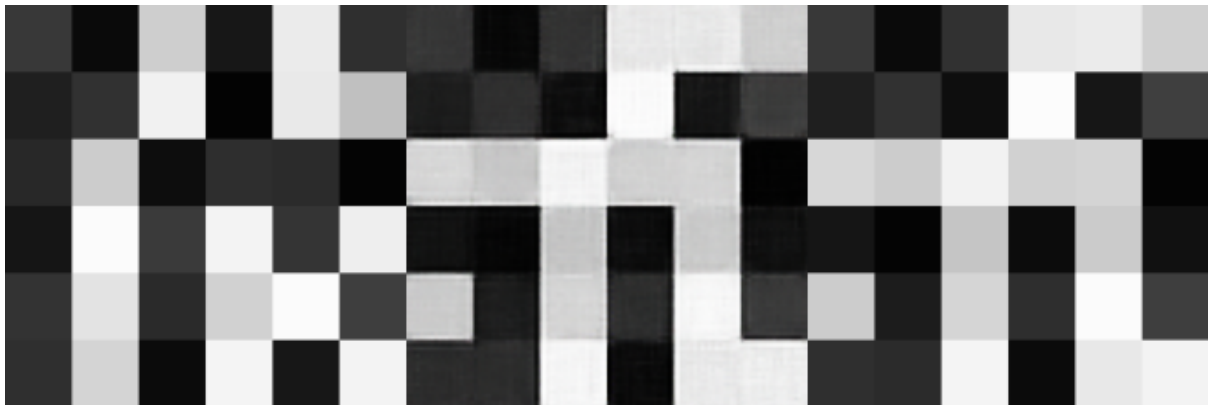
Rule evolution layers: 2

7.2.2 Experimental Results

After 3200 steps of training, the validation set mseloss had already reached 0.000899.

Here is the eval image at 3200 steps.

Left is input image, right is ground truth, middle is generated image.



Cellular automata interpolation experiment: 3200-step evaluation image. Left: input image; middle: model-generated image; right: ground truth.

7.2.3 Summary and Discussion

It can be seen that this experiment achieved great success. It strongly supports the judgment that continuous-value processing and rule learning can both emerge as intrinsic capabilities of neural networks under suitable data conditions, and in this task they are not simply parallel, but coupled together by the same training process. The model is not first understanding the cellular automata evolution rule in human-readable steps and then separately remembering the greyscale value of each grid point; what it is actually doing is still gradient descent, approximating both a discrete rule and a continuous-value mapping in a connectionist way.

7.3 The Unity of Pattern Recognition and Rule Learning

7.3.1 MNIST + Cellular Automata: From Pattern Recognition to Rule Execution

This section uses an experiment to show that the rule learning capability and pattern recognition capability of neural networks are not mutually exclusive, but can be coupled in the same task.

7.3.1.1. Experiment Description To verify the ability of neural networks to fuse discrete rule learning and pattern recognition in a single task, I designed an experiment that can prove both pattern recognition and symbolic reasoning are intrinsic capabilities of neural networks. This experiment is based on the cellular automata image generation task. In the input and output images, the 1D 36-bit cellular automata state is arranged in row-major order into a 6*6 checkerboard grid image, where black represents 1 and white represents 0. But the input was modified as follows: now I replace the input black grid points with the digit 1 from the MNIST dataset, and white grid points with the digit 0 from the MNIST dataset. It can be anticipated that it should not converge perfectly, but I can compare the eval loss of the cellular automata representation of the input position (i.e., evolved 0 layers) (all eval loss here is due to pattern recognition) with the eval loss after evolving, say, 3 layers, to confirm how much extra eval loss the evolution of the cellular automata position brings. The output format remains unchanged, still black and white checkerboard grid points.

This design ingeniously forces the model to simultaneously manifest pattern recognition capability to understand the initial cellular automata position, and execute discrete logical reasoning through symbolic reasoning capability, thereby testing its two core capabilities of rule learning and pattern recognition simultaneously in an indivisible task.

Dataset Code: `generate_mnist_ca_110.py`

Training Code: `train_image2image.py`

Loss: MSELoss

Some task parameters:

Grid dimension: 6×6

Grid size: 40×40 pixels

Image resolution: 240×240

Cellular automata rule: rule 110

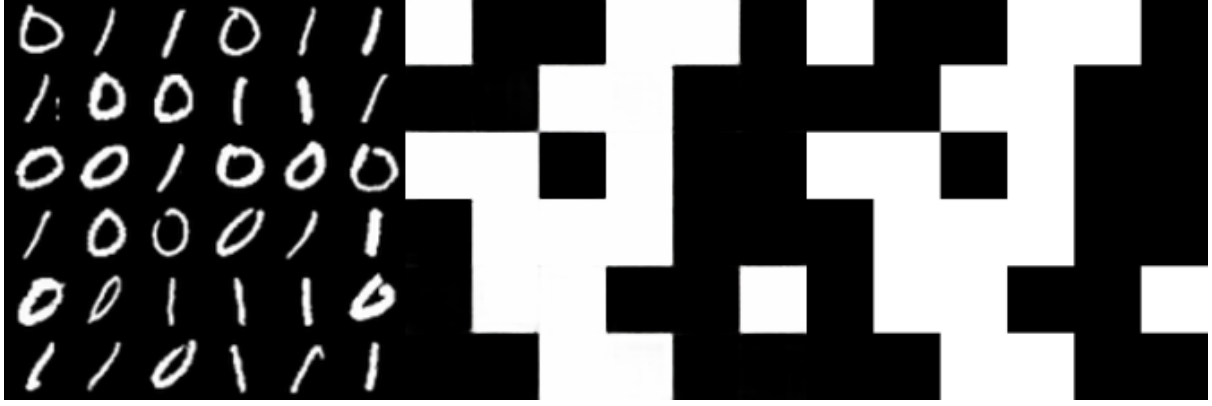
Rule evolution layers: 3

7.3.1.2. Experimental Results When the output is the cellular automata representation of the input position (i.e., evolved 0 layers), after 2600 steps of training, the validation set mseloss had already reached 0.000393. This is the eval image at 2600 steps. It can be seen that the network has no problem recognizing MNIST digits. Left is input image, right is ground truth, middle is generated image.



MNIST+cellular automata experiment: 2600-step evaluation image for 0-layer evolution. Left: input image; middle: model-generated image; right: ground truth.

When the output is the cellular automata representation of the input position evolved by 3 layers according to rule 110, after 6800 steps of training, the validation set mse loss had already reached 0.001320. This is the eval image at 6800 steps. Left is input image, right is ground truth, middle is generated image.



MNIST+cellular automata experiment: 6800-step evaluation image for rule-110 evolution over 3 layers. Left: input image; middle: model-generated image; right: ground truth.

7.3.1.3. Summary and Discussion It can be seen that this experiment achieved great success. It fully demonstrates that both pattern recognition and rule learning are intrinsic essential capabilities of neural networks. Moreover, they may not be two separate abilities, but may form a parallel unity, because the same network structure exhibits two very different capabilities. As shown above, this network structure is not the source of symbolic rule learning capability; rather, that capability comes from the nature of neural networks. In this example, the model did not try to first recognize the MNIST digit values, convert them into a black-white checkerboard representation of the initial position, and then evolve the position according to the cellular automata rule. The learning processes of pattern recognition and symbolic rule learning are also coupled, with no clear sequential order. What it actually does should still be gradient descent. As discussed earlier, connectionist gradient descent can approximate discrete symbolic rules. Moreover, since pattern recognition and symbolic rules are coupled capabilities, a more accurate description may be that connectionist gradient descent can simultaneously approximate discrete symbolic rules and fit pattern recognition. Which capability a neural network manifests mainly depends on the dataset.

7.3.2 MNIST + Cellular Automata: Rule Control and Image Transformation

This section uses an experiment to show that the rule learning capability and pattern recognition capability of neural networks are not mutually exclusive, but can be coupled in the same training process.

7.3.2.1. Experiment Description To verify the ability of neural networks to fuse discrete rule learning and pattern recognition in a single task, while also incorporating rule control, I designed an experiment to further show that pattern recognition and symbolic reasoning can be naturally coupled in the same model, strongly suggesting that both may be endogenous capabilities of neural networks. This experiment is based on the cellular automata image generation task. In the input and output images, the 1D 36-bit cellular automata state is arranged in row-major order into a 6×6 checkerboard grid image. But the input image is constructed as follows: first, randomly take an MNIST image of digit 0 or 1 and stretch it to a full 240×240 image; then modify the initial image according to the initial cellular automata state. Where the cellular automata state is 0, keep the grid point unchanged; otherwise, invert the color. The output image is similarly produced from the same digit image before input modification, according to the evolved cellular automata state. The cellular automata evolution layer remains 3, but the evolution rule is determined by the MNIST digit 0 or 1: 0 represents rule 30, and 1 represents rule 110.

This design ingeniously forces the model to simultaneously manifest pattern recognition capability to understand the initial cellular automata position, recognize the digit of the initial position, and correspond

to the evolution rule, then execute discrete logical reasoning through symbolic reasoning capability, and finally modify the initial MNIST digit image according to the final evolution state of the cellular automata to correspond to the final output image. The complexity of this task not only tests the neural network's two core capabilities of rule learning and pattern recognition simultaneously but also strongly supports the coupling of neural-network symbolic reasoning and pattern recognition across temporal and spatial scales through its long chain of steps.

Additionally, the reason why this experiment design is feasible is also because MNIST is white digits on black background; the edge is certainly black, giving the neural network an anchor to understand and correspond to the state of the cellular automata.

Dataset Code: `generate_rulemnist_ca.py`

Training Code: `train_image2image.py`

Loss: MSELoss

Some task parameters:

Grid dimension: 6×6

Grid size: 40×40 pixels

Image resolution: 240×240

Cellular automata rules: digit 0 in the input image corresponds to rule 30; digit 1 corresponds to rule 110.

Rule evolution layers: 3

7.3.2.2. Experimental Results After 10800 steps of training, the validation set mseloss had already reached 0.001011. This is the eval image at 10800 steps. Left is input image, right is ground truth, middle is generated image.



MNIST+cellular automata rule-control experiment: 10800-step evaluation image (I).



MNIST+cellular automata rule-control experiment: 10800-step evaluation image (II).

7.3.2.3. Summary and Discussion It can be seen that this experiment achieved great success. It strongly supports the judgment that pattern recognition and rule learning can both emerge as intrinsic capabilities of neural networks under suitable data conditions, and that they may not be completely separate abilities, but can appear in a coupled way within the same model and the same training process. The neural network has to recognize the digit, map it to the corresponding cellular automata evolution rule, identify the initial cellular automata state, evolve the cellular automaton, and modify the original MNIST digit image into the output according to the evolved cellular automata state. Such a long chain further shows that the model is unlikely to be executing the task serially in human-readable step order; it is more likely learning the overall mapping within a single coupled optimization process. The later discussion will examine this point more directly.

Therefore, the learning processes of pattern recognition and symbolic rule learning capabilities are also coupled, with no sequential distinction. What it actually did should just be gradient descent. Still as mentioned earlier, connectionist gradient descent can approximate discrete symbolic rules. Moreover, since pattern recognition and symbolic rules are coupled capabilities, a more accurate description might be that connectionist gradient descent can simultaneously approximate discrete symbolic rules and fit pattern recognition; as for what capability the neural network manifests, it is mainly determined by the dataset.

8 Act VIII: Assisting Training with Decoupling — Rethinking the Relationship Between Symbolism and Connectionism

In this section, I apply the decoupling idea from the Trapping Rain Water experiment above to accelerate the training of some tasks, and even make tasks that are difficult to converge under direct-output settings become learnable—for example, the binary number mod 3 task. Finally, I show that not all decoupling formats accelerate training; bad decoupling formats can even slow training down. I also discuss the profound philosophical ideas contained in this phenomenon.

The following details what decoupling is, that is, considering that direct prediction of the final output by the neural network may be difficult, one or several steps in the process are manually given as additional labels to assist the neural network in training. For example, in the Trapping Rain Water experiment mentioned above, I did not predict the binary representation of the total rainwater amount required by the original problem, but separately predicted the binary representation of the rainwater amount held by each bar. This method greatly reduced the time required for convergence. But this can be challenged as me changing the original goal. Below I introduce two decoupling methods that do not change the training goal.

The first method is similar to taking the output of the Trapping Rain Water decoupling experiment above—i.e., the amount of rainwater held by each bar—as input, and the original goal—total rainwater amount—as output, and training with another neural network (of course, the same neural network can be used without changing the network structure, just changing the number of outputs of the linear head layer). This method is equivalent to requiring different networks to predict different steps and then joining them together. Of course, a larger network can also be manually cut into several parts, with each part responsible for a segmented goal. The advantage of this method is that it is relatively consistent with human natural intuition: the front network completes the front task, and the back network completes the back task. But this method imposes an extra burden on people, making this method not the most natural one. That is, people need to manually specify which part of the network completes which stage of the task. Apart from intuition and experience, it is difficult to have better guidance. Moreover, this method is difficult to fully utilize network capacity. In addition, below, I will present a study on an already converged neural network, fully illustrating that a naturally converged neural network does not calculate step by step serially logic as conceived by this decoupling method, but is a computational logic implementation method where different stages and steps are quite coupled.

The second method is to directly concatenate the decoupled labels into the final result. If there are more detailed decoupling steps, they can also be added very conveniently because only the output labels

need to be changed. Moreover, the advantage of this method is that there is no need to choose which part of the network performs the calculation of which stage, allowing for fuller utilization of network capacity.

Below I consistently choose the second decoupling method.

8.1 Binary Number Mod 3 Experiment

8.1.1 Experimental Part

The non-decoupled format has been introduced earlier. Under the initial experimental setting used in this paper, the model showed almost no effective loss decrease within a reasonable training time. Later experiments showed that when the number of input bits is reduced, this task is not absolutely untrainable; rather, the training landscape is very difficult and the required convergence time increases significantly. For this reason, mod 3 is a good example: it shows that some rules may be simple in description, yet highly unfriendly to the training dynamics of neural networks. The input and output formats of the decoupled experiment are introduced below.

- **Input Format:** 30-bit binary number.
- **Output Format:** DFA trajectory, a total of 60 bits, where every two bits correspond to the mod 3 result of the preceding part of the input binary number. For example, bits 1-2 correspond to the mod 3 result of the 1st binary bit, bits 3-4 correspond to the mod 3 result of the 2-bit binary number composed of the 1st and 2nd bit, so clearly, bits 59-60 correspond to the mod 3 result of the entire 30-bit binary number.
- **Decoupling Format Discussion:** This decoupling format represents a state with every two bits, containing only three states:
 - **State S0:** Represents the current processed prefix value V satisfies $V \pmod{3} = 0$. If read $b = 0$, then $S_{next} = (0 * 2 + 0) \pmod{3} = 0$. State transition: $S0 \rightarrow S0$. If read $b = 1$, then $S_{next} = (0 * 2 + 1) \pmod{3} = 1$. State transition: $S0 \rightarrow S1$.
 - **State S1:** Represents the current processed prefix value V satisfies $V \pmod{3} = 1$. If read $b = 0$, then $S_{next} = (1 * 2 + 0) \pmod{3} = 2$. State transition: $S1 \rightarrow S2$. If read $b = 1$, then $S_{next} = (1 * 2 + 1) \pmod{3} = 0$. State transition: $S1 \rightarrow S0$.
 - **State S2:** Represents the current processed prefix value V satisfies $V \pmod{3} = 2$. If read $b = 0$, then $S_{next} = (2 * 2 + 0) \pmod{3} = 1$. State transition: $S2 \rightarrow S1$. If read $b = 1$, then $S_{next} = (2 * 2 + 1) \pmod{3} = 2$. State transition: $S2 \rightarrow S2$.

Training Set Script: `generate_binary_mod3_dfa_explain.py`

Training Script: `train_mlp.py`

Training Result: Using an MLP network, convergence was rapid, reaching 99.96% DFA sequence exact match in 25 epochs.

8.1.2 Discussion of Experimental Results

This is a very unexpected result. It is not only that this decoupling format assists training; it can also make a problem that is difficult to learn under direct-output settings become highly learnable. This is clearly not simply a matter of network capacity. What is even more puzzling is: if the problem is so simple in representation, why must DFA auxiliary sequences be added for convergence? Why is training only on the final result insufficient? One possible explanation is that the decoupling information changes the training landscape, making it easier for gradients to guide the model toward the correct rule structure. I cannot fully answer this question yet, but at least in this experiment, the effectiveness of this decoupling method is clear: although it increases the length of output labels and seems to add burden, it makes training feasible and faster. This is a highly counterintuitive result.

8.2 Another Decoupling of the Trapping Rain Water Experiment

In this section, I decouple the Trapping Rain Water experiment, which has already been decoupled above, once again, exposing different algorithmic details.

8.2.1 Experimental Part

Input Format Introduction:

$N \times \text{BITS_PER_HEIGHT}$ bit binary number, where N is the number of bars, and BITS_PER_HEIGHT is the number of binary bits for the height of each bar.

Output Format Introduction:

Algorithm decoupling process part + result part.

Result Part:

$N \times \text{BITS_PER_HEIGHT}$ bit binary number, where N is the number of bars, and BITS_PER_HEIGHT is the number of binary bits for the amount of water held by each bar.

Algorithm Decoupling Part:

Three algorithm ideas are considered, corresponding to [42. Trapping Rain Water - LeetCode](#).

DP Decoupling Format:

A vector of length $2 \times N \times \text{BITS_PER_HEIGHT}$. It is directly concatenated from two parts: the binary representation of the `leftMax` array and the binary representation of the `rightMax` array. The i -th element of `leftMax` represents the highest bar height from position 0 to i in the input sequence; the i -th element of `rightMax` represents the highest bar height from position i to $N-1$. Each height value is encoded as a BITS_PER_HEIGHT bit binary number.

Monotonic Stack Decoupling Format:

A vector of length $N \times (2 \times \text{BITS_PER_INDEX} + \text{BITS_PER_HEIGHT})$. This vector is concatenated from N fixed-size “event slots.” Each event slot records a triplet (`left_idx`, `right_idx`, `top_height`): `left_idx` is the index of the left boundary wall of the concave groove, encoded as BITS_PER_INDEX bits; `right_idx` is the index of the right boundary wall, also encoded as BITS_PER_INDEX bits; `top_height` is the height of the groove-bottom bar where water is calculated, encoded as BITS_PER_HEIGHT bits. Unused event slots are filled with 0.

Two Pointers Decoupling Format:

A vector of length $N \times \text{BITS_PER_HEIGHT}$. The i -th element block records the value of `leftMax` or `rightMax` relied upon when calculating the rain at position i . If the rain at position i is determined by `leftMax`, the `leftMax` value at that time is recorded; if it is determined by `rightMax`, the `rightMax` value at that time is recorded. If position i holds no water or acts as a boundary wall, its corresponding value is 0.

Specific Parameters: $N=15$, $\text{BITS_PER_HEIGHT}=5$, trained for 10 epochs on a training set of 500,000 entries.

Training Set Script: `generate_rain_water_final_showdown.py`

Training Script: `train_mlp.py`

Experimental Results (Validation Set):

	DP	Monotonic Stack	Two Pointers	No Decoupling Format
Decoupling part exact match	96.02%	17.36%	95.46%	
Result part exact match	75.60%	6.20%	85.16%	56.60%

If only caring about the exact match of the result part, it can be seen that the sorting of training effect from good to bad is Two Pointers > DP > No Decoupling Format > Monotonic Stack.

8.2.2 Discussion of Experimental Results

It can be seen that the DP algorithm and Two Pointers algorithm promoted the convergence of the task, but the Monotonic Stack algorithm actually slowed down the convergence of the task! First, the fact that a certain decoupling method can promote convergence once again proves that end-to-end training is not necessarily the best choice. Since the era of deep learning, there has been a belief: do not introduce too many manual biases, let it discover how to solve the problem. I personally believe that on one hand, this approach might be naturally effective for perceptual problems, and on the other hand, it is also because in the pattern recognition paradigm, it is difficult to give a clear auxiliary label without information loss and without introducing artificial bias. Now, in the paradigm I discovered, the latter difficulty no longer exists; for complex rule problems, different decoupling information can be given very simply to assist training. So, this experiment clearly illustrates that if some decoupling is given as guidance, the difficulty of learning the task can be completely reduced. Moreover, it should be noted that this is just a small algorithmic problem, and convergence can already be accelerated by this means. So it is easy to imagine that for a complex multi-stage large-scale rule/algorithm learning task, if decoupling step information is not added, simply extending the training time will be of almost no help.

Additionally, the Monotonic Stack decoupling method actually slowed down task convergence, which is a very interesting phenomenon. According to the description of this decoupling format, it is generally the longest of the three decoupling labels. Moreover, it describes not just information but the execution process of a serial algorithm, which is completely different from the decoupling formats of the DP algorithm and Two Pointers algorithm. The fact that it slows down task convergence suggests a possibility: for the neural network, the cost of fitting the serial process of the Monotonic Stack itself may exceed the benefit it brings to the final task. So, the final convergence speed is actually even worse than training the task objective directly without decoupling auxiliary labels.

8.3 Discussion on Related Philosophical Meanings

If end-to-end connectionist learning is viewed as intuition, although many experiments above have shown that this intuition can cope with many tasks related to symbolic rules, experiments have also proven that this intuition is far from perfect. Whether in terms of convergence speed or learnability for certain problems, adding some decoupling—meaning information about intermediate process stages—might promote convergence. And I believe the significance of this point goes far beyond a mere technical experimental observation; just like humans cannot cope with the world relying solely on intuition, some logical thinking is absolutely necessary. The trust in end-to-end training of neural networks since the arrival of the deep learning era basically corresponds equivalently to the currently popular so-called “first principles thinking” or “deconstruction.” Its root lies in the fact that many ideas are outdated and can no longer reflect facts, such as the aforementioned computer vision relying on handcrafted features (like [17] SIFT, [18] HOG), which can be seen as a “prior symbolic knowledge” of the visual world. Or the aforementioned string of logical chains without context leading to wrong conclusions; in this context, end-to-end training is akin to re-examining the problem without relying on past experience. As for the Monotonic Stack algorithm decoupling method in the Trapping Rain Water experiment mentioned above, if the neural network learned that Monotonic Stack algorithm step by step, could it solve the Trapping Rain Water problem? Yes! But, ultimately, it is found to be unnecessary, because adopting this decoupling method leads to even slower convergence than not using decoupling assistance. This is somewhat like: ideas were originally meant to help people solve problems and live better, but sometimes they completely detach from context and lose efficacy, becoming worse than a person dealing with it based on their own independent thinking—the neural network performed better based on its own native capabilities. But this does not deny that some decoupling methods accelerate training, such as the DP and Two Pointers methods mentioned above.

Maybe decoupling can be regarded as an operation that destroys the distributed representation of thought and makes it explicit, similar to making the subconscious conscious in psychoanalysis. This operation can clarify thoughts entangled together in some scenarios, making thinking easier, while in other

scenarios, it might reduce the computational efficiency of distributed representation due to excessive symbolization.

This is absolutely not a simple contradiction. As the old Chinese saying goes: *Learning without thought is labor lost; thought without learning is perilous*. In MBTI personality types, there are P-types biased towards perception and sensing, and J-types biased towards planning and judging. In reinforcement learning, there is the contradiction between explore and exploit. Perhaps the formation of human concepts corresponds to the choice that maximizes cognitive economic utility, and with the passage of time and environmental changes, cognition is destined to be deconstructed and reconstructed time and again. Thinking involves making judgments; judgments will be wrong and might be corrected in the future, but not making judgments might be even worse. There is no absolutely correct choice; there only exist certain cognitive tendencies.

But this decoupling format returns to human manual selection again; it cannot emerge naturally, which seems to violate the philosophy of deep learning. So is it possible for neural networks to spontaneously form this decoupling format information or increase insight into the problem structure to accelerate training? I cannot answer this question, but if neural networks have the ability to do so, an even more incredible scenario will appear: neural networks having the ability to quickly fit any extremely complex problem. It's too good to be true; I personally believe intelligence itself is not that simple.

But regarding this problem, I think there is another roundabout way. If the same neural network is trained on multiple tasks simultaneously, when the neural network is forced to solve a large number of different but related symbolic tasks simultaneously within the same parameter space, the cognitive economic pressure of gradient descent will urge it to discover and construct those intermediate representations that are common and reusable across tasks—this might be related to my previously manually designed decoupling formats. For example, if images of cats from ImageNet and images of 1s from MNIST are taken out and merged together, asking the neural network to distinguish these two classes, it might find a shortcut based on texture or color; but a network that needs to simultaneously distinguish 1000 fine-grained categories in ImageNet is forced to form reusable, hierarchical abstract representations internally, from edges to parts to objects. So, the continuation of the philosophy of spontaneous emergence in deep learning might depend on multi-task simultaneous training. The birth of this insight into problem structure might not lie in deep drilling into a single task, but in the breadth of experience. Just as we observe in human cognition, there is hardly depth of cognition without breadth of experience. Of course, whether this hypothesis is correct still depends on future experiments and theoretical research.

9 Act IX: Research on Already Converged Neural Networks

The main purpose of this section is to study a neural network that has already converged on a specific problem, including its calculation method, the information carried by each hidden layer, and so on. The neural network used here is an MLP. The main research method is to freeze a certain hidden layer after the neural network has converged, connect another MLP behind it to try to decode specific information, and then indirectly judge, from convergence speed and final convergence accuracy, whether this information already exists relatively accessibly in that hidden-layer representation and whether it is excessively entangled with other information. The reason final convergence accuracy alone is insufficient is that, in some cases, given enough time, many kinds of specific information may eventually converge. For example, in some extreme cases, exact input information can even be decoded from certain hidden layers, so the neural network might first decode the input and then fit the designated specific information. However, in such cases, the convergence speed for different types of specific information often differs, so the mode in which information exists in the hidden layer can only be judged through convergence speed.

9.1 Studying the Last Hidden Layer via Trapping Rain Water Experiment

9.1.1 Experimental Part

I used the result part of the Trapping Rain Water from Section 8.2 as labels to train an MLP network. After it converged, I extracted the last hidden layer and connected a new MLP network behind it, with the output pointing to some designated information, to detect the existence and mode of existence of this information in this hidden layer.

These designated pieces of information are the decoupling information of the three algorithms introduced in Section 8.2.1—DP, Monotonic Stack, Two Pointers—which will not be repeated here.

Dataset Parameter Settings:

Number of bars: 10

Binary bits occupied by bar height: 4

Training Script: `train_mlp_prefer.py`

Experimental Results:

Decoupling Algorithm	Best exact match
DP	26.90%
Monotonic Stack	42.34%
Two Pointers	92.74%

9.1.2 Discussion of Experimental Results

This experimental result indicates that after convergence, the neural network will have a large amount of task-related information in the last hidden layer. This information can be probed to try to understand how the neural network learns this task. Moreover, when learning a certain task, the neural network will form spontaneous cognitive style tendencies. For example, on this problem, the neural network's preference for these algorithms: Two Pointers > Monotonic Stack > DP. So, why is it like this? This is a very interesting question that will not be easily answered. Even considering that these decoupled formats cannot be completely equivalent to the corresponding algorithms, answering this question is still not that easy. Plus, considering that the Monotonic Stack decoupling format is actually longer than the DP decoupling format, this result is even more interesting.

9.2 Probing the Hidden Layers of a Converged MLP with Cellular Automata Task

9.2.1 Experimental Part

Experimental Setup: First, fit the state of a 1D 30-bit cellular automata after 8 layers of evolution under rule 110. Then sequentially extract each hidden layer and fit the states after 1 to 8 layers of evolution using another MLP network, training for the same number of epochs, and observe the exact match values.

Dataset Parameter Settings:

Cellular automata bits: 30

Evolution rule: 110

Evolution times: 8

Training Script: `train_mlp_ctscan.py`

Experimental Results: See the table below, S_1 EM represents the exact match for fitting the state of 1 layer of evolution.

Start fitting from which hidden layer	S_1 EM	S_2 EM	S_3 EM	S_4 EM	S_5 EM	S_6 EM	S_7 EM	S_8 EM
H_0	100.00%	100.00%	99.96%	59.20%	3.40%	0.04%	0.00%	0.00%
H_1	100.00%	100.00%	99.90%	90.12%	76.56%	33.72%	1.50%	0.24%
H_2	95.02%	86.64%	72.88%	65.08%	28.02%	11.42%	8.26%	99.88%
H_3	32.92%	29.18%	31.52%	35.40%	21.24%	21.90%	41.08%	100.00%

9.2.2 Discussion of Experimental Results

Based on the experimental result chart above, several qualitative conclusions can be drawn:

- Fitting directly from H_0/H_1 , the more evolution layers, the harder to fit. This is easy to imagine and reasonable because H_0/H_1 are very close to the initial input, so as the problem difficulty increases, it naturally becomes harder to fit.
- The difficulty of fitting the final state decreases gradually from H_0 to H_3 . This is also reasonable. If the neural network is viewed as a way of distributed computing, the sum of the entire neural network's calculations is fitting this task. The earlier neural network layers have already undertaken a lot of calculations, making the representations of later hidden layers closer to the final target to be fitted.
- From H_0 to H_3 , the fitting ability for $S_1/S_2/S_3$ gradually decreases. For $S_4/S_5/S_6$, the fitting ability basically follows a rise then fall pattern (H_2 seems to be an exception for S_6 , but it does not affect the final conclusion). For S_7/S_8 , the fitting ability gradually increases. This is an interesting phenomenon, indicating that as the hidden layers move from the front to the back of the neural network, the information they contain becomes closer to the final task, gradually starting to lose the initial input information and shallow evolution state information, but not completely losing it, because these data are only obtained with very small epochs and small-scale MLPs. Below we will see that for a cellular automata with fewer evolution layers, it is even possible to recover complete evolution information, including complete input, from the last hidden layer of a converged neural network.
- Another conclusion that can be drawn is that the neural network's calculation method does not follow the clear evolution logic of rule 110 layer by layer, because if it did, the exact match of H_3 or H_2 for S_7 would also be very high. The experimental results prove that H_3 fitting S_7 is far less easy than imagined; for example, actually, H_3 's exact match for S_7 is less than 10% higher than H_3 's exact match for S_1 . So, the entire neural network is just pointing towards the fitting of S_8 , and S_7 does not naturally occupy any special status.

9.3 Decoding the Complete Evolution Sequence of Cellular Automata from the Last Hidden Layer

9.3.1 Experimental Part

Experimental Setup: First, fit the state of a 1D 30-bit cellular automata after 4 layers of evolution under rule 110, training for 15 epochs. Then extract the last hidden layer, use another MLP network to fit the concatenated states of 1 to 4 layers of evolution, and observe the exact match values.

Dataset Parameter Settings:

Cellular automata bits: 30

Evolution rule: 110

Evolution times: 4

Training Script: `train_mlp_fulltrace.py`

Experimental Results: After sufficient training, an exact match of 100% can be achieved, i.e., 100% sequence accuracy for the evolution sequence of the cellular automata on the validation set.

9.3.2 Discussion of Experimental Results

This experimental result shows that the last hidden layer indeed contains a large amount of information related to the task evolution sequence. For tasks with fewer evolution layers and lower difficulty, this information can even be fully recovered. The probing experiment above also shows that in more complex tasks, hidden layers likewise contain evolution-state information at different levels, but the recoverability and entanglement of this information vary with hidden-layer position and task difficulty. This property

may not be limited to the last hidden layer, but later hidden layers may contain more high-level semantics corresponding to the task objective.

10 Act X: Some Further Explorations

This section briefly describes some additional explorations, only as supplementary observations, without excessive in-depth discussion and analysis.

10.1 Training Visualization

In those experiments above where the output is an image, it has been shown that the learning process can be visualized through periodic eval images during the eval process. So how to do this for general tasks? I found that as long as eval loss and accuracy statistics are performed separately for each bit during each eval, a relatively good visualization can be completed to study the learning process of the neural network.

When this method is used on binary addition, two phenomena were discovered. One is the easy-to-imagine serial learning, i.e., the accuracy gradually rises from low bits to high bits, because addition is difficult to predict high bit results without knowing accurate low bit results. The other phenomenon is that the highest bit of the result also converges relatively quickly; analysis suggests that some kind of heuristic simple pattern might have been discovered.

When this method is used on cellular automata experiments, it is found that almost every bit converges synchronously, and the convergence degree of each bit of the cellular automata is about the same in each eval.

Training Script: `train_mlp_visualize.py`

10.2 Autoregression

Surprisingly, using a decoder-only transformer can also fit the cellular automata dataset, only by completely changing its format to a string, for example:

`1011...1110 -> 1010...0001`

And then predicting the next token (a certain bit output of the cellular automata) from any position after the arrow; experiments prove that it can converge. But the problem is, this experiment might not actually depart from this paradigm. Imagine that predicting each bit of the output is actually a task type, and the neural network can distinguish which task it is through the input format and learn to predict. This is actually just a symbolic rule learning with branches. As for whether the current large model paradigm can precisely and hallucination-free evolve cellular automata to a relatively deep layer, I cannot give a definitive answer, but I tend to think it is unlikely to be done without the aid of tools, and it indeed made errors in actual tests, just as large models cannot perform reliable large number multiplication without tools, etc.

Dataset Script: `generate_ca_text_format_dataset.py`

Training Script: `train_ar_transformer.py`

11 Act XI: Theoretical Interpretation

This section attempts a rough phenomenological analysis of these experimental phenomena. If we momentarily set aside some tentative experiments conducted later, and also temporarily ignore experiments that only converge with added decoupling information, we can summarize it as such a fact: given a very small amount of data relative to the task space, for the vast majority of tasks, neural networks can universally fit to the precise rules behind them. Here we might need to define “task” precisely. Considering that I did not have any restrictions when conceiving and selecting tasks, its real restriction is the input-to-output transformation written by python programs. So now considering this fact, neural networks can universally fit to the precise rules behind them with extremely small amounts of data, what does this

mean? First, we can see that because the given training set data is extremely small, it is an extremely underdetermined problem, because corresponding to the given training set, there can actually correspond almost infinite rules. Moreover, considering such an experimental observation that the training set samples need a certain quantity to converge, we can view the process of increasing training set samples as a process of increasing constraints. Each sample is a constraint, and the training of the neural network is similar to solving some equation, or it can also be seen as a process of Bayesian belief update as samples increase. In this way, only when the number of samples is more than a certain amount, based on the constraints of samples and the form of the equation solved by the neural network, the solution starts to stabilize, and from the perspective of Bayesian belief update, the posterior also gradually narrows. So the question is, why can the neural network always choose the real rule behind our generated training set? We can now only draw this conclusion: the neural network has its internal preference or bias. The neural network solves equations according to some preference or bias, and so is Bayesian belief update. So what is this preference or bias? Considering our experimental results, the neural network consistently fits the real rule behind it. The common point among them is that this is the simplest way to describe the dataset, and because the best way to describe or compress the training set is to find this logical transformation rule. So we can conclude that under ideal training data, the neural network implicitly executes [25] Kolmogorov compression or searches for the shortest program description by finding flat minima through SGD, which is the [26] MDL principle.

This is a strong assumption, so we need to give as precise a definition as possible for the rule-learning tasks focused on in this paper. In simplified form, they can be defined as programmatically generated deterministic transformations.

The rules focused on in this study can be formalized as a special class of functions whose core characteristics stem from their generation process:

- **Source-code definition:** They are defined not by data points, but by deterministic, finite-length computer programs. This means their Kolmogorov complexity is far smaller than the size of their input spaces.
- **Zero-entropy determinism:** For any input, the program-generated output is unique. This eliminates chance and uncertainty, making the theoretical minimum loss absolute zero, and thus giving the task algorithmic compressibility.

Below we consider specific supporting evidence.

11.1 Theoretical Assumptions and Evidence

It should be emphasized that this phenomenon does not rely on complex specialized architectures. Many key later-stage experiments can already be completed by a plain MLP: the model has no external symbolic system, no explicit algorithmic module, and no handcrafted reasoning structure. It consists only of parameters, nonlinear transformations, and gradient descent. This makes the observation in this paper closer to a property of connectionist systems themselves, rather than the result of a particular architectural trick.

First, we observed in a large number of experiments that as task difficulty increases, the number of samples needed in the training set to support neural network fitting increases. This is exactly consistent with the MDL principle because complex rules need more samples to cover patterns. Moreover, it is observed that as task difficulty increases, the time required for training also increases. We can phenomenologically understand why neural networks learn rules: the neural network makes a choice between learning rules and memorizing samples. For symbolic rules: the complexity of the rule itself is constant and relatively low. However, the difficulty of memorization increases with the increase of sample size. Once the sample size exceeds a certain low threshold, fitting rules becomes “cheaper” than memorization, thereby forcing the model to choose generalization. Contrast with pattern recognition: in traditional tasks (like images), learning is from coarse-grained to fine-grained. In the late stage of training, the difficulty of mining extreme details exceeds the difficulty of memorization, leading the model

to turn to memorization and overfitting. Regarding the rapid growth of memorization pressure, perhaps it can be explained this way: the input and output of symbolic data are discrete step jumps, lacking local smoothness, i.e., changing one bit of input may change the entire output. This rugged topological structure makes the cost of memorization based on interpolation extremely high. We might call it the endogenous regularization of data.

This suggests a three-stage hypothesis about sample size and the generalization process. When data is severely insufficient, the training constraints are not enough to exclude a large number of memorization-like or accidental solutions, and the model remains in an overfitting state for a long time. When the data reaches an intermediate range, the model may first memorize the training set and then undergo grokking under the implicit regularization of SGD, with validation accuracy suddenly rising to nearly 100%. According to observations in grokking-related experiments, this transition usually becomes faster as the amount of data increases. When the data becomes sufficiently abundant and pattern coverage is adequate, training loss and validation loss can decrease together, without a clear phase of overfitting followed by generalization. This corresponds to the ideal training behavior observed in many experiments in this paper.

If we consider this problem with the above theory, when the number of samples is less than the number of samples needed to fit the rule, the neural network shows overfitting during training. But here comes a question: if we understand that the neural network chooses memorization when it finds memorization easier during training, and chooses fitting rules when it finds fitting rules easier, is this phase transition point consistent with the phase transition point mentioned above where complex rules need more samples to cover patterns according to the MDL principle? I think they are very likely consistent. In microscopic neural network dynamics, this manifests as gradient direction and magnitude; in phenomenological theory, it may manifest as a comparison of the difficulty of memorization and fitting rules; macroscopically, it may correspond to the description length principle of MDL: if training samples are few, the fitting mode of memorization or biased towards memorization may be smaller than the description length of the rule. And these three perspectives should correspond to the same thing; they should completely align.

Thinking from the perspective of [34] Implicit Regularization theory, the characteristics of stochastic gradient descent make it implicitly prefer flat regions: optimization trajectories continuously escape sharp minima because they are prone to failure due to perturbations, and finally settle in flat minima regions. In weight space, flat minima (broad regions with approximately constant error) allow weights to be stored with low precision, while sharp minima (narrow valley bottoms) require high precision. From the MDL perspective, the former corresponds to short program descriptions (only a few bits needed to locate “any point within the basin”), and the latter corresponds to long programs (need precise coordinates). Or thinking from another angle, the gradients of rule solutions are coherent across different training set samples and will reinforce each other, while the gradients of non-rule solutions or memorization solutions are incoherent across different training set samples and will cancel each other out. This explains our observation. Therefore, SGD achieves implicit optimization for the shortest program description by finding flat minima. This is some inevitable phenomenon at the statistical level. The randomness of SGD is like a small ball perturbed in an energy landscape. Due to its free energy minimization tendency—pursuing both low energy (low loss) and high entropy (wide flat regions, allowing low precision storage)—it inevitably settles in flat basins. This is essentially the formation of dissipative structures in non-equilibrium thermodynamics: as an open system, the network spontaneously emerges low-entropy ordered rule structures from high-entropy chaos by continuously dissipating gradient energy. Its statistical essence follows the free energy minimization principle—the system automatically selects the path with the lowest complexity cost under constraints, making intelligence an inevitable emergence of energy-information conversion under non-equilibrium conditions.

From this angle, neural network training can be heuristically understood as a process of free-energy minimization: the loss function corresponds to an energy term, flat basins correspond to an entropy term, and the stochasticity of SGD makes optimization not only pursue low loss, but also prefer wider, more stable parameter regions with lower description cost. In other words, training is not merely searching for

any solution that fits the training set; it is performing an implicit trade-off among energy, entropy, and complexity. This thermodynamic analogy is not a strict physical derivation, but it points to a possible theoretical direction: neural network training may be understood as a transformation among energy, information, and structural compression.

Finally, linking this to the concept of **compression is intelligence**, combining [32] Ilya’s 2023 talk and [33] Jack Rae’s online talk, their main idea is that optimal unsupervised learning is regular Kolmogorov compression (rather than compression conditional on data), and that neural networks + SGD are currently our best practical method for approximating the uncomputable Kolmogorov compressor. Here, our experiments provide a particularly pure test of this theory. Because we use programmatically generated, zero-noise ideal data, we eliminate the irreducible error inevitable in real-world data. Therefore, neural network training is choosing a way to compress the mapping from input to output in the training set. And our large number of experiments show that, given enough training samples, neural networks consistently fit the true rule. This gives strong experimental evidence for the above proposition. It echoes the core assertion of “compression is intelligence”: when a neural network selects a short program description for a generative rule under zero-noise conditions, it is performing lossless compression of the data—not superficial statistical pattern matching, but compression coding of the underlying causal mechanism. The limit of this compression, i.e. the minimum program length, is exactly the goal pursued by [24] Solomonoff Induction.

11.2 Relevant Experimental Evidence

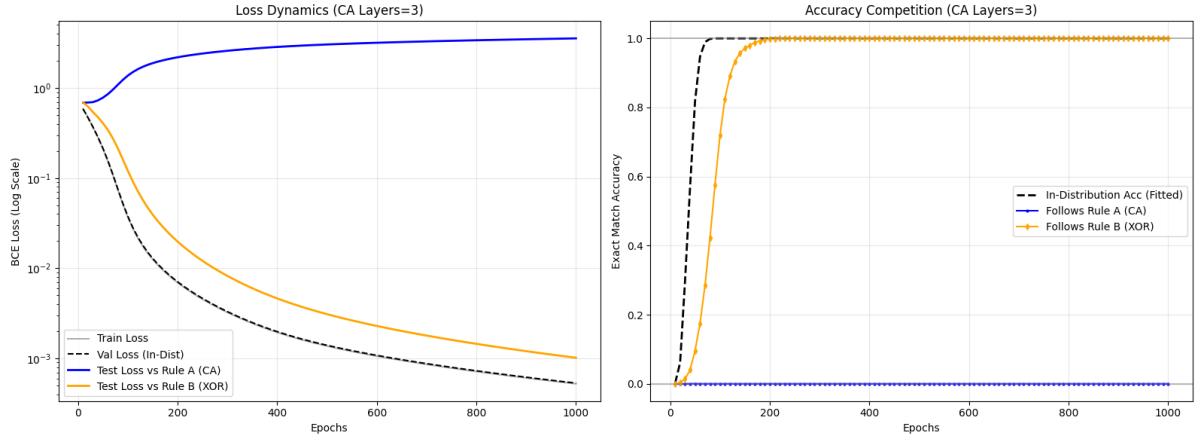
Relevant scripts for this part can be found in the `research` subdirectory under the code repository root. For extreme over-parameterization experiments, refer to `train_mlp.py` in the root directory, changing hidden layers to 8192.

11.2.1 [27] NTK Related Experiments

I used an experimental script to roughly test whether my discovery only involves lazy learning in NTK theory. The answer is negative. For a task of a 1D 30-bit cellular automata evolving 3 layers of rule 110 with a training set of 30,000 samples, the analytical solution of NTK generalization only has 1.4% exact match accuracy on the test set, while with the same training set, real neural network training can reach 100% test set exact match accuracy. This is not explainable by error. Such contrast has also been confirmed in some other experiments.

11.2.2 Rule Preference Experiment

In this experiment, I designed two rules coexisting in the training set samples that can both precisely deduce output from input, and then tested which rule the neural network is more inclined towards internally on the test set. Experiments show that when the difficulty gap between the two rules is too large, after training for a period, the neural network will completely accept one rule and abandon the other. This clearly proves the neural network’s tendency to choose simpler rules. Additionally, interestingly, the fitting speed of the in-distribution validation set is faster, reaching 100% accuracy in advance. Note that this is not memorization because it does not overlap with the training set; at this time, it should have learned some entangled state of simultaneously using both rules and corresponding data to predict. Some time later, the neural network completely turns to the simpler rule, completely turning from the entangled state of utilizing two rules and data to the simpler rule, and the accuracy on the test set of this rule reaches 100%. This is actually the [28] **Grokking** phenomenon! And this also explains that even after the neural network completely fits the training set, the implicit regularization of SGD is still driving the network weights to drift towards simpler rules. Therefore, SGD achieves implicit optimization for the shortest program description by finding flat minima.



As shown in the figure above. When the difficulty gap between the two tasks is not large, i.e., when the cellular automata evolution layer is 1 or 2, both rules generalize poorly out-of-distribution, presenting an entangled state of rules.

11.2.3 Extreme Over-parameterization Experiment

Given a 269M naive MLP network, asking it to fit some training sets with extremely small samples, it still does not overfit. Including, for 1D 30-bit cellular automata, evolving 1 layer of rule 110 requires only 1500 samples to ensure no overfitting; evolving 2 layers of rule 110 requires only 4500 samples to ensure no overfitting, and other experiments. Such extreme experimental configuration makes memorizing training set samples extremely easy, yet there is no overfitting, challenging the common sense of Large Model + Small Data = Overfitting, and supporting the hypothesis of deep neural networks as MDL solvers, i.e., neural networks are looking for a shortest program description.

11.2.4 Rule OOD Generalization Experiment

There are three related experiments. In the first experiment, the input consists of the initial state of the cellular automata (CA) and the CA evolution rules, with a default of 2 layers of evolution. The second experiment adds a rule bit to control whether the evolution proceeds for 1 or 2 layers. The third experiment adds another bit to control whether the output is shifted right by 15 positions.

The experimental method is to randomly sample the initial states of the CA, while the rule part is consistently restricted to a subset randomly sampled from all possible rules. This subset remains consistent for a given dataset, with the proportions of the total space set to 90%, 70%, and 70% respectively.

The observed phenomenon is that the model converges very quickly on the in-distribution validation set (i.e., rules within the restricted subset). However, if training continues, rules in the validation set that never appeared in the training set also converge.

This experiment proves once again that even after converging within the distribution, neural networks continue to drift toward simpler algorithmic descriptions. This explains the generalization to out-of-distribution rules—the neural network considers a structural understanding of the rule bits to correspond to a simpler algorithmic description.

11.3 Some Additional Thoughts

- Further thinking on the essence of MDL: Neural networks might be performing some kind of logical auto-simplification. Taking various experiments in this paper as examples, it is very likely that simplified logic is unintentionally introduced. For instance, creating a trivial example, if my program calculates $(x+1)+(x-1)$, then the data received by the neural network is just $2x$. It can only see data, so it will only fit data. As for how many equivalent logics describe the data, this is unknown and uncared for by the neural network. But the logic of the program is $(x+1)+(x-1)$,

and maybe human understanding is also the same as the program. This is just an analogy; the actual situation is much more complicated. This situation is more likely to occur when superimposing many layers of logical steps; some simplifiable situations or logical shortcuts might appear, and at this time, the neural network will notice and perform simplification. A more practical example is the experiments I did; I did not specify (and of course impossible to specify) to the neural network which input corresponds to what meaning and which output corresponds to what meaning. When writing programs, these are mostly my own arbitrary designations and for the convenience of neural network training, but the neural network still fitted. Here, we have no reason to believe that the logic fitted by the neural network is consistent with human logic or the logic of dataset generation. So, if there is some logical shortcut, the neural network might utilize it. Note that the shortcut here refers to some logical simplification, not [35] shortcut learning different from the original logic. Moreover, I think the situation goes beyond this. Considering that the way neural networks carry logic is distributed representation and distributed computing, I think it might further simplify logic that appears unsimplifiable to humans in its own form and self-organizationally distribute it to different positions of neural network weights to satisfy its own MDL principle. Therefore, from this perspective, there exists such a possibility: looking for the interpretability of neural networks is inherently impossible because the calculation and representation of neural networks are not human-understandable formal logic. It is possible to preliminarily associate network weights with human-understandable concepts, but if precise correspondence is insisted upon, transforming into formal logic, the neural network will immediately lose its advantage and essence; it is no longer a neural network.

- My experiment on combining MNIST and cellular automata discussed in Section 7.3 implies that pattern recognition and rule learning might both be native capabilities of neural networks, or even the same capability. Then, a very natural inference is that our explanation of rule learning above, i.e., neural networks searching for the shortest program description, is very likely also applicable to the pattern recognition tasks we are familiar with, but this hypothesis needs further experimental verification and theoretical research.
- Regarding the problem of out-of-distribution generalization, first, the input and output formats and number of bits of my experimental network structure are constant, so it cannot be extended to more input or output bits, and thus does not have the characteristics of composability of true symbolism. Another case, if the network structure is unchanged, then the performance in out-of-distribution generalization is mainly determined by the MDL principle, i.e., jointly determined by training data distribution and the MDL principle.
- The rule OOD generalization experiment also raises a deeper question: under what conditions does a symbolic representation become the lower-Kolmogorov-complexity explanation? If the model only memorizes each rule as an independent pattern, it cannot stably generalize to unseen rules. Only when it somehow understands the rule bits as part of a rule table, logical condition, or transformation program can it generalize to OOD rules. This suggests that under some data distributions, symbolic representation is not an externally imposed prior, but may be the shortest description of the data-generating mechanism itself. The question is: when the data is scarce, to what extent is direct memorization or intuition-like fitting cheaper; and when the data increases, at what point does abstracting symbolic rules become the lower-complexity solution? This question may touch the deeper relationship between symbolism and connectionism, logic and intuition.
- A key question: Is the neural network's preference completely equivalent to the MDL principle? A more precise definition might be needed: Traditional "simple" usually refers to shorter description length and lower Kolmogorov complexity, but the "simple" for neural networks is more likely simple in the dynamical sense—that is, flat minima that are easy for SGD trajectories to reach and stable. If so, then how microscopic neural network dynamics correspond to the macroscopic MDL principle might be a question requiring serious discussion. The difficulty of tasks like mod 3 might

reveal this difference: it is simple in description length, but might be located in a region difficult for SGD dynamics to reach. Currently, there seem to be few methods to study the properties of neural networks without going through actual experiments, so more experiments might be needed in the future to establish a relevant dynamic theoretical system and study how it corresponds to the macroscopic MDL principle. Consequently, viewing neural networks and gradient descent as a practical approximation of Kolmogorov compression is likely a suitable hypothesis.

11.4 Related Work

- **Connection with Grokking:** The previous rule preference experiment is highly isomorphic to Grokking experiments and also suggests a possible explanation for the Grokking phenomenon: after the neural network fits the training set, the implicit regularization of SGD is still driving network weights to drift towards simpler rules. Therefore, SGD achieves implicit optimization for the shortest program description by finding flat minima. And my extensive experiments also prove that for good datasets, including sufficient data volume and moderately difficult tasks, there is completely no need to go through a process of first overfitting then turning to generalization. The training process of the neural network will directly show the situation where train loss and val loss decrease synchronously, with val loss always slightly lower, until precise fitting is reached.
- **[29] Othello-GPT:** Their experiment proved that within a limited length, the autoregressive method can also achieve good precision. This experiment can be interpreted as the Transformer maintaining a world model, or it can be interpreted as performing multi-task learning, where different tasks correspond to different input lengths. If another similar game allows extension to longer variable inputs, whether it can still maintain high-precision prediction might still be a question temporarily. Additionally, my hidden layer probe experiment also has similarities with the probe experiment in this paper.
- **Amortized Planning with Large-Scale Transformers: A Case Study on Chess:** This paper inspired my research. They also reached an important conclusion that Transformers may be general algorithm approximators. My current research suggests that similar capabilities may not be limited to Transformers, nor limited to approximation; in some controlled tasks, standard neural networks can fit and execute algorithmic mappings with near-exact precision. Moreover, the problem studied in that paper is itself very interesting. Letting a Transformer directly fit the algorithm for playing chess achieved significant success, indicating that as algorithm fitters, neural networks can do far more than the discrete rule transformations emphasized in this paper.
- **[30] A Generalist Neural Algorithmic Learner:** My discovery shows that some algorithm fitting can be completed without specific architectures and intermediate signal supervision. For these tasks, the intermediate steps of the algorithm can in fact emerge spontaneously internally.
- **[31] What Can Neural Networks Reason About?:** This paper emphasizes the adaptability of different architectures to different problems. My discovery does not deny this point, but my paper mainly wants to use the fitting ability of general neural network architectures for different ideal data to trigger some kind of phase transition in neural networks, thereby exploring the properties of neural networks themselves, rather than focusing on the efficiency of specific neural network architectures in solving specific tasks.

12 Act XII: From Alchemy to Science in Deep Learning

The end of this study provides a clear and profound answer to a fundamental question that has plagued the field of artificial intelligence for decades: Can a neural network born for probability and statistics truly understand and execute precise, deterministic logical laws?

The experimental results strongly suggest that the answer is yes. The key does not lie in larger or more complex architectures, but in a fundamental experimental shift: from noisy real-world data to ideal data programmatically generated by pure rules, and from fragile autoregressive prediction chains to a

parallel, one-shot holistic solving framework.

This paper does not stop at demonstrating the superiority of any particular advanced architecture. It further conducts systematic architectural tests and arrives at a surprising but first-principles-consistent finding: even a multilayer perceptron, with no special structural bias and consisting only of parameters and nonlinear layers, can exhibit significant, near-zero-error generalization on complex logical evolution tasks. The success of recurrent neural networks pushes this observation toward an even deeper theoretical question: it suggests that neural networks may form an iterative process that humans cannot intuitively understand, but that is nevertheless functionally correct, in order to satisfy the final mathematical constraints.

Together, these experiments point to a strong judgment: the ability to learn precise, algorithmic rules is not the “patent” of any specific architecture, but is likely a general potential that connectionist systems can reveal under suitable data conditions. Architectural differences do not only determine whether this potential can be expressed; they also determine the efficiency, stability, and boundary conditions under which it appears.

More importantly, these experiments show that the capability boundary of neural networks may be broader than commonly assumed. They can not only recognize patterns in perceptual tasks, but also learn near-exact rule mappings under ideal conditions; they can not only fit statistical structures in continuous images and natural data, but also approximate discrete, deterministic, and verifiable algorithmic relations. Further perception-rule composite experiments suggest that pattern recognition and rule execution may not be two externally connected abilities, but may be different manifestations of the same connectionist learning ability under different data forms.

From a theoretical perspective, the experimental observations in this paper provide direct support for the compression-as-intelligence hypothesis. Conversely, compression as intelligence also explains the core phenomena observed here: if only training-set fitting were considered, the model could have selected many complex programs that fit the training samples equally well, or even simply memorized the training set. Yet in the experiments, under sufficient data constraints, neural networks stably converge to the true underlying rules. This indicates that the training dynamics contain some mechanism biased toward low-complexity explanations. From this perspective, rule learning, pattern recognition, and intelligence itself can all be re-understood within the framework of compression and minimum description length.

Therefore, the main significance of this paper is to point out an experimental fact that future AI research should take seriously: neural networks may not be merely probabilistic imitators or statistical pattern recognizers. Under suitable data and training conditions, they can reveal abilities closer to rule learning, algorithm execution, and structural compression. This fact may influence how we understand neural networks, build reliable AI, study interpretability, and rethink the relationship among intelligence, compression, and rules.

About the Nature of This Report

This paper is the first systematic report completed by the author as an independent researcher. It covers symbolic rules, algorithmic tasks, cellular automata, structured image transformations, architectural comparisons, perception-rule composite tasks, and several other directions. The focus of the current version is to report this series of experimental phenomena, provide reproducible code entry points, and propose an initial explanatory framework; it does not attempt to complete a closed final theory. Because the experimental scope is broad, some experiments have not yet been rerun under fully unified conditions. Complete metric records, stricter reproduction experiments, and more systematic comparison with related work will be supplemented in later versions. Constructive feedback and collaboration are welcome.

Acknowledgments

First and foremost, the author sincerely thanks family members for their unwavering support and understanding during the high-intensity experiments and paper writing over the past few months. Without

their tolerance, this work would not have been possible.

At the same time, the smooth progress of this research has largely benefited from the powerful assistance provided by frontier Large Language Models. Throughout the research process, from early GPT-4o to mid-to-late Gemini 2.5 Pro and Kimi K2, and then to Gemini 3.0 Pro and Kimi K2.5 during the late stage of paper writing and deep theoretical discussion, these AI assistants played an indispensable role as collaborative partners in writing dataset generation scripts, discussing experimental ideas, verifying mathematical derivations, and optimizing paper structure, greatly accelerating the iterative process of the research.

Specifically, the author needs to thank the DeepMind team for their work [16] *Amortized Planning with Large-Scale Transformers: A Case Study on Chess*. This research inspired the author to shift the model’s output paradigm from autoregressive generation to holistic classification (Amortized Inference). This shift was the key breakthrough unlocking the model’s generalization capability within enormous problem spaces and directly prompted this research’s systematic exploration of neural network symbolic learning capabilities.

Finally, due to time constraints, some calculations and details in this text may contain omissions or errors. The author would be very grateful for criticisms and corrections from readers.

References

1. Krizhevsky A, Sutskever I, Hinton G E. ImageNet classification with deep convolutional neural networks[C]//Proc. of the 25th International Conference on Neural Information Processing Systems. Lake Tahoe, NV: Curran Associates, 2012: 1097-1105.
2. Vaswani A, Shazeer N, Parmar N, et al. Attention is all you need[C]//Proc. of the 31st International Conference on Neural Information Processing Systems. Long Beach, CA: Curran Associates, 2017: 6000-6010.
3. Dosovitskiy A, Beyer L, Kolesnikov A, et al. An image is worth 16x16 words: Transformers for image recognition at scale[C]//Proc. of the 9th International Conference on Learning Representations. 2021.
4. Liu Z, Lin Y, Cao Y, et al. Swin Transformer: Hierarchical vision transformer using shifted windows[C]//Proc. of the IEEE/CVF International Conference on Computer Vision. Montreal, QC: IEEE, 2021: 9992-10002.
5. Liu Z, Mao H, Wu C Y, et al. A ConvNet for the 2020s[C]//Proc. of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. New Orleans, LA: IEEE, 2022: 11976-11986.
6. Graves A, Wayne G, Danihelka I. Neural Turing machines[arXiv]. arXiv preprint arXiv:1410.5401, 2014.
7. Graves A, Wayne G, Reynolds M, et al. Hybrid computing using a neural network with dynamic external memory[J]. Nature, 2016, 538(7626): 471-476.
8. Rocktaschel T, Riedel S. End-to-end differentiable proving[C]//Proc. of the 31st International Conference on Neural Information Processing Systems. Long Beach, CA: Curran Associates, 2017.
9. Manhaeve R, Dumancic S, Kimmig A, et al. DeepProbLog: Neural probabilistic logic programming[C]//Proc. of the 32nd International Conference on Neural Information Processing Systems. Montreal, QC: Curran Associates, 2018: 3749-3759.
10. Battaglia P W, Hamrick J B, Bapst V, et al. Relational inductive biases, deep learning, and graph networks[arXiv]. arXiv preprint arXiv:1806.01261, 2018.
11. Andreas J, Rohrbach M, Darrell T, et al. Neural module networks[C]//Proc. of the IEEE Conference on Computer Vision and Pattern Recognition. Las Vegas, NV: IEEE, 2016: 39-48.
12. Chollet F. On the measure of intelligence[arXiv]. arXiv preprint arXiv:1911.01547, 2019.
13. Chollet F, Knoop M, Kamradt G, et al. ARC-AGI-2: A new challenge for frontier AI reasoning systems[arXiv]. arXiv preprint arXiv:2505.11831, 2025.
14. Shojaei P, Mirzadeh I, Alizadeh K, et al. The illusion of thinking: Understanding the strengths and limitations of reasoning models via the lens of problem complexity[arXiv]. arXiv preprint arXiv:2506.06941, 2025.
15. Wolfram S. A new kind of science[M]. Champaign, IL: Wolfram Media, 2002: 1-1197. ISBN 1-57955-008-8.
16. Ruoss A, Deletang G, Medapati S, et al. Amortized planning with large-scale transformers: A case study on chess[arXiv]. arXiv preprint arXiv:2402.04494, 2024.
17. Lowe D G. Distinctive image features from scale-invariant keypoints[J]. International Journal of Computer Vision, 2004, 60(2): 91-110. DOI:10.1023/B:VISI.0000029664.99615.94.
18. Dalal N, Triggs B. Histograms of oriented gradients for human detection[C]//Proc. of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition. San Diego, CA: IEEE, 2005: 886-893. DOI:10.1109/CVPR.2005.177.
19. LeetCode. Trapping rain water[EB/OL]. [2026-02-04]. <https://leetcode.cn/problems/trapping-rain-water/description/>.
20. Yang A, Yang B, Hui B, et al. Qwen2 technical report[arXiv]. arXiv preprint arXiv:2407.10671, 2024.
21. Karpathy A. Software 2.0[EB/OL]. (2017-11-11)[2026-02-04]. <https://karpathy.medium.com/software-2-0-a64152b37c35>.

22. Wei J, Wang X, Schuurmans D, et al. Chain-of-thought prompting elicits reasoning in large language models[C]//Proc. of the 35th International Conference on Neural Information Processing Systems. New Orleans, LA: Curran Associates, 2022: 24824-24837.
23. Bengio Y, Louradour J, Collobert R, et al. Curriculum learning[C]//Proc. of the 26th Annual International Conference on Machine Learning. Montreal, QC: ACM, 2009: 41-48.
24. Solomonoff R J. A formal theory of inductive inference. Part I[J]. Information and Control, 1964, 7(1): 1-22.
25. Kolmogorov A N. Three approaches to the quantitative definition of information[J]. Problems of Information Transmission, 1965, 1(1): 1-7.
26. Rissanen J. Modeling by shortest data description[J]. Automatica, 1978, 14(5): 465-471.
27. Jacot A, Gabriel F, Hongler C. Neural tangent kernel: Convergence and generalization in neural networks[C]//Proc. of the 32nd International Conference on Neural Information Processing Systems. Montreal, QC: Curran Associates, 2018: 8571-8580.
28. Power A, Burda Y, Edwards H, et al. Grokking: Generalization beyond overfitting on small algorithmic datasets[arXiv]. arXiv:2201.02177, 2022.
29. Li K, Hopkins A K, Bau D, et al. Emergent world representations: Exploring a sequence model trained on a synthetic task[arXiv]. arXiv:2210.13382, 2022.
30. Ibarz B, Kurin V, Papamakarios G, et al. A generalist neural algorithmic learner[C]//Proc. of the 1st Learning on Graphs Conference. Virtual Event: PMLR, 2022.
31. Xu K, Li J, Zhang M, et al. What can neural networks reason about?[C]//Proc. of the 8th International Conference on Learning Representations. 2020.
32. Sutskever I. An observation on generalization[EB/OL]. (2023-08-14)[2026-02-04]. https://www.youtube.com/watch?v=AKMuA_TVz3A.
33. Rae J. Compression for AGI[EB/OL]. (2023-02-27)[2026-02-04]. <https://www.youtube.com/watch?v=dO4TPJkeaaU>.
34. Hochreiter S, Schmidhuber J. Flat minima[J]. Neural Computation, 1997, 9(1): 1-42.
35. Geirhos R, Jacobsen J H, Michaelis C, et al. Shortcut learning in deep neural networks[J]. Nature Machine Intelligence, 2020, 2(11): 665-673.

Appendix

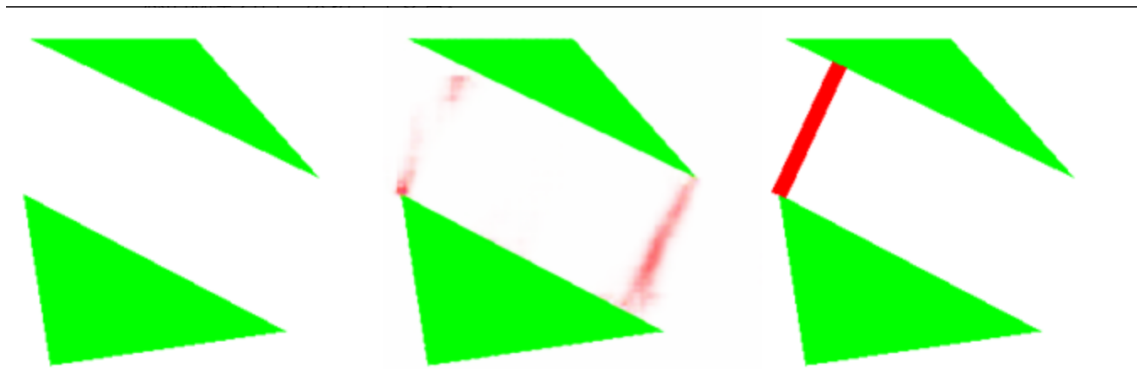
A. Some Interesting Experimental Images

- **Visualization of Learning Results**

Task Description: The input image has a white background with two arbitrary non-overlapping green solid triangles. The task is to draw a red line segment indicating the closest distance between the two triangles.

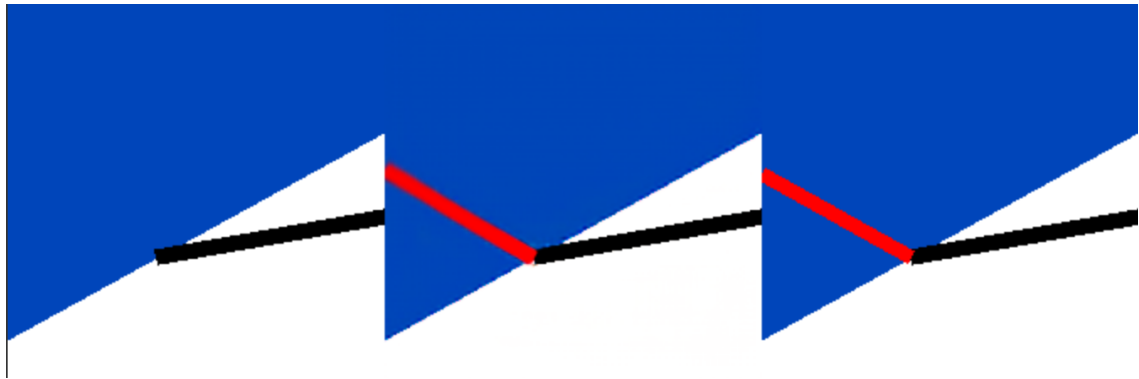
Analysis: It seems the model learned that the closest distance line segment is either the perpendicular line from a vertex to the corresponding edge of the other triangle, or the connecting line between two vertices. However, here it is difficult for it to determine which distance appears closer, so it drew both perpendicular lines, but both are blurry.

Left is input image, middle is model generated image, right is ground truth.

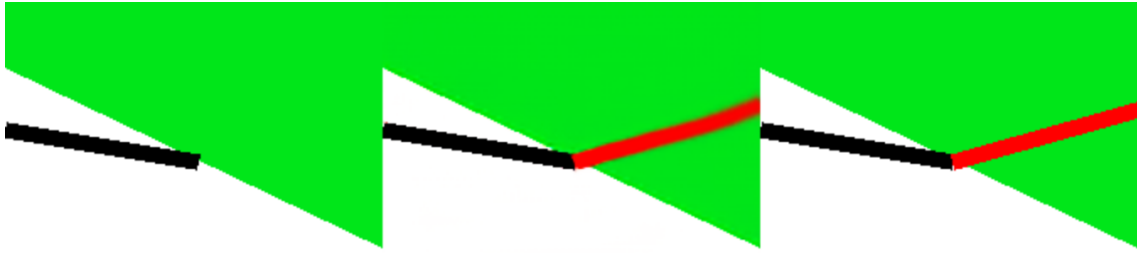


Visualization of learning results.

- **Training Results of Variable Refractive Index**

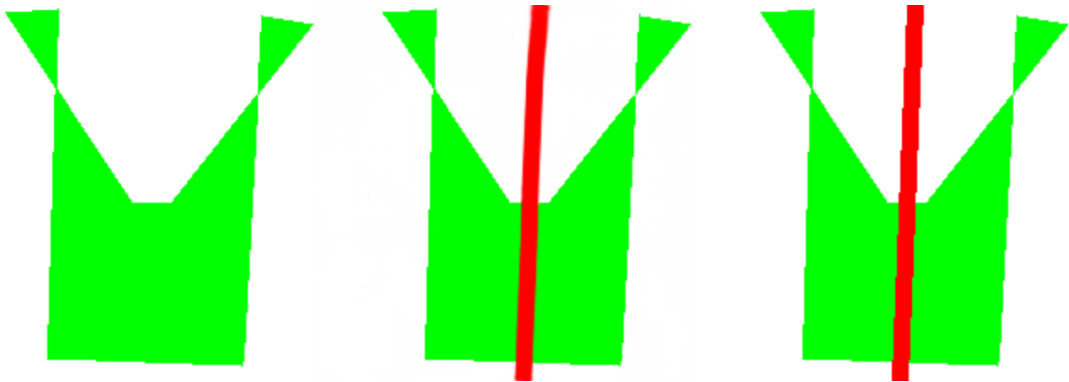


Training results of variable refractive index (I).

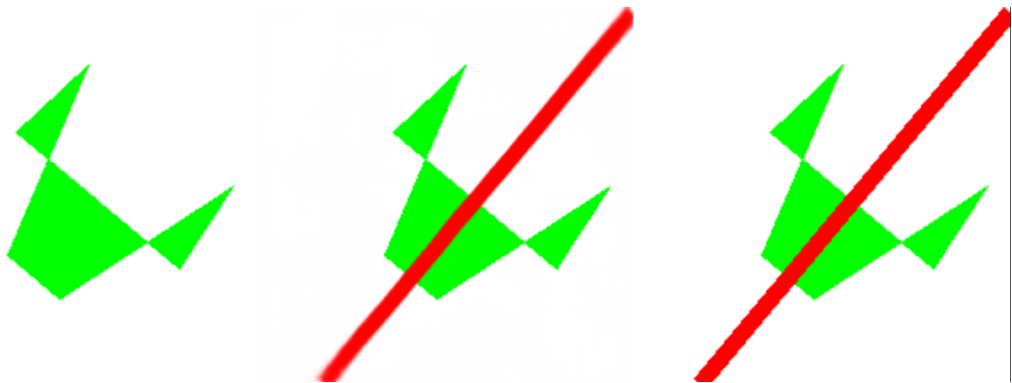


Training results of variable refractive index (II).

- **Symmetry Axis Learning**

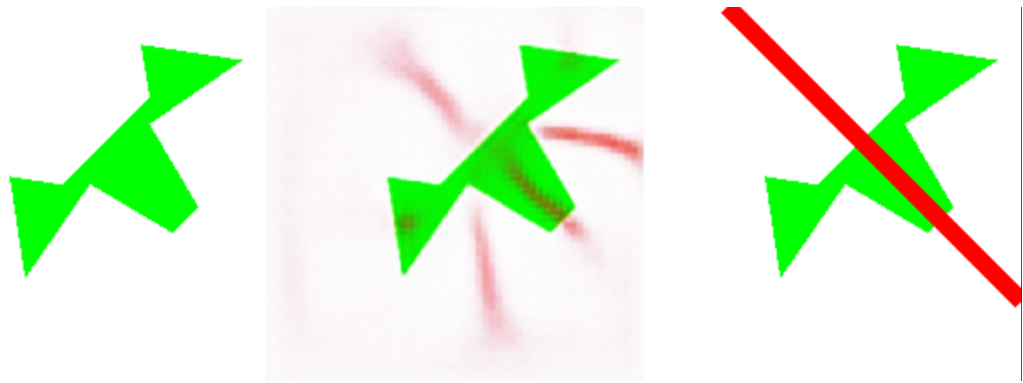


Symmetry axis learning (I).

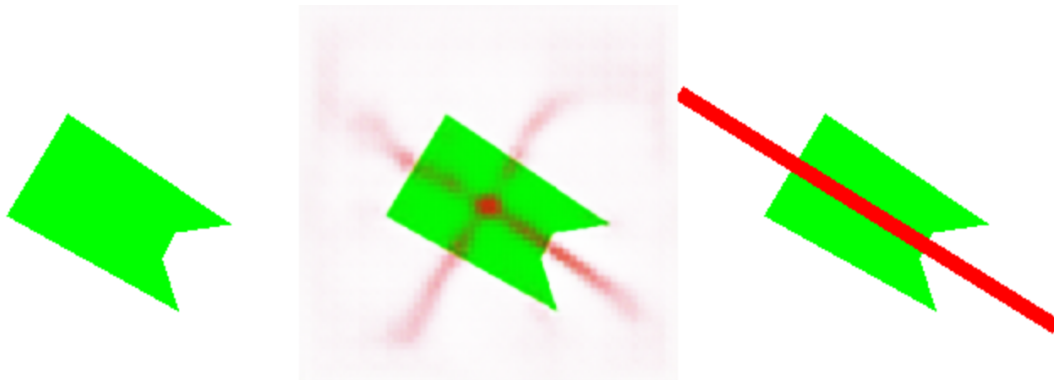


Symmetry axis learning (II).

Learning Intermediate States

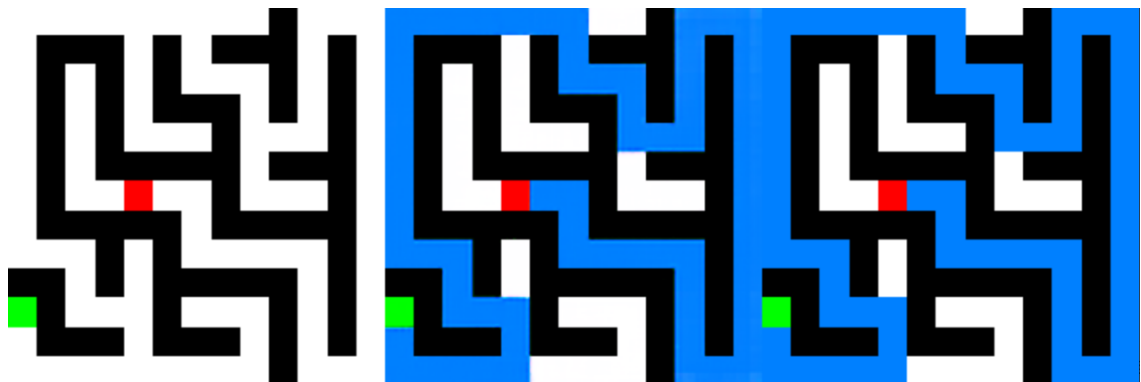


Learning intermediate states (I).

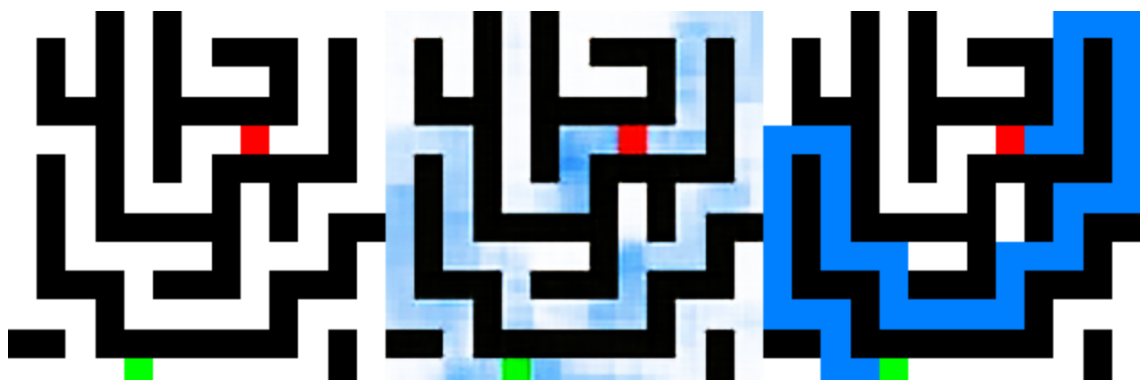


Maze experiment.

- Learning Intermediate States



Learning intermediate states (II).



B. List of All Experiments Conducted

B.1 Symbolic Math Logic

- `generate_conditional_add_subtract.py`: Generates addition or subtraction (absolute value) problems for two N-bit integers. Contains two modes.
- `generate_add_binary_modulo.py`: An early basic arithmetic experiment to test the model’s ability to learn modular addition (or “truncated addition”), common in fixed-width integer arithmetic in computer hardware.
- `generate_multiply_binary.py`: A benchmark test for binary arithmetic capability, generating N-bit integer multiplication datasets.
- `generate_multiply_binary_no_carry_phase1.py`: Phase 1 of the multiplication “decoupling” experiment. Aims to test if the model can learn the first step of multiplication: bitwise multiplication and shifted addition without carries, decomposing a complex multiplication problem into a simpler counting problem.
- `generate_multiply_binary_from_counts_phase2.py`: Phase 2 of the multiplication “decoupling” experiment. Aims to verify if an independent model can learn to handle complex carry logic, calculating the final binary product from a “carry-free count vector.”
- `generate_add_hexadecimal.py`: Compares the model’s learning capability under different symbol systems. This script aims to verify if the model learns the abstract mathematical concept of addition or just patterns specific to binary symbols.
- `generate_multiply_decimal.py`: Tests the model’s ability to process non-binary symbol inputs (0-9 characters) and perform arithmetic operations (multiplication).
- `generate_add_binary_with_position_shuffle.py`: The “position shuffle” part of the “semantic shuffling” series of experiments. Aims to verify if the model relies on the fixed spatial structure of imports or can learn abstract relationships independent of position.
- `generate_symbol_add_shuffle_dataset.py`: A key decisive experiment in my research, aiming to thoroughly separate the model’s “superficial pattern matching” capability from its “abstract structure learning” capability.
- `generate_add_hidden_constant.py`: Tests the model’s ability to infer hidden rules or parameters from large numbers of samples without any direct clues. This is similar to a simplified System Identification problem.
- `generate_multitask_alu.py`: Aims to build a multi-task learning scenario simulating an Arithmetic Logic Unit (ALU). Tests if the model can execute multiple different, well-defined computing tasks in parallel on the same input in a single forward pass.
- `generate_modulo_operation.py`: Explores the model’s ability to learn Modulo Operation, a crucial but “cyclic” operation in number theory and computer science.
- `generate_rsa_encryption.py`: Tests the model’s ability to learn highly non-linear deterministic rules considered “hard” computationally. RSA encryption is a typical example.
- `generate_deduction_chain_text.py`: Generates multi-step logical reasoning tasks, testing the model’s ability to perform symbolic deduction, similar to a simplified theorem prover.
- `generate_deduction_multirule_text.py`: Tests whether the model can correctly “route” to the corresponding rule based on a Query and make judgments when facing multiple independent, unrelated rules.
- `generate_deduction_multirule_text_v2.py`: Tests whether the model can correctly “route” to the corresponding rule based on a Query and make judgments when facing multiple independent, unrelated rules.
- `generate_deduction_multirule_binary.py`: An optimized format version of the multi-rule reasoning task, aiming to test if compact binary coding is more conducive to model learning than sparse text formats.

- `generate_deduction_fixed_depth.py`: Tests the model’s multi-step reasoning capability in symbolic deduction tasks with clear structure and fixed depth.
- `generate_function_composition.py`: Tests the model’s capability to learn Function Composition. Requires the model to parse instructions sequentially and transform data like an interpreter.
- `generate_count_set_bits.py`: Tests the model’s capability to execute global aggregation operations. Unlike local rules, counting requires the model to synthesize information from the entire input sequence.
- `generate_sum_pattern_positions.py`: Tests the model’s capability to execute more complex, grouped parallel aggregation tasks. The model needs to segment the input, classify each segmented pattern, and finally accumulate position information for patterns belonging to the same class.
- `generate_sum_pattern_positions_v2.py`: Tests the model’s capability to execute more complex, grouped parallel aggregation tasks. The model needs to segment the input, classify each segmented pattern, and finally accumulate position information for patterns belonging to the same class.
- `generate_sum_pairwise_hamming_distance.py`: Tests the model’s capability to execute a complex task requiring two layers of nested aggregation operations. The model needs to perform global statistics on each bit first, and then accumulate results from all bits.
- `generate_circular_shift.py`: Tests the model’s capability to learn bitwise shift operations, especially circular shift, a common operation in cryptography and low-level programming.
- `generate_multiply_matrix_3x3.py`: Tests the model’s capability to learn structured algebraic operations (matrix multiplication), which requires more complex “data routing” and “multiply-accumulate” capabilities than simple scalar operations.
- `generate_evaluate_boolean_expression_text.py`: Tests the model’s capability to parse a simple Domain Specific Language (DSL) and execute evaluation, going further than fixed-structure expression evaluation.
- `generate_evaluate_arithmetic_expression.py`: Trains the model to execute symbolic expression evaluation tasks, requiring the model to understand operator precedence (implicitly expressed through tree structures), variable substitution, and arithmetic operations.
- `generate_evaluate_arithmetic_expression_no_multiply.py`: A simplified version of `generate_evaluate_arithmetic_expression.py`, aiming to reduce learning difficulty by removing multiplication to test the model’s capability on more basic arithmetic expression evaluation.
- `generate_evaluate_arithmetic_expression_no_multiply_small_range.py`: A further simplification based on the previous “no multiplication” version, further reducing learning difficulty by creating a smaller numerical range, used to precisely diagnose the model’s performance bottleneck on the simplest expression evaluation tasks.
- `generate_check_boolean_equivalence.py`: Tests the model’s judgment capability on Boolean algebra logical equivalence. This is an abstract symbolic reasoning task requiring the model to understand expression structure and Boolean operation laws.
- `generate_polynomial_shift_coefficients.py`: Tests the model’s capability to learn an abstract algebraic transformation rule. This task requires the model to understand the intrinsic structure of polynomial expansion.
- `generate_convolution_2d.py`: Tests the model’s capability to learn 2D convolution (Conv2D), a basic image processing operation, and explores whether it can infer hidden fixed rules (i.e., the convolution kernel itself) from input-output pairs.
- `generate_simple_block_cipher.py`: Tests the model’s capability to “crack” or learn a simple but non-trivial custom encryption algorithm. This task represents a class of complex symbolic transformation rules with high chaos and avalanche effects.
- `generate_sin_function_float32.py`: Tests the model’s capability to fit continuous, periodic, non-linear functions ($\sin(x)$) using standard 32-bit floating-point format for input and output.

- `generate_sin_function_float64_to_int12_deprecated.py`: Another encoding attempt for the sin function fitting task, aiming to explore the impact of using higher precision floating-point input and lower precision quantized binary output on learning effects.
- `generate_sin_function_float32_to_quantized_int.py`: Tests the model’s capability to fit continuous, periodic, non-linear functions ($\sin(x)$) and explores the impact of different input/output encoding schemes on learning effects.
- `generate_multiply_binary_modulo.py`: Part of the basic arithmetic experiments, tests the model’s mastery of truncated multiplication (or modular multiplication).
- `generate_explainable_two_step_calculation.py`: Tests the model’s capability to output calculation “intermediate steps” or “chain of thought,” a direct verification of “functional interpretability.”
- `generate_min_swaps_for_checkerboard.py`: Solves LeetCode 782 “Transform to Chessboard” - <https://leetcode.cn/problems/transform-to-chessboard/> - Minimum swaps of rows and columns to transform a 0/1 matrix into a “chessboard” pattern.
- `generate_min_flips_for_alternating_binary.py`: Tests the model solving a string optimization problem based on bit flips, which can be ingeniously mapped to a sliding window problem.
- `generate_min_swaps_for_checkerboard_v2.py`: Solves LeetCode 1536 “Minimum Swaps to Arrange a Binary Grid” - <https://leetcode.cn/problems/minimum-swaps-to-arrange-a-binary-grid/> - Minimum swaps of adjacent rows to transform a binary grid into upper triangular form.
- `generate_min_prefix_flips.py`: Tests the model’s capability to learn a greedy algorithm that depends on historical states and processes sequentially.
- `generate_min_flips_for_chunked_binary.py`: Tests the model’s capability to learn a string transformation optimization problem based on local chunks.
- `generate_largest_island_by_adding_one_cell.py`: Solves an algorithm problem involving graph traversal and global optimization ([LeetCode 827. Making A Large Island](#)). Model needs to evaluate all possible “land reclamation” positions.
- `generate_largest_island_by_adding_one_cell_v2.py`: Solves an algorithm problem involving graph traversal and global optimization ([LeetCode 827. Making A Large Island](#)). Model needs to evaluate all possible “land reclamation” positions.
- `generate_sat_solver_text.py`: Tests the model’s capability to solve a landmark NP-complete problem — Boolean Satisfiability (SAT).
- `generate_sat_solver_compact_text.py`: A variant of `generate_sat_solver_text.py` using different input encoding formats to solve the same 3-SAT problem.
- `generate_binary_mod3_dfa_explain.py`: An interpretability experiment aiming to test if the model can learn and output the internal state transition process of a Deterministic Finite Automaton (DFA).
- `generate_add_binary_explainable.py`: Tests if the model can learn the internal mechanism of binary addition (carry propagation). Requires outputting not just results but the carry state of each bit.
- `generate_add_quaternary_float_encoding.py`: Tests if MLP has the capability to parse real numbers, map them to encoded binary, and complete calculations. Proves that real numbers spaced at certain intervals can be used as pure symbols when training MLPs.
- `generate_add_binary_masked_output.py`: Tests the model’s capability on “conditional output” tasks. Requires the model to not only calculate addition but also dynamically decide which bits of the result to output based on input Mask. Similar to Attention mechanism.
- `generate_multitask_prefixed_ca110.py`: Generates a “Prefixed CA Evolution + 4 Post-tasks” multitask learning dataset to test the model’s capability to execute shared representation extraction and multiple independent downstream tasks in a single forward pass. An important extension to `generate_multitask_alu.py`.

- `generate_convergent_abstraction.py`: Generates a “Convergent Abstraction” (Multi-input -> Single Intermediate Representation Hub) experiment dataset. Aims to verify the key hypothesis in Act VIII: Can different input tasks naturally converge to a shared abstract intermediate representation and complete subsequent reasoning tasks based on it?
- `generate_multiply_binary_showdown.py`: Three-algorithm showdown for binary multiplication. Extends the idea of `generate_rain_water_final_showdown.py` to multiplication tasks, comparing three decoupling methods.

B.2 Algorithm Learning

- `generate_sort_integers.py`: Tests the capability to execute basic sorting algorithms, a classic non-local algorithm task requiring comparison and rearrangement of input elements.
- `generate_edit_distance.py`: Tests the capability to learn Dynamic Programming problems. Edit distance is a typical DP problem requiring the conceptual construction of a 2D solution matrix. [LeetCode 72. Edit Distance](#)
- `generate_edit_distance_explainable.py`: A core experiment for “functional interpretability.” Requires outputting the complete “chain of thought” (editing process) along with the final answer. [LeetCode 72. Edit Distance \(Explainable / Path Construction Version\)](#)
- `generate_maze_random_walls.py`: Tests basic wayfinding capability in randomly generated “porous” mazes.
- `generate_maze_dense.py`: Tests path planning capability in complex, “dense” mazes similar to human designs, more challenging than random wall mazes.
- `generate_blocks_world_arbitrary_goal.py`: Solves the classic “Blocks World” planning problem. As discussed in the Apple Research paper [15], this is a benchmark for evaluating reasoning capabilities in large language models. This script implements the general version allowing arbitrary initial and goal states.
- `generate_blocks_world_fixed_goal.py`: A simplified version of “Blocks World” with fixed goal states, aiming to test learning capability under clearer goals and more structured state spaces.
- `generate_blocks_world_fixed_goal_multilabel.py`: Improves “Blocks World” by allowing multiple optimal solutions, testing the capability to handle multi-label classification, reflecting equivalent optimal paths in planning problems.
- `generate_blocks_world_fixed_goal_multilabel_fixed_format.py`: The final optimized version of “Blocks World,” improving input representation with fixed-slot format to provide a clearer, more structured learning target.
- `generate_checkers_jump_1d.py`: Solves the 1D checkers jump planning problem. A core control experiment for verifying reasoning capability in complex planning problems, as discussed in [15].
- `generate_river_crossing_puzzle.py`: Solves the classic “River Crossing” constraint satisfaction and state space search problem. A benchmark for testing symbolic reasoning limitations in LLMs.
- `generate_trapping_rain_water_aggregate.py`: Initial attempt at “Trapping Rain Water” with aggregate output. Proved much harder to learn than the decoupled version, demonstrating the systematic impact of output format design. Corresponding LeetCode problem: [42. Trapping Rain Water](#).
- `generate_trapping_rain_water_decoupled.py`: Solves LeetCode Hard [42. Trapping Rain Water](#). Success demonstrates the capability to learn complex algorithms requiring global information via problem decoupling.
- `generate_trapping_rain_water_2d.py`: Extension to 2D Trapping Rain Water. Requires understanding “enclosure” and “boundary” in 2D space. LeetCode [407. Trapping Rain Water II](#).
- `generate_skyline_max_height_aggregate.py`: Initial attempt at the “Skyline” problem with aggregate output. Used to compare learning difficulty with decoupled output. LeetCode [1840. Maximum Building Height](#).

- `generate_skyline_all_heights_decoupled.py`: Tests capability to solve global optimization problems with 1D spatial constraints. LeetCode [1840. Maximum Building Height](#) (Decoupled version).
- `generate_hanoi_tower_path_strategy_sep_format.py`: Control Group A for Tower of Hanoi, using separator + binary coding input format.
- `generate_hanoi_tower_path_strategy_fixed_format.py`: Control Group B for Tower of Hanoi, using structured fixed-slot input format to verify the impact of data representation.
- `generate_hanoi_tower_compare_formats.py`: Comparative experiment script generating two input formats for the same Hanoi problem.
- `generate_hanoi_tower_compare_formats_and_strategies.py`: Comprehensive comparative script generating two input formats and two strategy datasets (path strategy vs. global strategy).
- `generate_hanoi_tower_build_full_state_graph.py`: A comprehensive tool for “Tower of Hanoi” research, building a full state graph to deeply analyze recursive structure understanding.
- `generate_hanoi_tower_sample_from_state_graph.py`: Sampling script using the full state graph to extract specific training data subsets like “twisted paths” for fine-grained ablation studies.
- `generate_hanoi_decoupled_scaffold.py`: An experimental script for the Tower of Hanoi problem using decoupled auxiliary labels. It transforms complex recursive logic into local state-transition tasks by generating intermediate goals (Scaffold) for each disk, solving the convergence issue in the full state space.
- `generate_sokoban_planning_astar.py`: Solves classic “Sokoban” planning problem. Harder than simple path planning due to state changes.
- `generate_sokoban_planning_full.py`: Solves “Sokoban” planning. A high-difficulty AI task involving search in a huge state space.
- `generate_sokoban_planning_claude_deprecated.py`: (Deprecated) Early attempt.
- `generate_matrix_flip_strategy.py`: Solves matrix optimization (Score After Flipping Matrix). Tests capability to learn a “strategy” rather than final result. LeetCode [861. Score After Flipping Matrix](#).
- `generate_matrix_flip_max_score.py`: Tests capability to learn matrix optimization requiring a two-step greedy strategy, outputting aggregate score. LeetCode [861. Score After Flipping Matrix](#).
- `generate_min_k_bit_flips.py`: Tests capability to learn a greedy algorithm dependent on historical states and sequential processing using variable ‘k’. LeetCode [995. Minimum Number of K Consecutive Bit Flips](#).
- `generate_min_k_bit_flips_fixed_k.py`: Same as above but with fixed/hidden ‘k’. Control experiment. LeetCode [995. Minimum Number of K Consecutive Bit Flips](#).
- `generate_special_binary_string_recursion.py`: Tests capability to learn a recursively defined string transformation rule. LeetCode [761. Special Binary String](#).
- `generate_count_connected_components.py`: Tests basic understanding of graph structure, specifically connectivity. LeetCode [323. Number of Connected Components in an Undirected Graph](#) (Note: Premium problem).
- `generate_check_graph_connectivity.py`: Tests judgment of path existence between two points in a graph. LeetCode [1971. Find if Path Exists in Graph](#).
- `generate_minimize_malware_spread.py`: Solves graph-based virus spread optimization. [924. Minimize Malware Spread](#). Requires understanding connectivity and impact of node removal.
- `generate_count_islands_1d.py`: Tests pattern recognition and counting on 1D sequences.
- `generate_find_articulation_points.py`: Tests identification of articulation points/bridges in graphs. [LeetCode 1568. Minimum Number of Days to Disconnect Island](#).
- `generate_nim_game_zeckendorf.py`: Tests ability to learn a non-intuitive game theory problem based on complex number theory (Zeckendorf representation).

- `generate_longest_subsequence_constrained.py`: Tests processing of complex optimization combining sequence operations and numerical constraints. [LeetCode 2311. Longest Binary Subsequence Less Than or Equal to K](#).
- `generate_treasure_hunt_tsp.py`: Solves complex state space search combining BFS and status compression DP (TSP variant).
- `generate_freedom_trail_dp.py`: [LeetCode 514. Freedom Trail](#). Tests capability to solve complex optimization requiring DP and path backtracking.
- `generate_sum_of_subset_with_mask.py`: Tests aggregation (sum) of elements selected by a binary mask. Verifies impact of task structure itself.
- `generate_sudoku_6x6.py`: Tests capability on strong Constraint Satisfaction Problems (Sudoku).
- `generate_valid_parentheses_path_random_deprecated.py`: (Deprecated) Early attempt at valid parentheses path.
- `generate_valid_parentheses_path_balanced.py`: Pathfinding on 2D grid constrained by stack structure (parentheses matching). [LeetCode Hard Check if There Is a Valid Parentheses String Path](#).
- `generate_point_in_polygon.py`: Tests learning of Ray Casting Algorithm in computational geometry.
- `generate_shortest_path_in_matrix_bfs.py`: Tests finding shortest path in 2D grid based on BFS.
- `generate_sudoku_4x4_stepwise_deprecated.py`: (Deprecated) Tests stepwise reasoning capability.
- `generate_tiling_problem_deprecated.py`: (Deprecated) Tests tiling cover optimization.
- `generate_hanoi_tower_twisted_path_deprecated.py`: (Deprecated) Intended to generate “twisted path” dataset for Hanoi.
- `generate_checkers_jump_1d_v2.py`: Sequence learning comparative experiment for checkers jump, generating multiple types of complete path datasets.
- `generate_maze_symbolic_to_image.py`: Converts symbolic maze datasets to image format for visual model testing.
- `generate_trapping_rain_water_visualizer.py`: Converts symbolic “Trapping Rain Water” dataset to image-to-image format for visual models.
- `generate_shortest_path_in_tree_deprecated.py`: (Deprecated) Early experiment on shortest path in trees from images.
- `generate_rain_water_final_showdown.py`: Algorithm comparison experiment providing DP, Monotonic Stack, and Two Pointers intermediate processes as auxiliary labels.
- `generate_bricks_falling.py`: [LeetCode 803. Bricks Falling When Hit](#). Tests understanding of physical stability and connectivity.
- `generate_maximal_rectangle_coords.py`: [LeetCode 85. Maximal Rectangle](#).
- `generate_median_island_median_finder.py`: Complex symbolic reasoning combining sequence segmentation, median calculation, and positioning.
- `generate_rain_water_summation.py`: Subtask B of “Trapping Rain Water.” Tests pure summation capability given pre-calculated water amounts.
- `generate_rain_water_then_cellular_automata.py`: Cross-domain chain reasoning Task D. Tests executing “Trapping Rain Water” then using results as initial state for “Cellular Automata.”
- `generate_maze_decoupled.py`: Decoupled version of dense maze, outputting full BFS distance map as explanation.
- `generate_edit_nextstep_mlp.py`: Decoupled experiment for Edit Distance focusing on “next optimal operation” with operation masks.
- `generate_edit_distance_dp_table_full.py`: Full DP table decoupling experiment for Edit Distance.

- `generate_median_one_finder.py`: “Median ‘1’ Locator” dataset. Basic statistical positioning.
- `generate_mean_of_medians.py`: “Mean of Medians of Odd/Even Isolated Islands” dataset. Complex logic task known as “Ninefold Purgatory.”
- `generate_exam_seats.py`: “Exam Room Arrangement” dataset. ([Maximum Students Taking Exam - LeetCode](#)). Tests status compression DP on 2D grids.
- `generate_parity_accumulator.py`: “Parity Instruction Accumulator” task. Complex logic with sequential dependency and conditional arithmetic.
- `generate_neural_turing_machine.py`: Simulates conditional write operations on a cyclic tape like a simplified NTM. Direct test of algorithmic operation learning.
- `generate_lights_out_forward.py`: “Lights Out” game forward simulation. Tests local rule propagation.
- `generate_lights_out_gaussian_elimination.py`: “Lights Out” inverse problem modeled as linear equation solving with Gaussian elimination decoupling. Proves capability to learn complex inverse problems with appropriate decoupling.

B.3 Visual Reasoning

- `generate_checkerboard_to_binary.py`: Basic visual-to-symbol conversion test.
- `generate_line_angle_to_vector.py`: Tests extracting precise geometric info (angles) from images.
- `generate_count_shapes_from_image.py`: Tests multi-visual task capability: object recognition (shape), attribute recognition (color), counting.
- `generate_sokoban_symbolic_to_image_no_labels.py`: Converts symbolic Sokoban to image-only format.
- `generate_sokoban_symbolic_to_image_with_labels.py`: Converts symbolic Sokoban to image classification dataset for CV models.
- `generate_triangle_to_incircle.py`: Landmark experiment for “sculpting precise rules with gradient descent.” Triangle Incircle.
- `generate_polygon_to_symmetry_axis.py`: Tests inferring implied symmetry axis from symmetric figures.
- `generate_triangle_to_centroid.py`: Tests learning centroids.
- `generate_triangle_to_tessellation.py`: Landmark display of paradigm capability. Tests infinite lattice-based generation rules (Tessellation) to rule out interpolation/memorization.
- `generate_shortest_distance_between_triangles.py`: Tests global geometric relationship reasoning (shortest distance).
- `generate_coords_to_triangle.py`: Basic symbol-to-geometry rendering (coordinates to pixels).
- `generate_triangle_coords_to_tessellation.py`: Advanced reasoning mixing symbolic instructions and geometric generation rules.
- `generate_trapping_rain_water_image_to_symbol.py`: Image-to-symbol conversion testing if CV models can extract precise physical quantities.

B.4 Cellular Automata

- `generate_cellular_automata_1d.py`: 1D CA evolution dataset logic learning test.
- `generate_game_of_life_2d.py`: 2D CA (Conway’s Game of Life) dataset. More complex spatial relations.
- `generate_cellular_automata_1d_multistate.py`: 1D CA extension to non-binary states.
- `generate_cellular_automata_programmable.py`: Tests “programmability” or “meta-learning.” Execution based on rules given in input.

- `generate_cellular_automata_inverse_rule90.py`: Tests “Inverse Problem” solving. Inferring input from output under constraints.
- `generate_game_of_life_image_to_image.py`: 2D CA image-to-image version.
- `generate_cellular_automata_spatial_conditional.py`: Tests partitioning and parsing “instruction” and “data” within a single image modality.
- `generate_cellular_automata_multimodal_deprecated.py`: (Deprecated) True multimodal (image+text) dataset.
- `generate_cellular_automata_1d_to_grid_image_interp.py`: Logic/Perception hybrid task testing unified rule learning and interpolation.
- `generate_cellular_automata_1d_to_grid_image.py`: Tests “rendering” 1D symbolic results to 2D structured images.
- `generate_cellular_automata_inverse_rule.py`: First attempt at Inverse Reasoning (inferring rule from input-output).
- `generate_cellular_automata_inverse_rule_and_steps.py`: Early version inferring rule and steps.
- `generate_cellular_automata_inverse_rule_and_steps_unique.py`: Major upgrade to unique solution inverse reasoning. Inferring applied rule and number of applications.
- `generate_cellular_automata_1d_perturbed.py`: Systematically tests robustness against imperfect data (input/output noise).
- `generate_ca110_full_trace.py`: Deep analysis experiment generating full evolution traces.
- `generate_ca_text_format_dataset.py`: Autoregressive experiment testing Decoder-only Transformers (e.g., GPT) on CA rules via text prompts.
- `generate_cellular_automata_image_and_label.py`: General dataset generator for image format and symbol format.
- `generate_mnist_ca_110.py`: Perception and Reasoning fusion. MNIST digits as CA cells.
- `generate_rulemnist_ca.py`: Advanced Perception and Reasoning fusion with rule control.
- `generate_ca_continuous_transform.py`: Continuous-Discrete-Continuous hybrid CA experiment.
- `generate_ca_nested_masked.py`: “Matryoshka” nested mask CA dataset testing multi-stage implicit reasoning under dynamic masks.
- `generate_ca_masked_output.py`: “Partially Observable” CA dataset.
- `generate_ca_to_abstract_real.py`: Tests encoding discrete evolution results into arbitrary real number symbol representations.
- `generate_ca_rule110_minimalist_trace.py`: Minimalist text explanation format for autoregressive models (CoT style).
- `generate_ca_multirule_cot_trace.py`: Extension with multi-rule support for CoT trace.
- `generate_ca_ood_hallucination_split.py`: CA OOD generalization test dataset (split by rules).
- `generate_hierarchical_ca_tree.py`: Hierarchical abstract CA tree dataset generator. Verifies emergence of intermediate representations.

B.5 Physics Simulation

- `generate_projectile_motion_simulation.py`: Tests learning simple dynamic physical processes (projectile motion).
- `generate_snell_refraction_simulation.py`: Tests learning Snell’s Law.
- `generate_snell_refraction_with_contextual_index.py`: Tests learning Snell’s Law with inferred physical parameters (refractive index) from context.
- `generate_reaction_diffusion_deprecated.py`: (Deprecated) Gray-Scott reaction-diffusion simulation. Continuous field simulation.
- `generate_cube_rotation_matplotlib_deprecated.py`: (Early version) 3D object rotation from abstract pose parameters.

- `generate_cube_rotation_pillow_v1.py`: (Technical upgrade) 3D rotation with better rendering.
- `generate_cube_rotation_pillow_with_anchor.py`: (Final version) 3D rotation with visual anchors.
- `generate_cube_rotation_pillow_wireframe.py`: (Variant) 3D rotation with wireframes and sparse input.
- `generate_catenary_curve_simulation_deprecated.py`: Early exploration of Catenary (non-linear curve from physical laws).
- `generate_catenary_curve_from_points.py`: Tests learning Catenary curve uniquely determined by minimum potential energy.
- `generate_orbital_path_from_initial_state.py`: Tests learning Kepler’s laws/Law of Universal Gravitation.

B.6 ARC-AGI Exploration

- `generate_arc_contextual_color_swap.py`: Tests learning rules from local context/examples applied globally. Imitates ARC-AGI core concept.
- `generate_arc_find_cross_pattern.py`: Visual pattern recognition (object detection) in noise.
- `generate_arc_find_odd_one_out.py`: “Find the Odd One Out” meta-reasoning task.
- `generate_arc_connect_colored_pairs.py`: Identifies multiple “connection tasks” and implicit layer/priority rules.
- `generate_arc_conditional_perpendicular_lines.py`: Geometric operations conditional on object attributes and global references.
- `generate_arc_column_projection.py`: Recognizes complex context relations (“below... and within range...”) for conditional column operations.
- `generate_arc_procedural_spiral.py`: Executes iterative, procedural generation algorithms (spirals).
- `generate_arc_fractal_stamping.py`: Recursive/fractal generation rules using input as “brush.”
- `generate_arc_flood_fill.py`: Classic Flood Fill algorithm. [ARC Prize - Puzzle #36a08778](#)
- `generate_arc_layered_fill.py`: Complex filling algorithm dependent on topological distance and conditional judgment.
- `generate_arc_fluid_simulation.py`: Simulates fluid dynamic process with specific rules in image space.
- `generate_arc_periodic_conditional_fill.py`: Complex conditional formatting rule with periodicity and special cases.
- `generate_arc_fill_square_holes.py`: Multi-step visual reasoning: identifying “foreground in background” (holes) -> geometric attribute judgment -> coloring.
- `generate_arc_conditional_recoloring.py`: Understands visual layers and conditional object attribute modification.
- `generate_arc_sort_by_length_remap_position.py`: Executes “Attribute-Position Decoupling and Remapping” complex sorting task.
- `generate_arc_jigsaw_puzzle_simple.py`: Visual matching and transformation (Simple). [ARC Prize - Puzzle #898e7135](#)
- `generate_arc_jigsaw_puzzle_advanced.py`: Visual matching and transformation (Advanced). [ARC Prize - Puzzle #898e7135](#)
- `generate_arc_connect_path_by_sequence.py`: Parses external instruction sequences to execute multi-step, stateful path connection in images.
- `generate_arc_reflection_simulation_deprecated.py`: (Deprecated) Tests understanding of complex physical optics rules (reflection, collision, color change).